



So ICT Do an tot nghiep Big Data

Advanced Topic in Technology (Hanoi University of Science)



Scan to open on Studocu

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống lưu trữ, xử lý
và phân tích dữ liệu sàn thương mại điện tử**

NGUYỄN QUANG LINH

linh.nq194092@sis.hust.edu.vn

Ngành Khoa học máy tính

Giảng viên hướng dẫn: TS. Trần Việt Trung

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 07/2022

LỜI CAM KẾT

Họ và tên sinh viên: Nguyễn Quang Linh
Điện thoại liên lạc: 0943523278
Email: linh.nq194092@sis.hust.edu.vn
Lớp: Khoa học máy tính 05 - K64
Hệ đào tạo: Cử nhân

Tôi – *Nguyễn Quang Linh* – cam kết Đồ án Tốt nghiệp (ĐATN) là công trình nghiên cứu của bản thân tôi dưới sự hướng dẫn của *TS. Trần Việt Trung*. Các kết quả nêu trong ĐATN là trung thực, là thành quả của riêng tôi, không sao chép theo bất kỳ công trình nào khác. Tất cả những tham khảo trong ĐATN – bao gồm hình ảnh, bảng biểu, số liệu, và các câu từ trích dẫn – đều được ghi rõ ràng và đầy đủ nguồn gốc trong danh mục tài liệu tham khảo. Tôi xin hoàn toàn chịu trách nhiệm với dù chỉ một sao chép vi phạm quy chế của nhà trường.

Hà Nội, ngày tháng năm 2023

Tác giả ĐATN

Nguyễn Quang Linh

LỜI CẢM ƠN

Tôi xin gửi lời cảm ơn chân thành đến bố mẹ và gia đình yêu thương của tôi đã luôn đồng hành, quan tâm, động viên và hỗ trợ vô điều kiện trong suốt thời gian học tập và hoàn thành đồ án tốt nghiệp này.

Tôi gửi lời cảm ơn sâu sắc đến thầy TS. Trần Việt Trung, người đã dành thời gian và công sức hướng dẫn tôi trong quá trình làm đồ án. Sự kiên nhẫn và tận tâm của thầy giúp tôi vượt qua khó khăn, khám phá và phát triển khả năng của mình.

Cảm ơn bạn bè đồng hành trong học tập, đã chia sẻ kiến thức và tạo môi trường tích cực. Sự hỗ trợ và đồng lòng của các bạn giúp tôi tiến bước.

Cuối cùng, xin gửi lời cảm ơn đặc biệt tới Trường Công nghệ thông tin và Truyền thông - Đại học Bách Khoa Hà Nội, đã tạo điều kiện thuận lợi và cung cấp kiến thức chất lượng để tôi phát triển và hoàn thành đồ án tốt nghiệp.

Tôi biết ơn và tự hào về chặng đường đã qua, những người đã đồng hành và ủng hộ tôi. Mọi đóng góp và sự hỗ trợ từ mọi người là nguồn động lực quý giá để tôi vượt qua mọi khó khăn và đạt thành tích tốt nhất trong đồ án tốt nghiệp.

Chân thành cảm ơn mọi người!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Cuộc cách mạng công nghiệp 4.0 đang diễn ra với sự phát triển mạnh mẽ của công nghệ thông tin và truyền thông. Điều này đã dẫn đến một lượng dữ liệu khổng lồ được tạo ra hằng ngày, gọi là dữ liệu lớn - Big Data. Dữ liệu lớn mang lại tiềm năng to lớn cho các doanh nghiệp và tổ chức khi có thể tận dụng thông tin từ dữ liệu để đưa ra quyết định thông minh, cải thiện hiệu suất và tạo ra giá trị mới.

Sàn thương mại điện tử là một môi trường tạo ra rất nhiều dữ liệu, có tính chất phức tạp và đa dạng. Khối lượng dữ liệu ngày càng tăng lên do sự gia tăng của số lượng người dùng và hoạt động trên sàn thương mại điện tử. Điều này tạo ra một thách thức lớn trong việc quản lý, xử lý và tận dụng hiệu quả thông tin từ dữ liệu.

Với mục tiêu giải quyết vấn đề này, tôi đã lựa chọn hướng tiếp cận cho đồ án tốt nghiệp là xây dựng một hệ thống lưu trữ dữ liệu lớn dựa trên công nghệ phân tán. Mô hình hệ thống của tôi sẽ bao gồm các thành phần như máy chủ xử lý, cơ sở dữ liệu phân tán và hệ thống xử lý dữ liệu song song. Tôi đã chọn hướng này vì tính linh hoạt và khả năng mở rộng của nó, cho phép xử lý dữ liệu lớn một cách hiệu quả và nhanh chóng.

Tổng quan giải pháp của tôi là xây dựng một quy trình xử lý dữ liệu bao gồm thu thập dữ liệu từ sàn thương mại điện tử, lưu trữ dữ liệu vào hệ thống phân tán và tiến hành các bước xử lý dữ liệu như tiền xử lý, phân tích và trực quan hóa. Sau đó đi đến một số bài toán về phân tích dữ liệu sàn thương mại điện tử. Để đạt được điều này, tôi sẽ áp dụng các kỹ thuật của học máy và khai phá dữ liệu.

Đồ án tốt nghiệp của tôi đóng góp chính là việc phát triển một hệ thống lưu trữ, xử lý và phân tích dữ liệu sàn thương mại điện tử. Bằng cách áp dụng giải pháp đã đề xuất, tôi hy vọng kết quả cuối cùng có thể cải thiện hiệu suất xử lý dữ liệu, giúp người dùng dễ dàng phân tích và trực quan hóa thông tin từ dữ liệu thương mại điện tử. Đồng thời cung cấp một hệ thống linh hoạt để phân loại bình luận thành tích cực, tiêu cực hoặc trung tính, cũng như gợi ý sản phẩm phù hợp cho người dùng.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.2.1 Mục tiêu.....	2
1.2.2 Phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	2
1.3.1 Phương pháp nghiên cứu.....	2
1.3.2 Phương pháp thực hành	3
1.4 Đối tượng và định hướng sử dụng.....	4
1.5 Bố cục đồ án	4
CHƯƠNG 2. DỮ LIỆU LỚN VÀ SÀN THƯƠNG MẠI ĐIỆN TỬ	6
2.1 Giới thiệu về Dữ liệu lớn	6
2.2 Dữ liệu lớn trong Sàn thương mại điện tử.....	7
2.3 Tiki - Tìm kiếm và Tiết kiệm	9
2.4 Kết luận	10
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	11
3.1 Công nghệ lưu trữ dữ liệu	11
3.1.1 Apache Kafka.....	11
3.1.2 Apache HBase	13
3.1.3 Elasticsearch.....	18
3.2 Công nghệ cho bài toán phân tích dữ liệu - Apache Spark.....	19
3.3 Công nghệ trực quan hóa dữ liệu - Kibana	21
3.4 Công nghệ giám sát hệ thống	22
3.4.1 Prometheus.....	22

3.4.2 Grafana	23
3.5 Kết luận	24
CHƯƠNG 4. KIẾN TRÚC HỆ THỐNG	25
4.1 Tổng quan hệ thống.....	25
4.2 Lớp thu thập dữ liệu	26
4.2.1 Nguồn dữ liệu.....	26
4.2.2 Phương pháp thu thập dữ liệu	27
4.2.3 Quy trình thu thập dữ liệu	29
4.3 Lớp xử lý và lưu trữ dữ liệu	31
4.3.1 Thiết kế cơ sở dữ liệu	31
4.3.2 Quy trình xử lý và lưu trữ dữ liệu.....	38
4.4 Lớp phục vụ.....	40
4.4.1 Bài toán phân tích dữ liệu	40
4.4.2 Trực quan hóa dữ liệu	40
4.5 Lớp trực quan hóa dữ liệu.....	41
4.5.1 Mục đích.....	41
4.5.2 Hệ thống trực quan hóa.....	42
4.6 Lớp giám sát hệ thống	43
4.6.1 Tính cấp thiết.....	43
4.6.2 Quy trình giám sát hệ thống	43
4.7 Kết luận	44
CHƯƠNG 5. MỘT SỐ BÀI TOÁN PHÂN TÍCH DỮ LIỆU	46
5.1 Bài toán phân loại bình luận sản phẩm.....	46
5.1.1 Tính cấp thiết của bài toán	46
5.1.2 Thuật toán Random Forest	46
5.1.3 Xây dựng thuật toán Random Forest	48

5.1.4 Tại sao thuật toán Random Forest tốt	50
5.2 Bài toán gợi ý sản phẩm	51
5.2.1 Tính cấp thiết.....	51
5.2.2 Tổng quát về TF-IDF	51
5.2.3 Sử dụng TF-IDF cho bài toán gợi ý.....	53
CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG.....	55
6.1 Môi trường thực nghiệm.....	55
6.2 Triển khai hệ thống	56
6.3 Kết quả thực nghiệm	61
6.3.1 Lớp thu thập dữ liệu	61
6.3.2 Lớp xử lý và lưu trữ dữ liệu	62
6.3.3 Lớp phục vụ	63
6.3.4 Lớp trực quan hóa dữ liệu	67
6.3.5 Lớp giám sát hệ thống	74
6.4 Các bài toán phân tích dữ liệu	77
6.4.1 Bài toán phân loại bình luận sản phẩm	77
6.4.2 Bài toán gợi ý sản phẩm.....	78
6.5 Kiểm thử.....	78
6.6 Đánh giá	80
CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	82
7.1 Kết luận	82
7.2 Hướng phát triển.....	82
TÀI LIỆU THAM KHẢO.....	84
PHỤ LỤC.....	86
A. CÔNG NGHỆ SỬ DỤNG	86
B. HÌNH ẢNH HỆ THỐNG.....	88

DANH MỤC HÌNH VẼ

Hình 2.1	Tiki - Tìm kiếm và Tiết kiệm	9
Hình 3.1	Apache Kafka	11
Hình 3.2	Kiến trúc Apache Kafka	12
Hình 3.3	Thành phần của một chủ đề	12
Hình 3.4	Apache HBase	14
Hình 3.5	Cấu trúc bảng trong Apache HBase	15
Hình 3.6	Kiến trúc Apache HBase	16
Hình 3.7	Elasticsearch	18
Hình 3.8	Kiến trúc Elasticsearch	18
Hình 3.9	Apache Spark	20
Hình 3.10	Thành phần Apache Spark	20
Hình 3.11	Kibana	21
Hình 3.12	Prometheus	22
Hình 3.13	Grafana	23
Hình 4.1	Kiến trúc hệ thống	26
Hình 4.2	API	27
Hình 4.3	Thu thập dữ liệu	39
Hình 5.1	Thuật toán Random Forest	47
Hình 6.1	Tài nguyên Docker	56
Hình 6.2	Triển khai các Container	61
Hình 6.3	Dữ liệu danh mục trong <code>category_topic</code>	62
Hình 6.4	HBase Region Server	62
Hình 6.5	Bảng <code>category</code> và <code>overview_rating</code>	63
Hình 6.6	Bảng <code>product</code> và <code>rating</code>	63
Hình 6.7	Dữ liệu sản phẩm trong <code>product_topic</code>	64
Hình 6.8	Dữ liệu tổng quan đánh giá sản phẩm trong <code>overview_topic</code>	64
Hình 6.9	Dữ liệu đánh giá sản phẩm trong <code>rating_topic</code>	64
Hình 6.10	Chỉ mục lưu trữ dữ liệu sản phẩm	65
Hình 6.11	Chỉ mục lưu trữ dữ liệu tổng quan đánh giá mỗi sản phẩm	66
Hình 6.12	Chỉ mục lưu trữ dữ liệu đánh giá	67
Hình 6.13	Số lượng sản phẩm	67
Hình 6.14	Danh sách sản phẩm được bán nhiều nhất	68
Hình 6.15	Danh sách sản phẩm giá hiện bán cao nhất	69

Hình 6.16	Số lượng danh mục tổng	69
Hình 6.17	Phân bố danh mục	70
Hình 6.18	Phân bố sản phẩm trong các thương hiệu	70
Hình 6.19	Danh sách thương hiệu có giá trị nhất	71
Hình 6.20	Số lượng cửa hàng	71
Hình 6.21	Phân bố sản phẩm theo cửa hàng	72
Hình 6.22	Số đánh giá	72
Hình 6.23	Biểu đồ số sao đánh giá	73
Hình 6.24	Danh sách sản phẩm có nhiều đánh giá 5 sao nhất	73
Hình 6.25	Danh sách tiêu đề đánh giá nhiều nhất	74
Hình 6.26	Prometheus	75
Hình 6.27	Biểu đồ giám sát tài nguyên hệ thống	76
Hình 6.28	Biểu đồ giám sát Kafka	76
Hình 6.29	Biểu đồ giám sát Elasticsearch	77
Hình 6.30	Dữ liệu dùng cho bài toán phân loại	78
Hình 6.31	Phân bố dữ liệu bài toán phân loại	78
Hình 6.32	Kết quả dự đoán	78
Hình A.1	Quá trình đọc, ghi dữ liệu của Apache HBase	87
Hình B.1	Danh sách sản phẩm có giá trị nhất	88
Hình B.2	Danh sách danh mục có giá trị nhất	89
Hình B.3	Danh sách cửa hàng có giá trị nhất	90
Hình B.4	Danh sách sản phẩm có đánh giá nhiều nhất	90
Hình B.5	Danh sách sản phẩm có nhiều đánh giá 1 sao nhất	91

DANH MỤC BẢNG BIỂU

Bảng 4.1	Bảng <i>category</i>	32
Bảng 4.3	Bảng <i>overview_rating</i>	35
Bảng 4.4	Bảng <i>rating</i>	37
Bảng 4.2	Bảng <i>product</i>	45
Bảng 6.1	Thông tin môi trường thực nghiệm	55
Bảng 6.2	Kiểm thử tính sẵn sàng	79
Bảng 6.3	Kiểm thử hiệu năng	80

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
API	Application Programming Inter- face	Giao diện lập trình ứng dụng
URL	Uniform Resource Locator	Địa chỉ của một tài nguyên duy nhất trên mạng
ID	Identification	Mã định danh của một đối tượng
DATN		Đồ án tốt nghiệp

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong thời đại cách mạng công nghệ 4.0, dữ liệu lớn (Big Data) đã trở thành một yếu tố quan trọng và phổ biến trong hầu hết các lĩnh vực, trong đó có lĩnh vực thương mại điện tử. Sự phát triển mạnh mẽ của công nghệ thông tin và truyền thông đã tạo ra một lượng dữ liệu khổng lồ từ nhiều nguồn khác nhau trên mạng. Điều này đặt ra một thách thức lớn trong việc thu thập, lưu trữ, xử lý và phân tích dữ liệu một cách hiệu quả và mang lại giá trị thực tế.

Sàn thương mại điện tử là một môi trường đang phát triển mạnh mẽ, thu hút ngày càng nhiều người dùng và hoạt động. Với sự phát triển của công nghệ và tiện ích mua sắm trực tuyến, việc mua và bán hàng trên các sàn thương mại điện tử đã trở nên phổ biến và tiện lợi hơn bao giờ hết. Điều này cũng đồng nghĩa với việc khối lượng dữ liệu được tạo ra trên các sàn thương mại điện tử được tăng lên với tốc độ chóng mặt. Sự gia tăng này đặt ra nhiều thách thức về quản lý, lưu trữ và xử lý dữ liệu, đồng thời tạo cơ hội cho việc phân tích thông tin và tìm kiếm thông tin giá trị từ các dữ liệu đó.

Việc hiểu dữ liệu có thể giúp các doanh nghiệp có thể hiểu rõ hơn về thị trường, xu hướng mua sắm và hành vi của khách hàng để đưa ra quyết định thông minh. Phân tích dữ liệu cũng giúp cải thiện trải nghiệm người dùng, tối ưu hoạt động và nâng cao hiệu quả chiến dịch marketing, quản lý kho hàng và phân phối cho doanh nghiệp. Đối với khách hàng, họ sẽ có cái nhìn tổng quan hơn về thị trường sản phẩm, giúp họ tiết kiệm thời gian tìm kiếm sản phẩm, các ưu đãi đặc biệt và thông tin tùy chỉnh trong quá trình mua sắm. Do đó, việc xây dựng một hệ thống để thu thập, lưu trữ, xử lý và phân tích dữ liệu từ thương mại điện tử để phục vụ các nhu cầu liên quan đến phân tích thị trường, dự báo xu hướng tiêu dùng, tối ưu hóa chiến lược kinh doanh và cung cấp thông tin hữu ích cho quyết định lãnh đạo trở nên cấp thiết.

Vấn đề xây dựng hệ thống để thu thập, lưu trữ, xử lý và phân tích dữ liệu từ sàn thương mại điện tử không chỉ áp dụng cho lĩnh vực thương mại điện tử mà còn có thể được áp dụng vào nhiều lĩnh vực khác. Với sự phát triển của kinh tế số, các doanh nghiệp, tổ chức và ngành công nghiệp khác cũng đang đối mặt với thách thức quản lý và tận dụng dữ liệu. Việc xây dựng một hệ thống mạnh mẽ và linh hoạt để thu thập, lưu trữ, xử lý và phân tích dữ liệu là một nhu cầu ngày càng tăng trong mọi lĩnh vực.

Tóm lại, với nguồn dữ liệu khổng lồ được tạo ra hiện nay, việc xây dựng hệ thống thu thập, lưu trữ, xử lý và phân tích dữ liệu sàn thương mại điện tử là vô cùng quan trọng và cấp thiết. Đó không chỉ là một bài toán trong lĩnh vực sàn thương mại điện tử mà còn mang tính chất đa ngành, đáp ứng nhu cầu của các tổ chức, doanh nghiệp và ngành công nghiệp khác trong việc quản lý và tận dụng dữ liệu một cách hiệu quả. Dựa trên tính cấp thiết của vấn đề này, em đã quyết định chọn đề tài "Xây dựng hệ thống lưu trữ, xử lý và phân tích dữ liệu Sàn thương mại điện tử".

1.2 Mục tiêu và phạm vi đề tài

1.2.1 Mục tiêu

Mục tiêu của đề tài là xây dựng một hệ thống lưu trữ, xử lý và phân tích dữ liệu sàn thương mại điện tử hiệu quả, đáp ứng nhu cầu của các tổ chức, doanh nghiệp và ngành công nghiệp trong việc quản lý và tận dụng dữ liệu một cách hiệu quả, đồng thời phát triển các khả năng mở rộng để đáp ứng yêu cầu ngày càng cao về quản lý và xử lý dữ liệu từ sàn thương mại điện tử.

1.2.2 Phạm vi đề tài

Trên sàn thương mại điện tử, có rất nhiều loại dữ liệu được tạo ra như dữ liệu sản phẩm, thông tin khách hàng, giao dịch, hành vi người dùng và dữ liệu quảng cáo,... Tuy nhiên, trong phạm vi của đề án này, chúng ta sẽ tập trung vào dữ liệu liên quan đến các danh mục, thông tin sản phẩm và đánh giá của người dùng về các sản phẩm. Với loại dữ liệu này, chúng ta sẽ thực hiện các công việc sau::

- Đề xuất mô hình kiến trúc và thiết kế chi tiết của hệ thống, bao gồm các thành phần chính như hệ thống thu thập dữ liệu, cơ sở dữ liệu phân tán, hệ thống xử lý dữ liệu và trực quan hóa dữ liệu.
- Phát triển và triển khai hệ thống dựa trên mô hình kiến trúc và thiết kế đã đề xuất. Đồng thời, áp dụng các kỹ thuật học máy và khai phá dữ liệu để phân tích thông tin từ dữ liệu thu thập được.
- Tiến hành thử nghiệm và đánh giá hiệu suất của hệ thống, đảm bảo tính nhất quán và chất lượng dữ liệu thu thập được.
- Đáp ứng các khả năng mở rộng và cải thiện hệ thống trong tương lai.

1.3 Định hướng giải pháp

1.3.1 Phương pháp nghiên cứu

Để đạt được mục tiêu đã đề ra, đề tài sẽ áp dụng các phương pháp nghiên cứu theo các bước sau đây:

- Nghiên cứu về kiến trúc Lambda¹: Tìm hiểu mô hình kiến trúc Lambda để xây dựng mô hình hệ thống cho bài toán để tận dụng sự linh hoạt của việc xử lý dữ liệu theo từng luồng riêng biệt. Điều này giúp tối ưu hóa hiệu suất, giảm thiểu độ trễ và đảm bảo tính nhất quán của dữ liệu. Ngoài ra, mô hình này cũng cho phép dễ dàng mở rộng hệ thống bằng cách thêm các bước xử lý cho mỗi luồng dữ liệu theo nhu cầu và yêu cầu của dự án.
- Thiết kế các lớp trong hệ thống: Thiết kế các lớp chính của hệ thống theo kiến trúc Lambda bao gồm lớp thu thập dữ liệu, lớp lưu trữ dữ liệu, lớp xử lý dữ liệu, lớp trực quan hóa dữ liệu và lớp giám sát hệ thống. Mỗi lớp sẽ có vai trò và chức năng cụ thể trong quy trình xử lý dữ liệu.
- Tìm hiểu để sử dụng công nghệ và công cụ phù hợp: Tìm hiểu các công nghệ như Apache Kafka² để thu thập và xử lý dữ liệu thời gian thực, Apache HBase³ để lưu trữ dữ liệu phân tán, Elasticsearch⁴ và Kibana⁵ để tìm kiếm và trực quan hóa dữ liệu, và Prometheus⁶ và Grafana⁷ để giám sát và phân tích hiệu suất hệ thống.

Đề tài sẽ đóng góp vào việc xây dựng một hệ thống toàn diện và hiệu quả để thu thập, lưu trữ, xử lý và phân tích dữ liệu sản thương mại điện tử. Các công nghệ và công cụ được sử dụng sẽ giúp tối ưu hóa hiệu suất và đáp ứng yêu cầu về quản lý và tận dụng dữ liệu từ sản thương mại điện tử.

1.3.2 Phương pháp thực hành

Sau khi hoàn tất quá trình nghiên cứu và thiết kế, đề tài sẽ tiến hành thực hiện thực hành để xây dựng hệ thống thông qua các bước sau:

- Triển khai hệ thống: Xây dựng và triển khai hệ thống dựa trên thiết kế đã đề xuất. Cài đặt và cấu hình các công nghệ và công cụ được sử dụng trong hệ thống như Apache Kafka, Apache HBase, Elasticsearch, Kibana, Prometheus và Grafana. Đảm bảo hệ thống hoạt động đúng cách và kết nối một cách liên mạch giữa các thành phần.
- Thu thập dữ liệu: Sử dụng Apache Kafka để thu thập dữ liệu thời gian thực từ nguồn dữ liệu sản thương mại điện tử một cách liên tục và đẩy nó vào hệ thống.

¹[1] N. Marz and J. Warren, *Big Data: Principles and best practices of scalable realtime data systems*, 2015

²[2] T.S.F Apache, Kafka 3.5 documentation. [Online]

³[3] T.S.F Apache, Apache hbase™ reference guide. [Online]

⁴[4] N. Elastic, Introducing elasticsearch service. [Online]

⁵[5] N. Elastic, Kibana—your window into elastic. [Online]

⁶[6] C. N. C. Foundation, What is prometheus? [Online]

⁷[7] I. Raintank, Get started with grafana and prometheus. [Online]

- Lưu trữ và xử lý dữ liệu: Sử dụng Apache HBase để lưu trữ dữ liệu phân tán và xử lý dữ liệu theo các quy tắc và quy trình đã được thiết kế. Tiến hành xử lý dữ liệu để trích xuất thông tin và tạo ra các thống kê hữu ích.
- Trực quan hóa dữ liệu: Sử dụng Elasticsearch và Kibana để tìm kiếm, truy vấn và trực quan hóa dữ liệu thu thập được. Tạo ra các biểu đồ, đồ thị và báo cáo trực quan để hiển thị thông tin quan trọng về hoạt động của sàn thương mại điện tử.
- Đánh giá hiệu suất: Tiến hành thử nghiệm và đánh giá hiệu suất của hệ thống. Đo lường và theo dõi các thông số hiệu suất bằng Grafana, đảm bảo rằng hệ thống đáp ứng được yêu cầu và hoạt động một cách ổn định.

Qua các bước này, ta có thể xác định được cách cài đặt và cấu hình hệ thống, đảm bảo hệ thống hoạt động đúng cách và đáp ứng yêu cầu một cách ổn định.

1.4 Đối tượng và định hướng sử dụng

Hệ thống lưu trữ, xử lý và phân tích dữ liệu sàn thương mại điện tử có thể được sử dụng bởi các tổ chức, doanh nghiệp và ngành công nghiệp liên quan đến sàn thương mại điện tử. Cụ thể, đối tượng sử dụng hệ thống bao gồm:

- Các cửa hàng, doanh nghiệp: Theo dõi xu hướng mua hàng của người dùng, chất lượng sản phẩm của các cửa hàng khác, cũng như đánh giá hiệu quả của các sản phẩm hiện có và định hướng phát triển sản phẩm mới.
- Các nhà phân tích dữ liệu và chuyên gia thị trường: Nghiên cứu và phân tích dữ liệu để hiểu xu hướng thị trường, hành vi người dùng và đưa ra khuyến nghị và chiến lược kinh doanh.
- Người dùng: Có góc nhìn tổng quan về thị trường, nhận thông tin chính xác về sản phẩm, đánh giá chất lượng sản phẩm, từ đó hỗ trợ trong quyết định mua hàng và tìm kiếm các sản phẩm phù hợp.

Định hướng sử dụng của đề tài này là cung cấp một giải pháp toàn diện cho việc quản lý và tận dụng dữ liệu sàn thương mại điện tử một cách hiệu quả. Hệ thống được thiết kế với mục tiêu mang lại hiệu quả và linh hoạt trong quản lý dữ liệu, đồng thời đáp ứng nhu cầu người dùng. Đề tài hướng đến giúp các cửa hàng, doanh nghiệp trên sàn thành công trong lĩnh vực thương mại điện tử bằng cách cung cấp giải pháp giám sát và phân tích dữ liệu để sử dụng và hiệu quả.

1.5 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau:

Chương 2: Giới thiệu tổng quan dữ liệu lớn, sàn thương mại điện tử, cùng với

các thách thức và cơ hội mà nó mang lại. Sau đó trình bày sự ra đời và phát triển của Tiki⁸.

Chương 3: Trình bày về các công nghệ sử dụng trong hệ thống, bao gồm Apache Kafka, Apache HBase, Elasticsearch, Kibana, Prometheus và Grafana. Chương này sẽ giới thiệu tổng quan các công nghệ, cách hoạt động, mục đích sử dụng cũng như trả lời câu hỏi tại sao lại dùng nó.

Chương 4: Tập trung mô tả về kiến trúc tổng quan của hệ thống sẽ xây dựng. Đầu tiên, xác định các lớp chính của hệ thống, bao gồm lớp thu thập dữ liệu, lớp xử lý và lưu trữ dữ liệu, lớp phục vụ, lớp trực quan hóa dữ liệu và lớp giám sát hệ thống. Sau đó, mô tả về mục đích sử dụng và quy trình hoạt động của mỗi lớp.

Chương 5: Trình bày một số bài toán phân tích dữ liệu được áp dụng trong hệ thống. Cụ thể, chương này đề cập đến bài toán phân loại bình luận sản phẩm sử dụng thuật toán Rừng ngẫu nhiên (Random Forest⁹) và bài toán xây dựng mô hình gợi ý sản phẩm sử dụng kỹ thuật TF - IDF¹⁰.

Chương 6: Trình bày về môi trường thực nghiệm, quá trình triển khai hệ thống và kết quả thực nghiệm thu được. Sau đó đi đến đánh giá hiệu suất và chất lượng của hệ thống.

Chương 7: Tổng kết các kết quả đạt được từ đề tài và đề xuất hướng phát triển và cải tiến cho hệ thống trong tương lai.

⁸Tiki - Mua hàng online giá tốt, hàng chuẩn, ship nhanh, <https://tiki.vn>

⁹[8] T. K. Ho, “Random decision forests”, ATT Bell Laboratories, 1995

¹⁰[9] B. Das and S. Chakraborty, “An improved text sentiment classification model using tf-idf and next word negation,” IEEE, 2018

CHƯƠNG 2. DỮ LIỆU LỚN VÀ SÀN THƯƠNG MẠI ĐIỆN TỬ

2.1 Giới thiệu về Dữ liệu lớn

Dữ liệu lớn, hay còn được gọi là Big Data, là thuật ngữ được sử dụng để miêu tả khối lượng lớn và phức tạp của dữ liệu mà không thể được xử lý bằng các công cụ và phương pháp truyền thống. Dữ liệu lớn có đặc điểm chung là khối lượng dữ liệu rất lớn, tốc độ sinh dữ liệu nhanh, đa dạng về định dạng và tính cấu trúc, tính chất không chắc chắn, và đòi hỏi khả năng xử lý dữ liệu mạnh mẽ để trích xuất thông tin hữu ích và kiến thức từ dữ liệu đó.

Thuật ngữ "Big Data"¹ được phát triển từ những năm 1990, khi nhà khoa học John Mashey sử dụng nó để mô tả các dự án nghiên cứu khoa học mà dữ liệu thu thập được vượt quá khả năng xử lý của công cụ và phương pháp hiện có. Tuy nhiên, Big Data chỉ trở nên phổ biến và quan trọng hơn trong thời kỳ Cách mạng Công nghiệp 4.0, khi mạng Internet, công nghệ di động, cảm biến và các công nghệ khác đã tạo ra một lượng dữ liệu khổng lồ từ các nguồn khác nhau như: Thương mại Điện tử, Mạng xã hội, Internet vạn vật (IoT)...

Đặc điểm chính của dữ liệu lớn được mô tả bởi "5V" như sau:

- i. Tốc độ (Velocity): Dữ liệu lớn được tạo ra, thu thập và truyền đi với tốc độ nhanh chóng. Dữ liệu có thể được tạo ra liên tục từ nhiều nguồn khác nhau trong thời gian thực hoặc gần thời gian thực.
- ii. Đa dạng (Variety): Dữ liệu lớn có tính đa dạng và độ phức tạp cao. Nó bao gồm không chỉ dữ liệu cấu trúc như trong cơ sở dữ liệu truyền thống, mà còn dữ liệu phi cấu trúc như văn bản, hình ảnh, video, âm thanh, dữ liệu xã hội và nhiều nguồn dữ liệu khác.
- iii. Khối lượng (Volume): Dữ liệu lớn được tạo ra và tích tụ với quy mô rất lớn. Khối lượng dữ liệu này có thể từ hàng terabytes đến hàng petabytes hoặc thậm chí exabytes, vượt quá khả năng xử lý của công nghệ thông tin truyền thống.
- iv. Chính xác (Veracity): Đề cập đến tính tin cậy và độ chính xác của dữ liệu, đảm bảo sự đáng tin cậy khi sử dụng dữ liệu trong quá trình phân tích và ra quyết định.
- v. Giá trị (Value): Khả năng trích xuất giá trị từ dữ liệu lớn bằng cách phân tích và khai thác thông tin để mang lại lợi ích kinh tế.

¹[10] S. Lohr, "The origins of 'big data': An etymological detective story," New York Times, 2013

Dữ liệu lớn mang đến cơ hội vô tận cho các tổ chức và doanh nghiệp. Khả năng phân tích và khai thác dữ liệu lớn giúp tối ưu hoá hoạt động kinh doanh, đưa ra quyết định thông minh và tạo ra lợi thế cạnh tranh. Ngoài ra, dữ liệu lớn còn là nguồn tài nguyên quý giá cho nghiên cứu khoa học, phát triển sản phẩm và dịch vụ mới. Bằng cách hiểu và khai thác tốt dữ liệu lớn, các tổ chức có thể tiếp cận thông tin chính xác và chi tiết, khám phá những tiềm năng, và tận dụng cơ hội để đạt được sự phát triển và thành công bền vững.

Tuy nhiên, dữ liệu lớn mang đến không chỉ cơ hội mà còn nhiều thách thức đáng kể. Xử lý và quản lý khối lượng dữ liệu lớn, đa dạng và không chắc chắn yêu cầu sự phát triển của công nghệ và phương pháp mới. Ngoài ra, tốc độ sinh ra dữ liệu ngày càng nhanh đòi hỏi khả năng xử lý kịp thời để đáp ứng hiệu quả và đáng tin cậy.

Tóm lại, dữ liệu lớn đem lại cả cơ hội và thách thức. Việc hiểu và vượt qua những thách thức này sẽ mở ra tiềm năng to lớn để tận dụng dữ liệu và đạt được lợi ích sâu sắc từ dữ liệu lớn.

2.2 Dữ liệu lớn trong Sàn thương mại điện tử

Sàn thương mại điện tử (E-Commerce Platform) là một mô hình kinh doanh trực tuyến cho phép các doanh nghiệp và cá nhân tham gia trong quá trình mua bán sản phẩm và dịch vụ. Với sự phát triển của công nghệ và Internet, sàn thương mại điện tử, điển hình như Shopee, Tiki, Lazada,... đã trở thành một phương thức mua sắm phổ biến và tiện lợi cho người tiêu dùng, kết nối giữa người mua và người bán. Người mua có thể dễ dàng tìm kiếm và lựa chọn sản phẩm từ một kho hàng đa dạng và phong phú. Người bán có thể tiếp cận một lượng khách hàng rộng lớn và tiếp cận thị trường một cách hiệu quả.

Sàn thương mại điện tử cho thấy nó đã và đang thay đổi cách thức mua sắm và giao dịch trên toàn cầu. Với sự phát triển hiện nay, dữ liệu lớn đã có một tác động đáng kể đến ngành thương mại điện tử, mang lại cơ hội và thách thức đối với các doanh nghiệp và cá nhân. Với số lượng người mua và người bán ngày càng tăng trên các nền tảng thương mại điện tử, lượng dữ liệu đang ngày càng tăng lên, bao gồm:

- Dữ liệu sản phẩm: Tên sản phẩm, mô tả sản phẩm, giá sản phẩm, hình ảnh sản phẩm, v.v..
- Dữ liệu của người dùng: Họ và tên, địa chỉ, lịch sử mua hàng, lịch sử tương tác, sản phẩm đã thêm vào giỏ hàng, v.v..
- Dữ liệu cửa hàng: Địa chỉ cửa hàng, thời gian hoạt động, v.v..

Và rất nhiều dữ liệu khác, về quảng cáo, giao dịch, tương tác,...

Nguồn dữ liệu này tạo ra nhiều cơ hội cho các sàn thương mại điện tử. Nguồn dữ liệu này có thể được sử dụng để cải thiện trải nghiệm người dùng bằng cách đề xuất sản phẩm liên quan, hiển thị quảng cáo phù hợp và cung cấp dịch vụ khách hàng tốt hơn. Bằng cách đó, sàn thương mại điện tử có thể tối ưu hóa các chiến dịch tiếp thị. Ví dụ, sàn thương mại điện tử có thể sử dụng dữ liệu đánh giá của người dùng, lịch sử mua sắm để đề xuất sản phẩm liên quan, hiển thị quảng cáo phù hợp,... Sàn thương mại điện tử cũng có thể sử dụng dữ liệu hoạt động của người dùng để xác định các sản phẩm mà người dùng có nhiều khả năng quan tâm, nhằm mục tiêu quảng cáo đến những người có nhiều khả năng quan tâm, và theo dõi hiệu quả của các chiến dịch, đồng thời xác định xu hướng, phân tích hiệu quả của các chiến lược, các sàn thương mại điện tử có thể đưa ra các quyết định kinh doanh sáng suốt.

Nhìn chung, dữ liệu lớn là một công cụ mạnh mẽ có thể được sử dụng để cải thiện hiệu quả của các sàn thương mại điện tử. Bằng cách sử dụng dữ liệu một cách hiệu quả, các sàn thương mại điện tử có thể tăng doanh thu, tăng thị phần và tăng lòng trung thành của khách hàng. Tuy nhiên, đi kèm với những cơ hội là những thách thức phải đối mặt:

Đối với các sàn thương mại điện tử, khi số lượng người dùng và giao dịch tăng lên, khối lượng dữ liệu cần xử lý cũng tăng theo. Điều này có thể gây ra các vấn đề về hiệu suất và khả năng mở rộng. Dữ liệu từ các sàn thương mại điện tử có thể rất phức tạp, bao gồm dữ liệu từ nhiều nguồn khác nhau, chẳng hạn như dữ liệu hoạt động của người dùng, dữ liệu từ các thiết bị đầu cuối, dữ liệu từ các hệ thống khác nhau, v.v.. Điều đó có thể khiến việc phân tích và hiểu dữ liệu trở nên khó khăn. Các sàn thương mại điện tử cần có khả năng phân tích, xử lý một lượng lớn dữ liệu khi việc xử lý dữ liệu có thể rất tốn thời gian và tài nguyên với lượng dữ liệu lớn.

Việc dữ liệu tăng lên cũng tạo khó khăn đối với người dùng. Người dùng có thể bị quá tải thông tin khi phải đối mặt với một lượng lớn dữ liệu. Điều này có thể khiến họ khó đưa ra quyết định về những gì họ muốn mua. Dữ liệu lớn có thể được sử dụng để theo dõi người dùng và thu thập thông tin cá nhân về họ. Điều này có thể khiến người dùng cảm thấy lo ngại về quyền riêng tư của họ. Ngoài ra, có những rủi ro khi thực hiện các hành vi gian lận, chẳng hạn như lừa đảo người dùng mua hàng hoặc đánh cắp thông tin cá nhân của họ.

Các nhà phân tích dữ liệu có thể gặp nhiều khó khăn trong việc thu thập, lưu trữ và phân tích dữ liệu khi lượng dữ liệu ngày càng nhiều và phức tạp. Chất lượng dữ liệu cũng không tốt khi dữ liệu có thể không chính xác hoặc không đầy đủ. Điều này có thể khiến khó khăn cho các nhà phân tích dữ liệu trong việc đưa ra quyết

định dựa trên dữ liệu.

Nhìn chung, dữ liệu lớn là một công cụ mạnh mẽ có thể được sử dụng để cải thiện hiệu quả của các sàn thương mại điện tử. Tuy nhiên, dữ liệu lớn cũng đặt ra một số thách thức, đòi hỏi các sàn thương mại điện tử phải giải quyết những thách thức này để có thể tận dụng dữ liệu lớn một cách hiệu quả.

2.3 Tiki - Tìm kiếm và Tiết kiệm

Trên thị trường hiện nay, có rất nhiều sàn thương mại điện tử hoạt động đáp ứng nhu cầu cao của mọi người như Shopee.vn, Tiki.vn, Lazada.vn,... Trong đề án này, chúng ta sẽ tập trung nghiên cứu và phân tích dữ liệu từ Tiki.vn, một trang thương mại điện tử do người Việt sáng lập và phát triển.



Hình 2.1: Tiki - Tìm kiếm và Tiết kiệm

Tiki là một trang web thương mại điện tử hàng đầu tại Việt Nam, có tên gọi được viết tắt từ cụm từ "Tiết kiệm và Tìm kiếm". Sàn thương mại điện tử này được thành lập và quản lý bởi ông Trần Ngọc Thái Sơn từ tháng 3 năm 2010. Tiki nhanh chóng trở thành một trong những nền tảng mua sắm trực tuyến phổ biến và được ưa chuộng tại Việt Nam.

Tiki ra đời ban đầu là một trang thương mại điện tử chuyên bán sách và nổi tiếng với cam kết sách thật 100% và giá bán cạnh tranh. Với hơn 10 năm phát triển, Tiki đã mở rộng hoạt động để phù hợp với xu hướng mua sắm của người tiêu dùng. Hiện nay, Tiki đã trở thành một trang thương mại điện tử đa ngành cung cấp hàng trăm nghìn sản phẩm từ sách, điện tử, đồ gia dụng, thực phẩm đến thời trang, phục vụ cho mọi lứa tuổi và nhóm người khác nhau. Bằng việc mua sắm trên Tiki, người dùng có thể tìm kiếm và mua đủ mọi thứ, với đồ điện tử và sách vẫn là những mặt hàng được ưa chuộng với giá tốt nhất.

Các gian hàng trên Tiki được kiểm định kỹ lưỡng về nguồn gốc và chất lượng sản phẩm. Chỉ những gian hàng đã được kiểm định và đáp ứng tiêu chí về hàng chính hãng mới được phép đăng bán trên sàn Tiki. Đối với các sản phẩm điện tử, khách hàng có thể yên tâm vì chúng sẽ được bảo hành đầy đủ như khi mua ở cửa hàng thông thường và đảm bảo là hàng chính hãng. Tiki cũng cung cấp dịch vụ lắp đặt miễn phí cho nhiều loại hàng như Tivi, điều hòa, tủ lạnh,... Đội ngũ nhân viên

chuyên nghiệp của Tiki luôn sẵn sàng hỗ trợ khách hàng và cam kết không làm người dùng thất vọng.

Tuy nhiên, Tiki vẫn còn phải đối mặt với thách thức về sự thiếu đa dạng trong ngành hàng so với các kênh thương mại khác và số lượng sản phẩm chưa đủ đa dạng. Mặc dù vậy, Tiki vẫn được công nhận là một sàn thương mại điện tử uy tín, tin cậy và đáng để chúng ta lựa chọn khi mua hàng.

Trong đồ án này, chúng ta sẽ sử dụng dữ liệu khổng lồ của Tiki để xây dựng hệ thống thu thập, lưu trữ và xử lý nguồn dữ liệu này, giải quyết 1 phần các thách thức mà dữ liệu lớn đặt ra với sàn thương mại điện tử trong nền công nghiệp 4.0.

2.4 Kết luận

Trong chương này, chúng ta đã nhận thấy rằng dữ liệu lớn đem lại cả thách thức và cơ hội cho các tổ chức và doanh nghiệp. Việc xử lý và phân tích dữ liệu ngày càng trở nên phức tạp hơn do sự gia tăng về khối lượng và tốc độ của dữ liệu. Tuy nhiên, nếu được khai thác và quản lý một cách thông minh, Big Data có thể mang lại thông tin quan trọng và tri thức để tối ưu hoá hoạt động kinh doanh. Trong bối cảnh sự bùng nổ của dữ liệu lớn, các sàn thương mại điện tử đã không thể tránh khỏi tác động đó, và Tiki - một sàn thương mại điện tử nổi tiếng tại Việt Nam - cũng không phải là ngoại lệ.

Trong các chương tiếp theo, chúng ta sẽ được giới thiệu về hệ thống thu thập, lưu trữ và xử lý dữ liệu thông minh mà Tiki sử dụng.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

3.1 Công nghệ lưu trữ dữ liệu

3.1.1 Apache Kafka

Apache Kafka là một hệ thống xử lý dữ liệu phân tán và hướng sự kiện (Event Streaming Platform) được phát triển bởi Apache Software Foundation¹. Nó được thiết kế để xử lý và lưu trữ hàng tỷ thông điệp theo thời gian thực từ nhiều nguồn dữ liệu khác nhau.



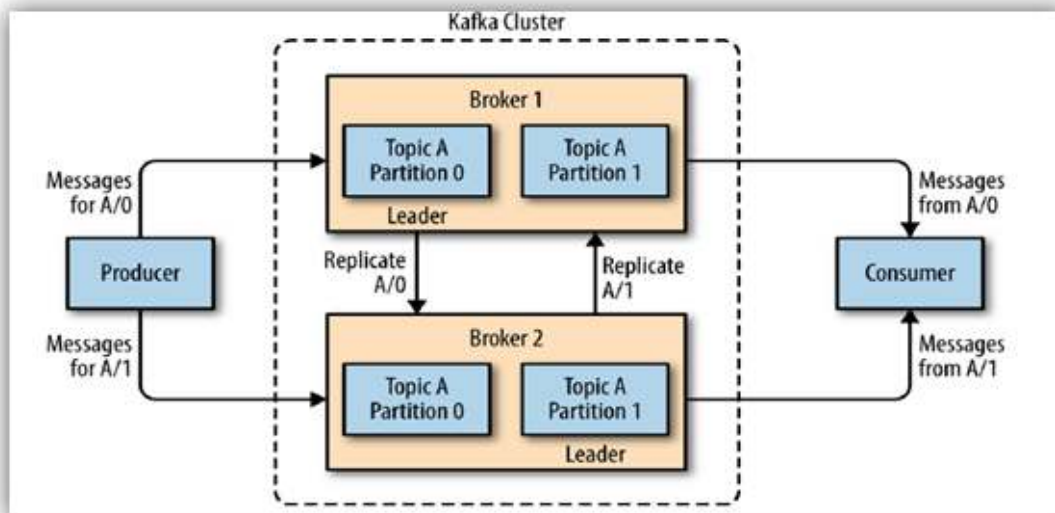
Hình 3.1: Apache Kafka

Kafka sử dụng mô hình lưu trữ log (Log-structured Storage) để lưu trữ dữ liệu, là các thông điệp (Message). Các thông điệp chứa một hoặc nhiều byte dữ liệu, được ghi vào các Partition theo thứ tự. Dữ liệu được lưu trữ trên đĩa chỉ, và chỉ được ghi thêm, không cho phép chỉnh sửa hoặc xóa bỏ.

Kafka cho phép các ứng dụng gửi và nhận dữ liệu theo kiểu "Publisher - Subscriber". Các nhà cung cấp dữ liệu (Publishers) gửi thông điệp vào các chủ đề (Topic), trong khi các ứng dụng tiêu thụ dữ liệu (Subscribers) đăng ký để nhận các thông điệp từ chủ đề tương ứng. Dữ liệu trong Kafka được lưu trữ trong các phân vùng (Partition), mỗi phân vùng là một chuỗi các thông điệp được sắp xếp theo thứ tự. Các phân vùng có thể được phân tán trên nhiều máy chủ Kafka (Broker) để tăng khả năng mở rộng và độ tin cậy của hệ thống.

Dưới đây là hình vẽ mô tả kiến trúc và các thành phần tổng quan của Apache Kafka.

¹<https://www.apache.org/foundation/>



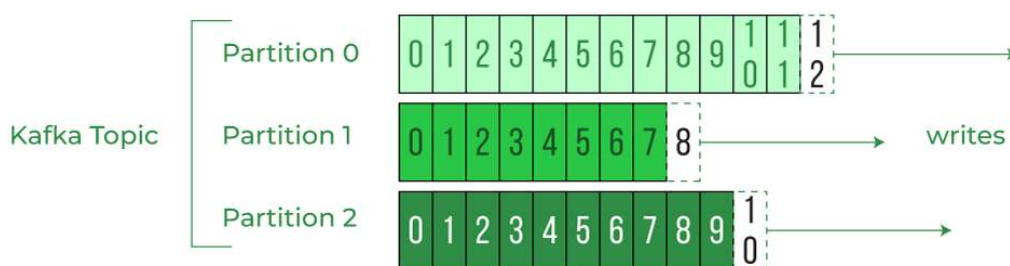
Hình 3.2: Kiến trúc Apache Kafka

Nguồn: Narayan Kumar, "Kafka Architecture Internal", Oct. 2020

Apache Kafka sẽ thu thập, lưu trữ và xử lý thông điệp từ các nhà cung cấp dữ liệu (Producer) và cung cấp cho các nhà tiêu thụ dữ liệu (Consumer). Dữ liệu trong Kafka được tổ chức thành các chủ đề (Topic) và được phân chia thành các phân vùng (Partition) để phân tán và lưu trữ dữ liệu. Mỗi thông điệp trong Kafka được gắn với một vị trí (Offset) duy nhất trong phân vùng.

Kafka hoạt động thông qua các máy chủ (Broker) trong mạng Kafka (Kafka Cluster), nơi các phân vùng được lưu trữ và xử lý. Khả năng mở rộng của Kafka cho phép thêm nhiều máy chủ để tăng cường khả năng xử lý và đáp ứng tải cao.

Sao chép (Replication) trong Kafka là quá trình sao chép dữ liệu từ một phân vùng sang các phân vùng khác trên các máy chủ khác nhau, giúp đảm bảo tính nhất quán và sẵn sàng của dữ liệu trong trường hợp một máy chủ gặp sự cố.



Hình 3.3: Thành phần của một chủ đề

Nguồn: AmiyaRanjanRout, "Topics, Partitions, and Offsets in Apache Kafka", Oct. 2022

Với kiến trúc linh hoạt, cùng cách hoạt động đặc biệt của mình, Apache Kafka

là một lựa chọn lý tưởng để xử lý sự kiện và dòng dữ liệu.

Trong phạm vi đề án này, chúng ta cần một công cụ để di chuyển dữ liệu từ thành phần này, lớp này sang thành phần khác, lớp khác, đảm bảo tính liên tục, toàn vẹn cũng như độ tin cậy của dữ liệu. Có nhiều loại Message Broker đảm nhiệm được công việc trên, như Kafka, RabbitMQ², Redis³,...

Với khả năng xử lý lên đến 1.000.000 thông điệp mỗi giây, Kafka có thể xử lý hiệu quả dữ liệu với lưu lượng cao. Trong khi đó, RabbitMQ chỉ có thể xử lý tới 50.000 thông điệp mỗi giây. Redis cũng có khả năng xử lý 1.000.000 thông điệp, nhưng do là cơ sở dữ liệu trên bộ nhớ (In-memory Database⁴), việc sử dụng Redis có thể gây thiếu tài nguyên cho hệ thống, không thể thực hiện các tác vụ khác.

Kafka như đã nói là một hệ thống xử lý sự kiện (Event Streaming) mạnh mẽ, cho phép phân tích thời gian thực, mở rộng dễ dàng, sử dụng cơ chế ghi theo thứ tự và lưu giữ dữ liệu trong một khoảng thời gian nhất định trong khi RabbitMQ là một hệ thống gửi nhận thông điệp (Messaging System) trung gian, thông điệp sẽ được xóa khỏi hàng đợi sau khi đã được xử lý và xác nhận bởi người tiêu thụ, còn Redis được sử dụng để lưu trữ các dữ liệu tạm thời, cache hoặc dữ liệu có thời hạn sử dụng ngắn hạn do cơ chế lưu trữ của nó là cơ sở dữ liệu trên bộ nhớ.

Tóm lại, Redis và RabbitMQ tập trung vào việc xử lý truy vấn và gửi/nhận tin nhắn một cách hiệu quả, trong khi Kafka được thiết kế để hỗ trợ phân tích thời gian thực và lưu trữ dữ liệu lâu dài, đồng thời có khả năng xử lý nhiều dữ liệu đồng thời. Vì dữ liệu cần giao tiếp và tương tác với các thành phần, là dữ liệu liên tục với khối lượng lớn, cần lưu trữ lâu dài, chúng ta sẽ sử dụng Kafka. Sự lựa chọn Kafka giúp tăng hiệu quả và linh hoạt trong việc truyền và xử lý dữ liệu trong hệ thống, đáp ứng yêu cầu về xử lý lưu lượng cao và khả năng mở rộng.

3.1.2 Apache HBase

a. Tổng quan

Apache Hbase là một hệ quản trị cơ sở dữ liệu dựa trên mã nguồn mở Hadoop thuộc dự án của Apache được mở rộng và phát triển từ dự án lưu trữ Bigdata của Google. Hbase được viết bằng ngôn ngữ lập trình Java, có thể lưu trữ được dữ liệu cực lớn từ terabytes đến petabytes.

²Rabbit Technologies Ltd, "Launch of RabbitMQ Open Source Enterprise Messaging", Feb. 2007

³AWS Whitepaper, "Database Caching Strategies Using Redis", Mar. 2021

⁴Dữ liệu được lưu trữ và truy xuất trực tiếp từ RAM thay vì trên ổ cứng



Hình 3.4: Apache HBase

HBase sử dụng cơ chế lưu trữ log-structured, cho phép truy vấn dữ liệu một cách nhanh chóng, nhờ đó lọc dữ liệu một cách hiệu quả, giúp giảm thời gian xử lý và cải thiện hiệu suất truy vấn. Khả năng lưu trữ hàng tỷ rows và columns giúp HBase đáp ứng nhu cầu lưu trữ của dữ liệu lớn, đồng thời có khả năng mở rộng quy mô để đáp ứng sự phát triển của dữ liệu.

HBase cung cấp khả năng truy vấn dữ liệu cũng như hỗ trợ các công cụ như bộ nhớ cache và bộ lọc Bloom để tăng tốc độ truy vấn dữ liệu theo thời gian thực, giúp việc truy xuất và phân tích dữ liệu một cách linh hoạt và nhanh chóng. Đồng thời, tính năng chuyển đổi dự phòng tự động giữa các Region Servers giúp bảo vệ dữ liệu và đảm bảo tính sẵn sàng của hệ thống.

b. Mô hình dữ liệu

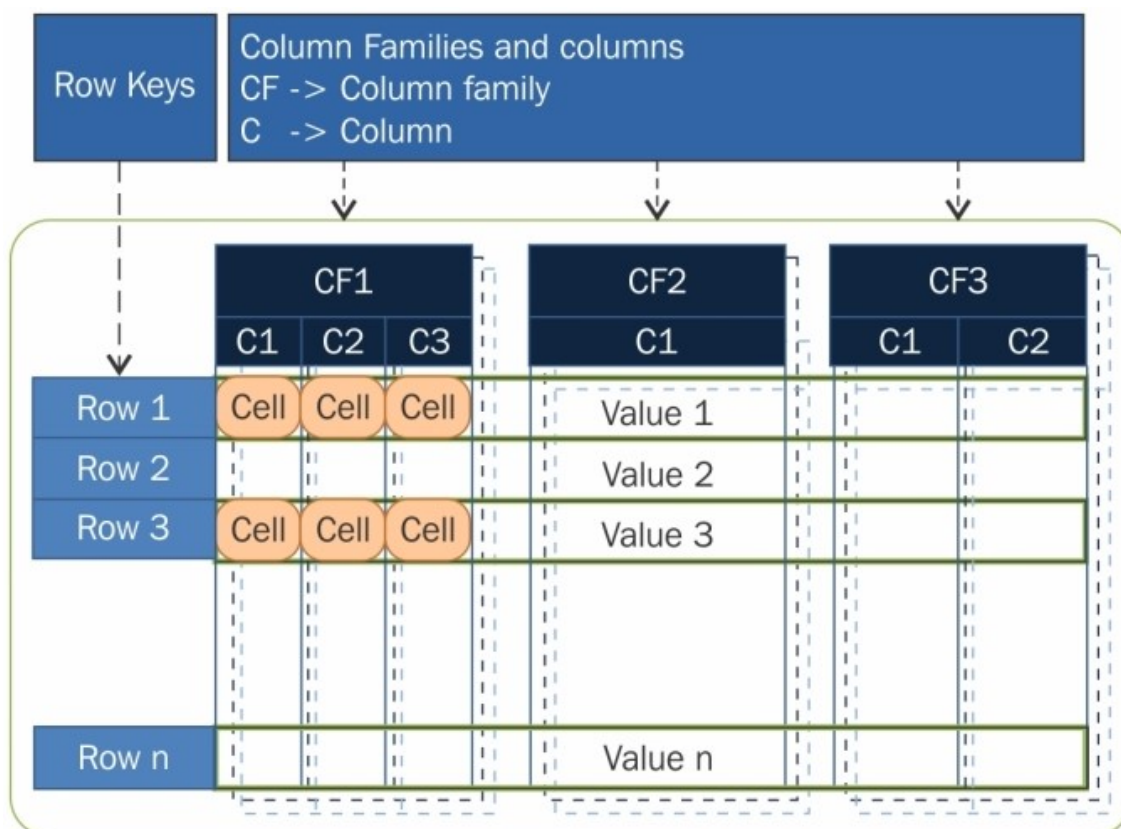
HBase là một cơ sở dữ liệu phân tán theo cột, trong đó dữ liệu được tổ chức thành các bảng. Mỗi bảng trong HBase bao gồm nhiều hàng (Row), và mỗi hàng lại bao gồm nhiều nhóm cột (Column Family) chứa các cặp key-value. Hàng trong HBase được xác định bởi một giá trị duy nhất gọi là Row Key, tương đương với khóa chính (Primary Key) trong các cơ sở dữ liệu thông thường.

Các hàng trong bảng được sắp xếp theo thứ tự từ điển dựa trên giá trị của khóa chính, giúp tối ưu hóa quá trình truy xuất dữ liệu. Mỗi hàng trong HBase bao gồm nhiều cột (Column Qualifier) khác nhau. Để tổ chức các cột một cách hiệu quả, HBase cho phép gộp các cột thành các nhóm cột gọi là Column Families. Mỗi Column Family được đặt tên dưới dạng "families: qualifier", trong đó "families" là tên của Column Families và "qualifier" được sử dụng để xác định cụ thể một cột trong Column Family đó. Dữ liệu của các cột được lưu trữ trên đĩa, mỗi ô trong bảng cũng chứa các thông tin Meta như Time Stamp và các thông tin khác.

Một bản ghi cụ thể trong HBase được xác định bởi các ô dữ liệu (Cells), bao gồm: hàng (Row), cột (Column), và phiên bản (Version). HBase sử dụng một loạt các tệp log (log files) để lưu trữ dữ liệu mới được ghi. Khi dữ liệu được ghi vào hệ thống, nó được thêm vào cuối của tệp log hiện có. Dữ liệu cũ cũng không bị xóa hoặc sửa đổi, chỉ có cập nhật mới nhất được thêm vào cuối tệp

log. Việc này giúp tối ưu việc ghi dữ liệu vì không cần phải di chuyển hoặc cập nhật dữ liệu đã tồn tại. Quá trình truy xuất dữ liệu trong HBase yêu cầu sử dụng khóa chính để xác định các bản ghi cụ thể. Cụ thể hơn sẽ được nói trong chương **Phụ lục**.

Dưới đây là mô tả trực quan cách một bảng trong HBase được tổ chức:

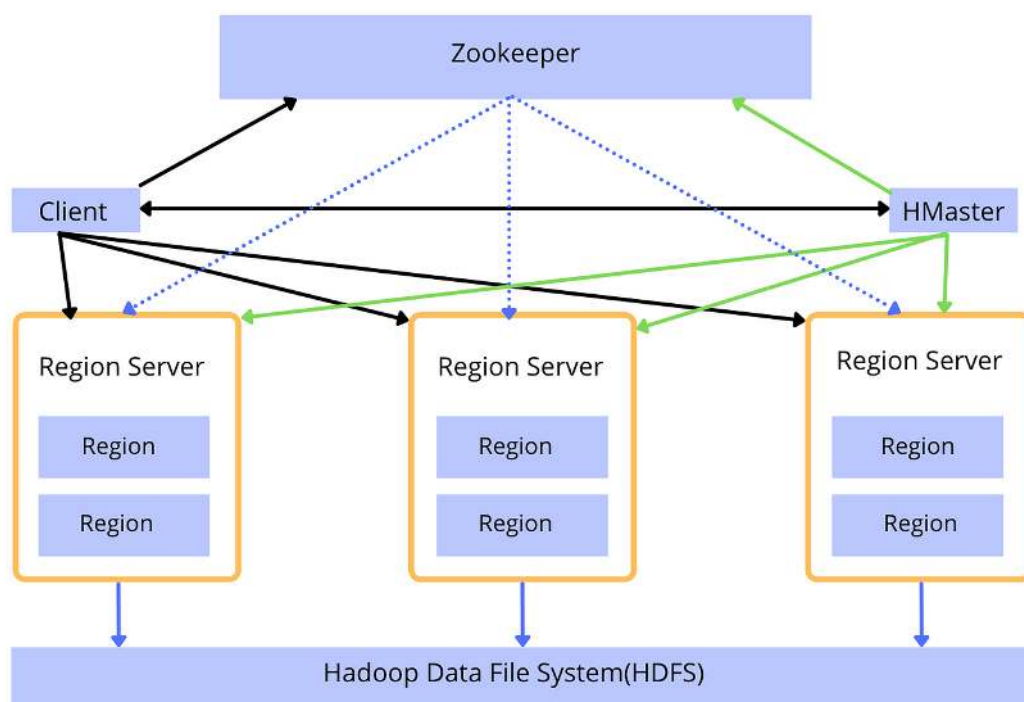


Hình 3.5: Cấu trúc bảng trong Apache HBase
Nguồn: oreilly.com, "The HBase data model"

Một trong những đặc điểm quan trọng làm cho việc lưu trữ dữ liệu trong HBase rất linh hoạt và mềm dẻo là tính không giới hạn của Column (hay Column Qualifier). Column có thể chứa dữ liệu với độ dài và kiểu không giới hạn, và dữ liệu có thể được thêm vào một cách không hạn chế. Điều này cho phép HBase thích ứng tốt với các tình huống lưu trữ dữ liệu đa dạng và phức tạp, đồng thời hỗ trợ việc mở rộng và điều chỉnh cấu trúc dữ liệu một cách dễ dàng. Đây chính là một trong những yếu tố làm việc lưu trữ dữ liệu HBase rất linh hoạt và mềm dẻo.

c. Kiến trúc

Để hiểu rõ hơn về Apache HBase, chúng ta sẽ xem kiến trúc của nó qua hình ảnh sau:



Hình 3.6: Kiến trúc Apache HBase

Nguồn: dshahid380, "Introduction to Apache HBase", Mar. 2022

Dựa vào hình ảnh trên, chúng ta có thể nhận thấy các thành phần chủ yếu của HBase bao gồm:

- **HMaster:** Là thành phần quản lý siêu dữ liệu (Metadata), quản lý cụm và điều phối các hoạt động của HBase. HMaster giám sát các RegionServer và quản lý các bảng, Row Key, và các chỉ mục liên quan.
- **ZooKeeper⁵:** Là dịch vụ phân tán chuyên dụng, được sử dụng để duy trì thông tin quản lý và đồng bộ giữa các nút trong cụm HBase. ZooKeeper giúp HBase theo dõi trạng thái của các RegionServer và HMaster, phát hiện và khắc phục lỗi và đảm bảo tính nhất quán của hệ thống.
- **RegionServer:** Là nơi thực tế lưu trữ và xử lý dữ liệu trong HBase. Mỗi RegionServer quản lý một hoặc nhiều region của các bảng. Khi một yêu cầu đọc hoặc ghi dữ liệu được gửi đến HBase, HMaster sẽ định vị RegionServer thích hợp để xử lý yêu cầu đó. RegionServer cũng xử lý các hoạt động như lưu trữ, truy xuất và xử lý dữ liệu.
- **Region:** Là một phần ngang hàng của bảng trong HBase. Mỗi bảng được chia thành nhiều region và được phân bố trên các RegionServer khác nhau

⁵The Apache Software Foundation, Zookeeper, <https://zookeeper.apache.org/doc/r3.4.7/zookeeperOver.pdf>

để phân tải công việc và tăng hiệu suất. Các region thay đổi kích thước dựa trên tải công việc và quản lý dữ liệu động.

- HDFS (Hadoop Distributed File System): Là hệ thống lưu trữ phân tán của Hadoop⁶[11] và là hệ thống lưu trữ dữ liệu chính cho HBase. Dữ liệu trong HBase được lưu trữ trong HDFS, đảm bảo khả năng chịu lỗi và mở rộng trong việc lưu trữ.
- HBase Client: Là một API được cung cấp bởi HBase để cho phép ứng dụng giao tiếp và thực hiện các thao tác đọc, ghi, tìm kiếm và quản lý dữ liệu trong HBase.

Tóm lại, kiến trúc của Apache HBase bao gồm HMaster làm quản lý cụm và ZooKeeper để đồng bộ dữ liệu, các RegionServer quản lý và xử lý dữ liệu trong các Region, dữ liệu được lưu trữ trong HDFS, và HBase Client cho phép ứng dụng tương tác với HBase. Các thành phần này cùng nhau tạo nên một hệ thống cơ sở dữ liệu phân tán, mạnh mẽ và mở rộng của HBase.

Trong khuôn khổ đề án này, HBase sẽ được sử dụng làm công nghệ lưu trữ chính cho dữ liệu hệ thống. Điều này là do HBase cung cấp một số lợi thế so với các hệ thống lưu trữ phân tán khác như Cassandra⁷, MongoDB⁸,...

- HBase là cơ sở dữ liệu key-value tối ưu cho truy cập giá trị riêng lẻ dựa trên key, thích hợp cho lưu trữ logs và hệ thống theo dõi. Cassandra là cơ sở dữ liệu cột phù hợp cho truy vấn các cột dữ liệu cụ thể, thích hợp cho ứng dụng phân tích dữ liệu theo cột và hệ thống phân tán. MongoDB là cơ sở dữ liệu dạng JSON⁹ cho phép lưu trữ và truy vấn dữ liệu dưới dạng các tài liệu linh hoạt, phù hợp cho các ứng dụng web có tính chất dữ liệu thay đổi thường xuyên.
- HBase được thiết kế để đạt hiệu suất cao cho việc đọc và ghi dữ liệu, dựa trên mô hình dữ liệu Bigtable. Cassandra linh hoạt hơn và có khả năng sử dụng cho nhiều loại ứng dụng, có thể mở rộng ngang và dọc. MongoDB là cơ sở dữ liệu tài liệu, hỗ trợ mô hình dữ liệu linh hoạt
- HBase được tích hợp tốt hơn với các công cụ khác của Apache, như Hadoop và ZooKeeper. Điều này giúp dễ dàng xây dựng, kết nối và tương tác với các thành phần khác trong hệ sinh thái Apache.

Nhìn chung, HBase là một lựa chọn tốt cho việc lưu trữ và xử lý một lượng lớn dữ liệu có cấu trúc. Vì vậy, Apache HBase sẽ được triển khai như một hệ thống cơ sở

⁶Dhruba Borthakur, "The Hadoop Distributed File System: Architecture and Design", 2006

⁷Hamilton, James, "Facebook Releases Cassandra as Open Source", Jul. 2008

⁸Gigaom, "10gen embraces what it created, becomes MongoDB Inc", Mar. 2016

⁹ISO/IEC 21778:2017 Information technology — The JSON data interchange syntax

dữ liệu để lưu trữ dữ liệu sàn thương mại điện tử.

3.1.3 Elasticsearch

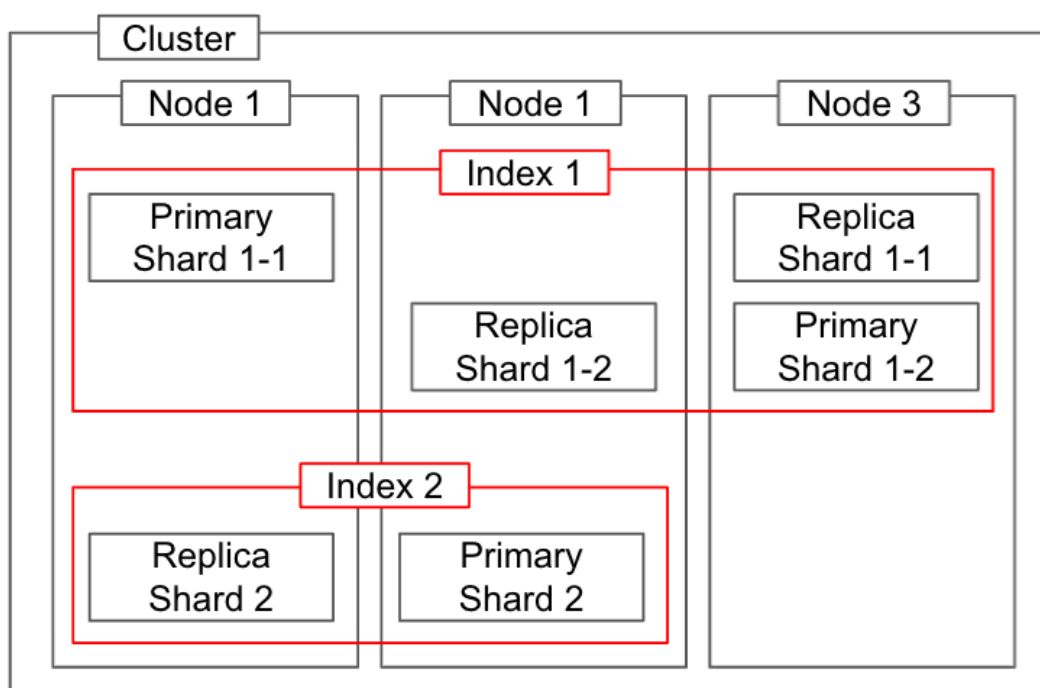
Elasticsearch là một hệ thống tìm kiếm và phân tích dữ liệu phân tán mã nguồn mở, được xây dựng trên nền tảng Apache Lucene¹⁰. Với Elasticsearch, việc lưu trữ, tìm kiếm và phân tích hàng tỷ bản ghi dữ liệu một cách nhanh chóng và hiệu quả. Elasticsearch được thiết kế để xử lý các tác vụ tìm kiếm phức tạp và truy vấn dữ liệu trong môi trường phân tán.



Hình 3.7: Elasticsearch

Elasticsearch có mô hình lưu trữ dữ liệu dạng tài liệu, trong đó mỗi tài liệu là một bản ghi độc lập có thể được tìm kiếm và truy xuất riêng lẻ. Dữ liệu trong Elasticsearch được tổ chức thành các chỉ mục (Index) và các loại (Type) trong mỗi chỉ mục. Elasticsearch hỗ trợ nhiều loại truy vấn, từ truy vấn đơn giản theo từ khóa cho đến truy vấn phức tạp sử dụng ngôn ngữ truy vấn JSON.

Kiến trúc của Elasticsearch như sau:



Hình 3.8: Kiến trúc Elasticsearch

¹⁰A. Białecski, R. Muir, G. Ingersoll, L. Imagination, "Apache Lucene 4", 2012

Trong đó:

- Nút (Node): Phiên bản Elasticsearch chạy trên một máy chủ riêng biệt trong cụm Elasticsearch.
- Cụm (Cluster): Tập hợp các nút hoạt động cùng nhau để lưu trữ và xử lý dữ liệu.
- Document (Tài liệu): Bản ghi dữ liệu độc lập trong một chỉ mục, lưu trữ dưới dạng JSON.
- Chỉ mục (Index): Bộ sưu tập các tài liệu có liên quan, cho phép tìm kiếm và truy xuất dữ liệu.
- Phân vùng (Shard): Đơn vị nhỏ nhất của dữ liệu trong Elasticsearch, được phân chia và lưu trữ trên nhiều nút.
- Bản sao (Replica): Bản sao của phân vùng trong cùng một chỉ mục, giúp đảm bảo tính sẵn sàng và đáng tin cậy.

Một trong những lợi ích quan trọng của Elasticsearch là khả năng mở rộng theo chiều ngang (Horizontal Scaling). Bằng cách thêm các nút (Nodes) vào cụm, Elasticsearch tự động phân chia và phân phối dữ liệu trên các nút để tăng cường hiệu suất và đáp ứng tải cao. Hệ thống cũng hỗ trợ sao chép dữ liệu tự động để đảm bảo tính sẵn sàng và đáng tin cậy.

Elasticsearch là một trong ba thành phần của ELK Stack¹¹, bao gồm: Elasticsearch - một hệ thống tìm kiếm và phân tích dữ liệu phân tán; Logstash¹² - công cụ thu thập và xử lý dữ liệu và Kibana (sẽ nói ở phần sau) - giao diện người dùng đồ họa để trực quan hóa dữ liệu. Trong phạm vi đồ án này, chúng ta sử dụng Elasticsearch kết hợp với Kibana để thực hiện truy vấn dữ liệu và tạo các biểu đồ, đồ thị để hiển thị dữ liệu một cách trực quan.

3.2 Công nghệ cho bài toán phân tích dữ liệu - Apache Spark

Apache Spark là một hệ sinh thái phân tán và mạnh mẽ được sử dụng để xử lý và phân tích dữ liệu lớn. Nó được thiết kế để xử lý dữ liệu lớn và phức tạp một cách hiệu quả, cung cấp tốc độ xử lý nhanh chóng và khả năng mở rộng linh hoạt. Spark hỗ trợ nhiều ngôn ngữ lập trình như Scala, Java, Python và R, thuận tiện tích hợp và triển khai các ứng dụng xử lý dữ liệu phức tạp.

¹¹G. S. Sachdeva, "Introduction to the ELK Stack", Jan. 2017

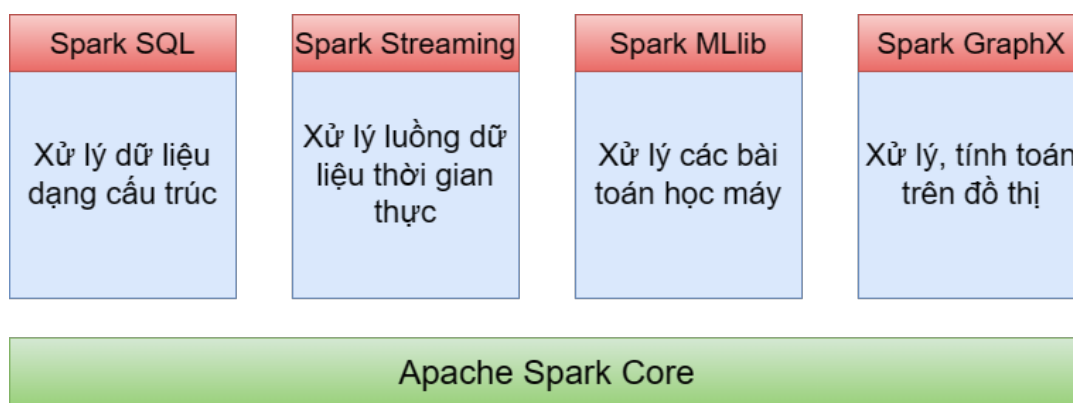
¹²S. Sanjappa, M. Ahmed, "Analysis of Logs by Using Logstash", Mar. 2017



Hình 3.9: Apache Spark

Các thành phần chính của Spark:

- Spark Core: Thành phần cốt lõi của Spark, cung cấp các chức năng cơ bản cho việc phân tán và xử lý dữ liệu trong môi trường phân tán.
- Spark SQL: Được sử dụng để xử lý dữ liệu dạng cấu trúc (Structured Data). Spark SQL cung cấp giao diện truy vấn dữ liệu SQL cho việc tương tác với dữ liệu lưu trữ trong DataFrame và DataSet.
- Spark Streaming: Dùng để xử lý dữ liệu dạng stream (dữ liệu liên tục) một cách trực tiếp và linh hoạt. Spark Streaming hỗ trợ xử lý dữ liệu từ các nguồn như Kafka, Flume,...
- Spark MLlib (Machine Learning Library): Thư viện hỗ trợ xây dựng và triển khai các mô hình học máy trên dữ liệu lớn. Spark MLlib cung cấp nhiều thuật toán máy học phổ biến như phân loại, gom cụm, hồi quy và phân tích lân cận gần.
- Spark GraphX: Thư viện hỗ trợ xử lý đồ thị và tính toán trên các đồ thị lớn. Spark GraphX cung cấp các công cụ cho việc xử lý, truy vấn và phân tích dữ liệu đồ thị.



Hình 3.10: Thành phần Apache Spark

Trong đồ án này, chúng ta sẽ sử dụng thư viện Spark MLlib để tận hết lợi ích từ những thuật toán học máy có sẵn và giải quyết các bài toán phân tích dữ liệu. MLlib cung cấp một loạt các thuật toán học máy phổ biến như hồi quy tuyến tính,

phân loại, gom cụm, rừng ngẫu nhiên,... Đặc biệt, MLlib cho phép chúng ta xử lý dữ liệu trên RDD (Resilient Distributed Dataset) và DataFrame, giúp tối ưu hóa hiệu suất xử lý dữ liệu lớn.

Spark MLlib là một thành phần quan trọng của Apache Spark và dễ dàng tích hợp với các công nghệ Apache khác. Hơn nữa, chúng ta cũng sẽ tích hợp Spark Streaming vào hệ thống để xử lý dữ liệu dạng liên tục (Stream). Spark Streaming cho phép chúng ta xử lý dữ liệu ngay lập tức từ các nguồn dữ liệu như Kafka, Flume, Kinesis... Tích hợp Spark Streaming vào hệ thống giúp chúng ta đáp ứng và xử lý dữ liệu một cách linh hoạt và nhanh chóng, từ đó tối ưu hóa quá trình phân tích và đưa ra các quyết định dựa trên dữ liệu mới nhất.

3.3 Công nghệ trực quan hóa dữ liệu - Kibana

Kibana là một công cụ phân tích và trực quan hóa dữ liệu mã nguồn mở, được phát triển bởi Elastic. Nó là một thành phần quan trọng trong ELK Stack (Elasticsearch, Logstash và Kibana), giúp người dùng dễ dàng khám phá, trực quan hóa và tìm hiểu dữ liệu từ Elasticsearch.



Hình 3.11: Kibana

Kibana cung cấp một giao diện người dùng đồ họa thân thiện và dễ sử dụng, cho phép người dùng thực hiện truy vấn, tìm kiếm, phân tích và trực quan hóa dữ liệu. Với Kibana, người dùng có thể tạo các biểu đồ, đồ thị, bản đồ, báo cáo và trực quan hóa dữ liệu trong thời gian thực.

Một số tính năng chính của Kibana bao gồm:

- **Trực quan hóa dữ liệu:** Kibana cho phép trực quan hóa dữ liệu từ Elasticsearch bằng cách tạo các biểu đồ, đồ thị, bản đồ, và các loại trực quan hóa dữ liệu khác. Người dùng có thể lựa chọn từ nhiều loại biểu đồ và tự tùy chỉnh để hiển thị dữ liệu theo cách phù hợp nhất.
- **Tìm kiếm và truy vấn dữ liệu:** Kibana cung cấp giao diện tìm kiếm để sử dụng cho phép người dùng tạo các truy vấn phức tạp để lấy dữ liệu từ Elasticsearch. Điều này giúp người dùng dễ dàng tìm hiểu thông tin cần thiết từ các tập dữ liệu lớn.

- Tích hợp mạnh mẽ với Elasticsearch và Logstash: Kibana được thiết kế để hoạt động tốt với Elasticsearch và Logstash. Dữ liệu từ Elasticsearch và Logstash có thể được trực tiếp sử dụng để tạo các trực quan hóa và báo cáo trong Kibana.

Nhờ vào những ưu thế và tính năng mạnh mẽ mà Kibana và Elasticsearch mang lại, đồ án này sử dụng Kibana kết hợp với Elasticsearch để xây dựng lớp trực quan hóa dữ liệu thay vì sử dụng các công cụ khác như Metabase. Sự phổ biến, tính linh hoạt, giao diện người dùng đồ họa thân thiện và khả năng tối ưu hóa hiệu suất xử lý dữ liệu lớn của ELK Stack đã ra một lớp trực quan hóa dữ liệu hiệu quả và dễ sử dụng trong hệ thống.

Phần tiếp theo sẽ giới thiệu về các công nghệ sử dụng trong quá trình giám sát hệ thống.

3.4 Công nghệ giám sát hệ thống

3.4.1 Prometheus

Prometheus là một công cụ phân tích dữ liệu và giám sát mã nguồn mở phổ biến được sử dụng để thu thập và lưu trữ các số liệu (Metrics), cũng như cảnh báo về các sự kiện. Nó được thiết kế để có thể mở rộng và có thể sử dụng để giám sát các hệ thống khác nhau và cung cấp một giao diện người dùng thân thiện, đồ họa tuyệt đẹp và tích hợp linh hoạt với nhiều nguồn dữ liệu khác nhau.



Hình 3.12: Prometheus

Prometheus sử dụng mô hình kéo (Pull) để thu thập các số liệu, từ các mục tiêu giám sát, là các máy chủ hoặc ứng dụng cung cấp dữ liệu. Điều này làm cho Prometheus hiệu quả về mặt băng thông và có thể sử dụng để giám sát các mục tiêu từ xa, đảm bảo việc thu thập dữ liệu được thực hiện đều đặn và đáng tin cậy.

Prometheus sử dụng một cơ sở dữ liệu chuỗi thời gian (Time Series Database) để lưu trữ các số liệu. Điều này cho phép người dùng truy vấn và phân tích các số liệu theo thời gian, từ đó xác định các xu hướng và phát hiện các sự kiện bất thường trong hệ thống.

Prometheus có thể được sử dụng để cảnh báo về các sự kiện quan trọng. Các cảnh báo có thể được gửi qua email, SMS hoặc thông báo đẩy. Điều này giúp chúng ta nhận biết sớm các vấn đề xảy ra ảnh hưởng đến hệ thống để nhanh chóng

xử lý và giải quyết các vấn đề, duy trì hiệu suất và tin cậy của hệ thống.

So với việc dùng các công nghệ tương tự, như Graphite,... Prometheus được thiết kế để dễ dàng triển khai và cấu hình, có thể tích hợp linh hoạt với nhiều công cụ và hệ thống khác, như Grafana, Alertmanager, Elasticsearch, và nhiều hệ thống lưu trữ dữ liệu khác. Prometheus sử dụng mô hình dữ liệu số liệu dạng chuỗi thời gian, trong khi Graphite sử dụng mô hình dữ liệu số liệu dạng cây, mang lại tính linh hoạt và hiệu quả trong việc thu thập và lưu trữ dữ liệu chuỗi thời gian. Ngoài ra, Graphite sử dụng mô hình đẩy (Push Model) và có thể gặp khó khăn khi xử lý lượng dữ liệu lớn, trong khi Prometheus sử dụng mô hình kéo (Pull Model) để thu thập dữ liệu, giúp giảm tải cho các mục tiêu giám sát.

Những lợi ích của Prometheus mà đã được đề cập trước đây thực sự làm cho nó trở thành một sự lựa chọn tuyệt vời để xây dựng lớp giám sát trong phạm vi đề án này. Trong phần tiếp theo, chúng ta sẽ tìm hiểu về các công cụ có thể kết hợp với Prometheus để sử dụng số liệu đã được thu thập, nhằm hoàn thiện lớp giám sát của hệ thống.

3.4.2 Grafana

Grafana là một công cụ phân tích dữ liệu, giám sát mã nguồn mở được sử dụng rộng rãi để tạo và xem các biểu đồ thời gian thực cũng như các bảng điều khiển từ các nguồn dữ liệu khác nhau, bao gồm Prometheus, Elasticsearch, InfluxDB, các nguồn dữ liệu cục bộ. Grafana là một công cụ mạnh mẽ và linh hoạt có thể được sử dụng để giám sát một loạt các hệ thống và ứng dụng.



Hình 3.13: Grafana

Grafana hoạt động bằng cách thu thập dữ liệu từ các nguồn dữ liệu khác nhau. Dữ liệu sau đó được lưu trữ trong cơ sở dữ liệu của Grafana. Grafana sau đó sử dụng dữ liệu này để tạo các biểu đồ và bảng điều khiển. Biểu đồ và bảng điều khiển này có thể được sử dụng để giám sát hệ thống và ứng dụng, xác định các vấn đề và cải thiện hiệu suất.

Grafana là một công cụ mạnh mẽ và linh hoạt, có thể thu thập dữ liệu từ nhiều nguồn dữ liệu khác nhau, bao gồm Prometheus, Elasticsearch, InfluxDB và các nguồn dữ liệu cục bộ. Grafana có thể được sử dụng để tạo các biểu đồ và bảng điều khiển thời gian thực từ dữ liệu thu thập được. Và Grafana cũng có thể được tích hợp

với các công cụ khác, chẳng hạn như Slack, Jira và Confluence.

Ngoài ra Grafana, còn có rất nhiều công cụ giám sát hệ thống khác như Nagios, Icinga, Monit, Zabbix,... nhưng đề án này ưu tiên sử dụng Grafana, vì một số lý do sau:

- Grafana dễ sử dụng hơn nhiều công cụ khác. Giao diện người dùng của Grafana trực quan và dễ điều hướng, ngay cả đối với người mới bắt đầu.
- Grafana linh hoạt hơn nhiều công cụ khác. Grafana có thể được sử dụng để tạo các loại biểu đồ và bảng điều khiển khác nhau, trong khi các công cụ khác bị giới hạn trong một số loại biểu đồ và bảng điều khiển nhất định.
- Grafana có thể được tích hợp với nhiều công cụ hơn nhiều công cụ khác. Grafana có thể được tích hợp với nhiều công cụ khác nhau, như Prometheus, MySQL, Elasticsearch,... Các công cụ khác chỉ có thể được tích hợp với một số công cụ nhất định.

Có thể thấy, Grafana là một công cụ giám sát mạnh mẽ, linh hoạt và dễ sử dụng, có thể được tích hợp với nhiều công cụ khác nhau. Grafana và Prometheus hỗ trợ nhau một cách tốt, tạo thành một cặp công cụ mạnh mẽ trong việc giám sát hệ thống. Prometheus chịu trách nhiệm thu thập số liệu và lưu trữ chúng dưới dạng chuỗi thời gian, trong khi Grafana giúp hiển thị dữ liệu này dưới dạng các biểu đồ và bảng điều khiển thời gian thực. Sự kết hợp chặt chẽ này chính là lý do chính khiến Grafana trở thành một công cụ lý tưởng để xây dựng lớp trực quan hóa dữ liệu cho đề án.

3.5 Kết luận

Chương này đã chi tiết giới thiệu về các công nghệ được sử dụng trong hệ thống, bao gồm Apache Kafka, Apache HBase, Elasticsearch, Kibana, Prometheus và Grafana. Mỗi công nghệ đã được trình bày với cách hoạt động, mục đích sử dụng và lý do tại sao được lựa chọn trong đề án này, thay vì sử dụng các công nghệ khác.

Từ sự hiểu biết về mỗi công nghệ và vai trò của nó trong hệ thống, chúng ta sẽ xác định được cách kết hợp chúng để tạo thành một hệ thống mạnh mẽ và hiệu quả, phục vụ cho mục tiêu giám sát và quản lý dữ liệu từ các sản phẩm thương mại điện tử. Chương tiếp theo sẽ đi sâu hơn vào các khía cạnh cụ thể của việc tích hợp các công nghệ này để thiết kế hệ thống hoạt động, giải quyết các yêu cầu của hệ thống.

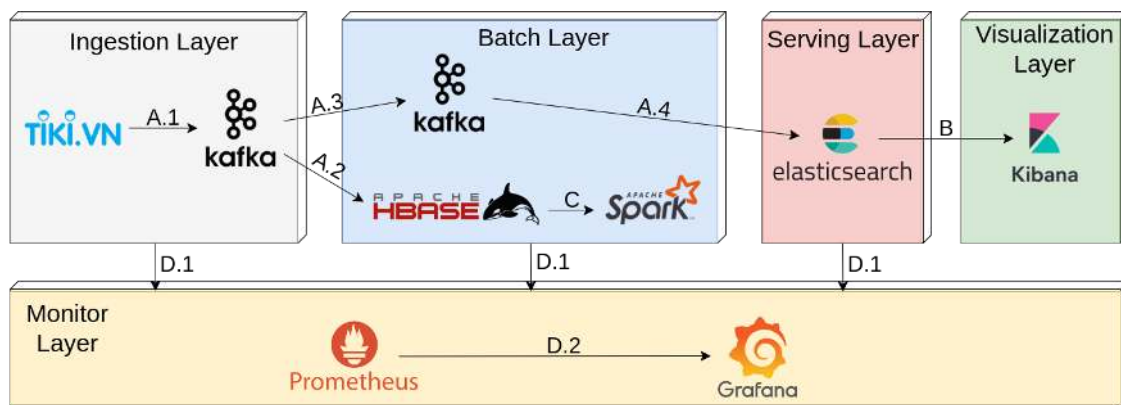
CHƯƠNG 4. KIẾN TRÚC HỆ THỐNG

4.1 Tổng quan hệ thống

Trong chương này, chúng ta sẽ tiến hành thiết kế kiến trúc của hệ thống sau khi đã xác định được các công nghệ sử dụng. Hệ thống sẽ được triển khai và xây dựng trên 5 lớp chính dựa trên kiến trúc Lambda như sau:

- **Lớp thu thập dữ liệu (Ingestion Layer):** Để bắt đầu, hệ thống sẽ tiến hành thu thập dữ liệu từ trang web Tiki.vn. Dữ liệu sẽ thu thập bao gồm thông tin về sản phẩm, các bình luận đánh giá từ khách hàng cùng với danh mục các sản phẩm. Chúng ta sẽ sử dụng API để trích xuất những mã danh mục sản phẩm hiện có trên Tiki và sau đó gửi dữ liệu này đến Topic tương ứng trong Kafka, giúp các thành phần khác có dữ liệu để tiếp tục công việc của mình.
- **Lớp xử lý và lưu trữ dữ liệu (Batch Layer):** Kafka Consumer sẽ lắng nghe dữ liệu danh mục đã được gửi bởi Kafka Producer. Sau đó một tiến trình sẽ lấy các mã danh mục để thực hiện truy vấn API tương ứng để trích xuất dữ liệu, tiến hành làm sạch, biến đổi và tổng hợp dữ liệu trước khi lưu vào Apache HBase. Quá trình này tạo ra một cơ sở dữ liệu tổng hợp và có cấu trúc, phục vụ cho các tác vụ xử lý và phân tích tiếp theo. Sau đó, tiếp tục biến đổi dữ liệu, gửi vào các Kafka Topic để đồng bộ hóa dữ liệu lớp phục vụ.
- **Lớp phục vụ (Serving Layer):** Các thành phần trong lớp này sẽ lắng nghe các Kafka Topic để đồng bộ hóa dữ liệu, thực hiện lưu trữ vào Elasticsearch, tạo khả năng truy vấn kết quả đã đạt được cũng như cung cấp dữ liệu để trực quan hóa ở lớp trực quan hóa dữ liệu.
- **Lớp trực quan hóa dữ liệu (Visualization Layer):** Sau khi dữ liệu đã được biến đổi và lưu trữ tại Elasticsearch, Kibana được sử dụng để xây dựng các biểu đồ và đồ thị từ dữ liệu đã được lưu trữ. Qua việc trực quan hóa dữ liệu bằng các biểu đồ, chúng ta có thể hiển thị thông tin dễ đọc và dễ hiểu, giúp thực hiện phân tích và khám phá thông tin từ dữ liệu thu thập được.
- **Lớp giám sát hệ thống (Monitor Layer):** Lớp này sử dụng Grafana và Prometheus để theo dõi và giám sát hoạt động của hệ thống. Prometheus thu thập và ghi lại các thông số quan trọng từ các thành phần của hệ thống, trong khi Grafana cho phép xây dựng các bảng điều khiển và biểu đồ tùy chỉnh từ thông số thu thập được. Sự kết hợp giữa Grafana và Prometheus sẽ tạo ra cái nhìn tổng quan về tình trạng và hiệu suất của hệ thống, cung cấp thông tin cần thiết để phát hiện và khắc phục sự cố một cách nhanh chóng.

Dưới đây là hình vẽ mô tả kiến trúc của hệ thống được thiết kế và triển khai:



Trong đó:

- A - Luồng thu thập dữ liệu
- B - Luồng trực quan hóa dữ liệu
- C - Luồng phân tích dữ liệu
- D - Luồng giám sát hệ thống

Hình 4.1: Kiến trúc hệ thống

4.2 Lóp thu thập dữ liệu

Trong quá trình xây dựng hệ thống, việc thu thập dữ liệu đóng vai trò quan trọng trong việc cung cấp nguồn thông tin cho hệ thống hoạt động. Dữ liệu có thể thu thập được từ nhiều nguồn khác nhau, bằng nhiều cách khác nhau, mang lại tiềm năng lớn cho doanh nghiệp và tổ chức. Tuy nhiên, để đảm bảo tính nhất quán và chất lượng của dữ liệu, việc xác định quy trình thu thập dữ liệu là cần thiết.

Quy trình thu thập dữ liệu định hình cách thức thu thập, xử lý và lưu trữ dữ liệu, đảm bảo quá trình thu thập dữ liệu diễn ra một cách có hệ thống và hiệu quả. Quy trình này bao gồm xác định nguồn dữ liệu, lựa chọn phương pháp thu thập phù hợp, thiết kế rõ ràng, đảm bảo thời gian, tần suất thu thập và xử lý các trường hợp ngoại lệ. Từ đó đảm bảo tính nhất quán, chất lượng và hiệu quả của dữ liệu thu thập được, tạo nền tảng cho hoạt động ổn định cho hệ thống.

4.2.1 Nguồn dữ liệu

Như đã trình bày ở mục 2.3 - "Tiki - Tìm kiếm và Tiết kiệm", chúng ta sẽ tập trung vào sàn thương mại điện tử Tiki. Tiki là một sàn thương mại điện tử phổ biến và phục vụ người dùng Việt Nam. Việc lựa chọn sàn thương mại điện tử này đảm bảo rằng nguồn dữ liệu phục vụ cho đề tài là đáng tin cậy và phù hợp với đối tượng người dùng Việt Nam.

Trong phạm vi đề án này, hệ thống sẽ lấy dữ liệu từ Tiki bao gồm:

- Dữ liệu về các sản phẩm: Bao gồm thông tin chi tiết về sản phẩm, như tên sản phẩm, mô tả, giá cả, thông số kỹ thuật, hình ảnh sản phẩm, v.v. cho phép xây

dựng một cơ sở dữ liệu thông tin chi tiết của từng sản phẩm.

- Dữ liệu về các bình luận và đánh giá: Những ý kiến, nhận xét và đánh giá và số sao của người dùng về chất lượng sản phẩm, dịch vụ và trải nghiệm mua sắm.
- Dữ liệu về các danh mục sản phẩm: Dữ liệu này cung cấp thông tin về các danh mục sản phẩm có sẵn trên trang web, giúp người dùng tìm kiếm và so sánh sản phẩm trong cùng một loại.

Nhờ vào nguồn dữ liệu đa dạng và phong phú từ Tiki, chúng ta có thể thực hiện bài toán phân tích kinh doanh và đánh giá tổng quan về hoạt động của sàn thương mại điện tử Việt Nam. Việc phân tích này sẽ giúp hiểu rõ hơn về xu hướng tiêu dùng, ưu thế cạnh tranh, và nhu cầu thị trường trong lĩnh vực thương mại điện tử tại Việt Nam.

4.2.2 Phương pháp thu thập dữ liệu

Trong quá trình thu thập dữ liệu từ sàn thương mại điện tử Tiki, chúng ta có nhiều phương pháp khác nhau để thu thập thông tin từ nguồn dữ liệu này như sử dụng API, Web Scraping, Web Crawling, ... Một trong những phương pháp phổ biến và tiện lợi nhất là sử dụng các bộ API mà Tiki cung cấp.



Hình 4.2: API

Tiki cung cấp một bộ API đầy đủ và toàn diện để thu thập dữ liệu từ nền tảng của họ, cho phép chúng ta truy cập vào các endpoint API cụ thể và lấy dữ liệu về thông tin của sản phẩm, đánh giá, danh mục và nhiều thông tin khác từ Tiki.

Việc sử dụng các API cung cấp sẵn giúp chúng ta tiết kiệm thời gian và công sức trong việc thu thập dữ liệu, đồng thời đảm bảo tính chính xác và đáng tin cậy của thông tin thu thập được.

Các API được Tiki cung cấp được sử dụng ở đây như sau:

a. Danh sách các danh mục

Mỗi ngày, Tiki sẽ hiển thị các danh mục nổi bật, các danh mục đó có thể truy xuất qua API:

https://tiki.vn/api/personalish/v1/blocks/categories?block_code=<?>&page_size=<?>

- Tham số:
 - **block_code**: Là một mã định danh đại diện cho một khối dữ liệu cụ thể liên quan đến danh mục sản phẩm.
 - **page_size**: Tham số quy định số lượng danh mục được hiển thị trên mỗi trang khi truy vấn dữ liệu.
- Dữ liệu: Danh sách các danh mục nổi bật được hiển thị, bao gồm mã danh mục, tên danh mục, miêu tả, đường dẫn,...
- Mô tả: API cung cấp danh sách các danh mục được hiển thị trong một ngày nhất định, cho phép chúng ta truy xuất và hiển thị các danh mục sản phẩm đang hoạt động trên nền tảng Tiki.vn.

b. Danh sách các sản phẩm trong một danh mục Trong một danh mục có rất nhiều sản phẩm, các sản phẩm đó được truy xuất qua API:

<https://tiki.vn/api/personalish/v1/blocks/listings?category=<?>&limit=<?>&page=<?>>

- Tham số:
 - **category**: Là mã danh mục được truy vấn.
 - **limit**: Tham số giới hạn số lượng sản phẩm hiển thị trên một trang.
 - **page**: Tham số quy định số trang để hiển thị danh sách sản phẩm.
- Dữ liệu: Danh sách các sản phẩm có trong danh mục với thông tin mỗi sản phẩm được rút gọn bao gồm: tên sản phẩm, giá hiện bán, đường dẫn,...
- Mô tả: API này cung cấp một danh sách các sản phẩm thuộc danh mục cụ thể, tuy nhiên một số trường thông tin đã bị loại bỏ. Trong trường hợp yêu cầu bài toán đòi hỏi các thông tin đầy đủ hơn về từng sản phẩm, chúng ta có thể sử dụng API chi tiết về sản phẩm được mô tả ở phần dưới.

c. Thông tin chi tiết của sản phẩm

Thông tin của các sản phẩm được truy xuất bằng API trên không được đầy đủ, do đó, Tiki.vn cũng cấp thông tin đầy đủ của một sản phẩm qua API:

https://tiki.vn/api/v2/products/<product_id>

- Tham số:
 - **product_id**: Tham số này được thay thế bởi mã sản phẩm muốn truy vấn.

- Dữ liệu: Thông tin đầy đủ của sản phẩm: tên sản phẩm, giá bán, giá gốc, cửa hàng bán,...
- Mô tả: API được thiết kế để cung cấp chi tiết về sản phẩm sẽ cung cấp đầy đủ các thông tin hiện có về sản phẩm đó trên sàn thương mại điện tử Tiki. Điều này bao gồm thông tin chi tiết về tên sản phẩm, giá cả, nhà cung cấp, mô tả sản phẩm, hình ảnh sản phẩm, danh mục chứa sản phẩm, và nhiều thông tin khác liên quan.

d. Thông tin về đánh giá

Tiki.vn cung cấp thông tin chi tiết đánh giá, bình luận của người dùng về một sản phẩm qua API:

```
https://tiki.vn/api/v2/reviews?product_id=<?>&limit
=<?>&page=<?>
```

- Tham số:
 - **product_id**: Là mã sản phẩm được truy vấn để lấy đánh giá về sản phẩm đó
 - **limit**: Tham số giới hạn số lượng đánh giá hiển thị trên một trang.
 - **page**: Tham số quy định số trang để hiển thị danh sách các đánh giá.
- Dữ liệu: Thông tin đầy đủ của các đánh giá về sản phẩm: tên, giá, shop,...
- Mô tả: API sẽ cung cấp đầy đủ các bình luận và đánh giá của người dùng về sản phẩm, giúp ta tiếp cận tất cả các thông tin phản hồi từ khách hàng về chất lượng, trải nghiệm và đánh giá sản phẩm. Dữ liệu từ API này cung cấp cái nhìn rõ ràng về sự hài lòng của người dùng và giúp chúng ta nắm bắt các thông tin quan trọng để cải tiến và tối ưu hóa trải nghiệm mua sắm trên nền tảng Tiki.vn.

Bộ API mà Tiki cung cấp đóng vai trò quan trọng trong việc thu thập dữ liệu từ sàn thương mại điện tử này. Tùy thuộc vào mục đích và phân tích cần thực hiện, chúng ta có thể lựa chọn và truy cập vào từng API cụ thể để thu thập thông tin chi tiết và đa dạng từ sàn thương mại điện tử Tiki.

Bằng các kết hợp và sử dụng các API trên, chúng ta có thể xây dựng một quy trình linh hoạt để thu thập dữ liệu một cách toàn diện và tạo cơ sở để xây dựng hệ thống lưu trữ cho nguồn dữ liệu này, đáp ứng đầy đủ yêu cầu của đề án.

4.2.3 Quy trình thu thập dữ liệu

Để xây dựng quy trình thu thập dữ liệu từ sàn thương mại điện tử Tiki.vn, chúng ta sẽ làm theo các bước sau:

- i. Xác định nguồn dữ liệu và phương pháp thu thập: Như đã trình bày ở trên, chúng ta sẽ tiến hành thu thập dữ liệu từ sàn thương mại điện tử Tiki.vn. Việc thu thập dữ liệu sẽ thông qua các API mà Tiki.vn đã cung cấp sẵn. Các API có thể cung cấp thông tin về danh sách sản phẩm trong một danh mục cụ thể hoặc thông tin về một sản phẩm cụ thể kèm theo đánh giá của người dùng.
- ii. Truy vấn API danh sách danh mục: Ban đầu, chúng ta chỉ có thông tin về các API liên quan đến danh mục sản phẩm, danh sách sản phẩm trong một danh mục, hay API chứa thông tin sản phẩm bất kỳ cùng với đánh giá sản phẩm đó. Các API hầu như đều yêu cầu tham số đầu vào để có thể truy xuất thông tin (mục 4.2.2), nhưng chúng ta lại chưa có bất kỳ thông tin gì về các sản phẩm, hay các danh mục. Chỉ có duy nhất API về danh sách các danh mục sản phẩm không cần tham số nào có thể truy xuất thông tin bao gồm mã các danh mục. Chúng ta có thể lấy mã các danh mục này làm tham số đầu vào để truy cập vào API danh sách sản phẩm trong mỗi danh mục để lấy được mã sản phẩm. Từ đó giải quyết được bài toán tham số. Do đó, chúng ta sẽ bắt đầu từ việc truy vấn vào API danh sách danh mục của Tiki.

Khi đã lấy được danh sách các danh mục, chúng ta tiến hành trích xuất các trường cần thiết ở mỗi danh mục, ở đây mã và tên của danh mục đó.
- iii. Gửi dữ liệu vào Kafka Topic: Tạo một Kafka Producer để gửi dữ liệu danh mục này vào một Kafka Topic là **category_topic**. Kafka Producer này sẽ lắng nghe API liên tục, có thể sử dụng một luồng xử lý riêng biệt hoặc cơ chế lặp lại định kỳ để kiểm tra sự thay đổi trong danh sách danh mục. Mỗi khi có sự thay đổi hoặc một danh mục mới được thêm vào, Kafka Producer sẽ gửi dữ liệu danh mục này vào Topic để các Kafka Consumer có thể lắng nghe và xử lý.
- iv. Xử lý dữ liệu từ danh mục: Các Kafka Consumer có thể đăng ký để lắng nghe các tin nhắn từ **category_topic**. Khi nhận được một tin nhắn chứa thông tin về danh mục mới hoặc đã thay đổi, Kafka Consumer có thể thực hiện các xử lý như lấy mã danh mục và truy vấn API danh sách sản phẩm trong danh mục đó để thu thập thông tin về các sản phẩm. Qua quá trình này, chúng ta có thể trích xuất các mã sản phẩm từ các danh mục để thu thập nhiều dữ liệu hơn. Ngoài ra, chúng ta cũng có thể thu thập các bình luận và đánh giá của người dùng về các sản phẩm. Dữ liệu thu thập được có thể được lưu trữ trong một cơ sở dữ liệu hoặc được tiếp tục xử lý theo nhu cầu cụ thể. Công việc này sẽ được mô tả chi tiết hơn trong lớp xử lý và lưu trữ dữ liệu.

Trên đây là quy trình thu thập dữ liệu danh mục từ sàn thương mại điện tử

Tiki.vn. Bằng cách sử dụng các API của Tiki.vn và sử dụng Kafka để truyền dữ liệu, chúng ta có thể thu thập thông tin về danh mục sản phẩm, danh sách sản phẩm và đánh giá từ người dùng. Dữ liệu thu thập được có thể được lưu trữ hoặc tiếp tục xử lý theo nhu cầu cụ thể. Quy trình này mang lại cho chúng ta khả năng thu thập dữ liệu một cách tự động và liên tục từ Tiki.vn, cung cấp một cơ sở dữ liệu đáng tin cậy về danh mục sản phẩm, thông tin chi tiết và đánh giá từ người dùng.

4.3 Lớp xử lý và lưu trữ dữ liệu

Sau khi dữ liệu về mã và tên danh mục được Kafka Producer gửi vào **category_topic**, hệ thống cần tiếp nhận và xử lý dữ liệu này. Một tiến trình ở lớp này có vai trò lắng nghe và đọc các tin nhắn từ **category_topic** thông qua các Kafka Consumer, tiến hành xử lý, tạo các tham số để truy vấn API trích xuất dữ liệu. Sau khi xử lý dữ liệu được thu thập từ các API, hệ thống cần có một nơi lưu trữ dữ liệu đáng tin cậy. Với yêu cầu xử lý dữ liệu theo thời gian thực và khả năng lưu trữ và truy xuất dữ liệu nhanh chóng, chúng ta có thể sử dụng Apache HBase - một hệ thống cơ sở dữ liệu phân tán, cung cấp khả năng đọc/ghi dữ liệu theo thời gian thực, có khả năng mở rộng và xử lý dữ liệu với hiệu suất cao.

4.3.1 Thiết kế cơ sở dữ liệu

Với Apache HBase, chúng ta có thể tổ chức lưu trữ dữ liệu theo mô hình bảng (Table) dựa trên các hàng (Row) và cột (Column). Cụ thể, chúng ta sẽ tổ chức dữ liệu như sau:

Lưu ý: do độ rộng trang quá nhỏ, khi kẻ theo chiều ngang sẽ không thể hiện được hết các cột có trong bảng nếu kẻ theo chiều ngang. Do đó, các phần sau sẽ kẻ bảng tổng hợp các cột theo chiều dọc, các giá trị của Cột sẽ tương ứng với giá trị của khóa chính dữ liệu đó.

a. Bảng chứa dữ liệu danh mục

Đây là bảng chứa thông tin về các danh mục sản phẩm. Bảng này được thiết kế như sau:

- Tên bảng: category
- Khóa chính (Row Key): category_id (Mã danh mục),
- Nhóm cột (Column Family):
 - data: Chứa thông tin về danh mục.
- Cột (Column Qualifier):
 - name: Tên của các danh mục

Khóa chính	Nhóm cột	Cột	Giá trị
category_id	data	name	

Bảng 4.1: Bảng *category*

b. Bảng chứa thông tin sản phẩm

Đây là bảng chứa thông tin về các sản phẩm. Bảng này được thiết kế như sau:

- Tên bảng: product
- Khóa chính: time_product_id (Thời gian thu thập + Mã sản phẩm)
- Nhóm cột:
 - basic_info: Chứa các thông tin cơ bản về sản phẩm.
 - reference: Chứa các thông tin tham chiếu (URL, hình ảnh) của sản phẩm.
 - current_seller: Chứa thông tin về người bán hiện tại của sản phẩm.
 - brand: Chứa thông tin về nhãn hiệu (brand) của sản phẩm.
 - specifications: Chứa thông tin về các đặc điểm (specifications) của sản phẩm.
 - categories: Chứa thông tin về danh mục (category) của sản phẩm.
- Cột: Giá trị của các cột trong nhóm cột tương ứng được lưu trữ như sau:
 - basic_info:
 - * id: Mã sản phẩm.
 - * name: Tên sản phẩm.
 - * price: Giá sản phẩm.
 - * time: Thời gian thu thập sản phẩm.
 - * original_price: Giá gốc sản phẩm.
 - * all_time_quantity_sold: Số lượng sản phẩm đã bán.
 - * rating_average: Điểm đánh giá trung bình của sản phẩm.
 - * review_count: Số lượng đánh giá của sản phẩm.
 - * favourite_count: Số lượt yêu thích của sản phẩm.
 - * day_ago_created: Số ngày tính từ khi sản phẩm được tạo.
 - * description: Mô tả sản phẩm

- reference:
 - * short_url: URL được rút gọn của sản phẩm
 - * thumbnail_url: URL hình ảnh nhỏ của sản phẩm
- current_seller:
 - * id: Mã của người bán hiện tại.
 - * sku: Stock Keeping Unit - Đơn vị lưu kho của sản phẩm từ người bán hiện tại
 - * name: Tên người bán hiện tại
 - * link: Đường dẫn đến người bán hiện tại
 - * logo: Logo của người bán hiện tại
 - * price: Giá sản phẩm từ người bán hiện tại.
 - * product_id: Mã sản phẩm.
 - * store_id: Mã cửa hàng.
 - * is_best_store: Có được đánh giá là cửa hàng tốt nhất hay không.
 - * is_offline_installment_supported: Trạng thái hỗ trợ trả góp offline từ người bán hiện tại.
- brand:
 - * id: Mã của nhãn hiệu.
 - * name: Tên nhãn hiệu.
 - * slug: Slug - Phần cuối cùng thuộc URL đầy đủ của nhãn hiệu.
- specifications:
 - * spec_name:attr_code: Giá trị thuộc tính (attribute) của đặc điểm (specification).
- categories:
 - * id: Mã danh mục của sản phẩm.
 - * name: Tên danh mục của sản phẩm
 - * is_leaf: Có thuộc danh mục con nhỏ nhất hay không.
 - * parent_id: Mã danh mục cha của danh mục.
 - * parent_name: Tên danh mục cha của danh mục.

c. Bảng chứa dữ liệu tổng quan đánh giá trong một sản phẩm:

Đây là bảng chứa thông tin tổng quan nhất về đánh giá của người dùng của từng sản phẩm. Bảng này được thiết kế như sau:

- Tên bảng: `overview_rating`
- Khóa chính: `product_id` (Mã sản phẩm)
- Nhóm cột:
 - `stars`: Chứa thông tin về số lượng đánh giá và tỷ lệ phần trăm của từng số sao.
 - `rating`: Chứa thông tin về điểm đánh giá trung bình.
 - `reviews_count`: Chứa thông tin về tổng số lượt đánh giá.
 - `review_photo`: Chứa thông tin về tổng số lượt đánh giá có hình ảnh.
- Cột: Giá trị của các cột trong nhóm cột tương ứng được lưu trữ như sau:
 - `stars`:
 - * `star:count`: Số lượng đánh giá có số sao `star`.
 - * `star:percent`: Tỷ lệ phần trăm đánh giá có số sao `star`.

Trong đó, *star* đại diện cho số sao mà người dùng đánh giá cho sản phẩm, nhận giá trị từ 1 đến 5.
 - `rating`:
 - * `rating_average`: Điểm đánh giá trung bình.
 - `reviews_count`:
 - * `count`: Tổng số lượt đánh giá.
 - `review_photo`:
 - * `total`: Tổng số lượt đánh giá.
 - * `total_photo`: Tổng số lượt đánh giá có hình ảnh.

Khóa chính	Nhóm cột	Cột	Giá trị
product_id	stars	1:count	
		1:percent	
		2:count	
		2:percent	
		3:count	
		3:percent	
		4:count	
		4:percent	
		5:count	
		5:percent	
	rating	rating_average	
	reviews_count	count	
	review_photo	total	
		total_photo	

Bảng 4.3: Bảng *overview_rating*

d. Bảng chứa dữ liệu đánh giá, bình luận

Đây là bảng chứa thông tin chi tiết của từng bình luận, đánh giá có trong một sản phẩm mà người dùng đánh giá. Bảng này được thiết kế như sau:

- Tên bảng: rating
- Khóa chính: id (Mã đánh giá).
- Nhóm cột:
 - reviewinfo: Chứa các thông tin về đánh giá.
 - images: Chứa thông tin về hình ảnh liên quan đến đánh giá.
 - created_by: Chứa thông tin về người tạo đánh giá.
 - timeline: Chứa thông tin về thời gian của đánh giá.
- Cột:

Giá trị của các cột trong nhóm cột tương ứng được lưu trữ như sau:

- review_info:
 - * title: Tiêu đề đánh giá
 - * content: Nội dung đánh giá
 - * status: Trạng thái của đánh giá
 - * thank_count: Số lượt cảm ơn đánh giá.
 - * score: Điểm đánh giá.

- * new_score: Điểm đánh giá mới.
- * customer_id: Mã khách hàng.
- * comment_count: Số lượng bình luận của đánh giá.
- * rating: Điểm đánh giá.

– images:

- * i:id: Mã hình ảnh.
- * i:full_path: Đường dẫn đến hình ảnh đầy đủ
- * i:status: Trạng thái của hình ảnh

Trong đó, *i* đại diện cho số thứ tự của hình ảnh trong danh sách hình ảnh liên quan đến đánh giá.

– created_by:

- * id: Mã người tạo đánh giá.
- * name: Tên người tạo đánh giá
- * full_name: Họ và tên đầy đủ của người tạo đánh giá
- * joined_time: Thời gian tham gia của người tạo đánh giá
- * region: Vùng/Địa điểm của người tạo đánh giá
- * avatar_url: URL ảnh đại diện của người tạo đánh giá
- * created_time: Thời gian tạo đánh giá
- * group_id: Mã nhóm của người tạo đánh giá.
- * purchased: Trạng thái "đã mua hàng" của người tạo đánh giá.
- * purchased_at: Thời điểm mua hàng của người tạo đánh giá.
- * total_review: Tổng số đánh giá của người tạo đánh giá.
- * total_thank: Tổng số cảm ơn của người tạo đánh giá.

– timeline:

- * review_created_date: Ngày tạo đánh giá
- * delivery_date: Ngày giao hàng
- * current_date: Ngày hiện tại
- * content: Nội dung thời gian
- * explain: Giải thích

Khóa chính	Nhóm cột	Cột	Giá trị
id	review_info	title	
		content	
		status	
		thank_count	
		score	
		new_score	
		customer_id	
		comment_count	
		rating	
	images	image_1:id	
		image_1:full_path	
		image_1:status	
		image_2:id	
		...	
	created_by	id	
		name	
		full_name	
		joined_time	
		region	
		avatar_url	
		created_time	
		group_id	
		purchased	
		purchased_at	
		total_review	
		total_thank	
	timeline	review_created_date	
		delivery_date	
		current_date	
		content	
		explain	

Bảng 4.4: Bảng *rating*

Mỗi bảng có thể được phân chia thành các phân vùng (Regions) để phân tải dữ liệu và tăng hiệu suất truy xuất. Việc phân vùng dữ liệu trong HBase có thể dựa trên khóa chính (chẳng hạn như category_id, product_id) hoặc tùy thuộc vào yêu cầu cụ thể của hệ thống.

Bên cạnh việc tổ chức dữ liệu trong các bảng, chúng ta cũng có thể sử dụng Apache HBase để xử lý các truy vấn và tìm kiếm nhanh chóng trên dữ liệu. HBase hỗ trợ các phương thức truy vấn linh hoạt như truy vấn theo khóa chính, truy vấn theo phạm vi, hoặc truy vấn sử dụng các bộ lọc (Filter) để lấy ra các dữ liệu phù hợp.

Tổ chức và thiết kế các bảng trong Apache HBase phụ thuộc vào yêu cầu cụ thể của hệ thống và các loại truy vấn dự kiến. Quan trọng nhất là thiết kế sao cho phù hợp với mô hình dữ liệu và đảm bảo hiệu suất truy xuất dữ liệu cao nhất.

4.3.2 Quy trình xử lý và lưu trữ dữ liệu

Dựa trên các API được Tiki.vn cung cấp, lớp thu thập dữ liệu đã trích xuất thông tin về danh mục sản phẩm (mã danh mục và tên danh mục) và gửi chúng vào topic **category_topic** trong hệ thống Kafka. Sử dụng dữ liệu từ topic này, cùng với sự lựa chọn của chúng ta là Apache HBase là công nghệ lưu trữ dữ liệu, chúng ta thiết kế quy trình xử lý và lưu trữ dữ liệu như sau:

- Lưu danh sách danh mục: Chúng ta sẽ triển khai một Kafka Consumer để lắng nghe dữ liệu được gửi vào **category_topic**. Mỗi khi có tin nhắn mới chứa thông tin về mã danh mục và tên danh mục, Kafka Consumer sẽ ghi nhận, sau đó thực hiện lưu trữ dữ liệu này vào bảng **category** trong Apache HBase.

Bằng cách này, chúng ta có thể duy trì và cập nhật thông tin về các danh mục sản phẩm từ Tiki.vn trong hệ thống lưu trữ đáng tin cậy của mình, cho phép chúng ta duy trì một danh sách các danh mục sản phẩm cập nhật và dễ dàng truy xuất thông tin về các danh mục đó từ Apache HBase.

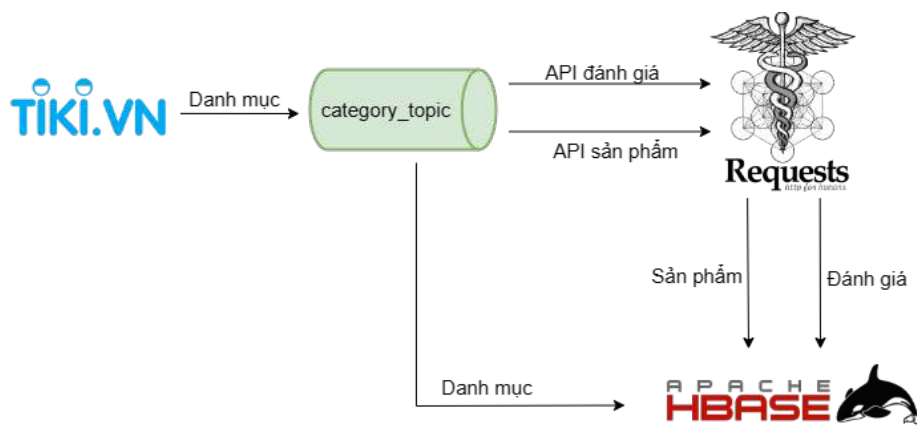
- Truy vấn API chứa danh sách sản phẩm: Sử dụng một Kafka Consumer khác, truy xuất ra mã danh mục từ dữ liệu được gửi vào topic **category_topic**. Sau đó, chúng ta sẽ sử dụng mã danh mục làm tham số đầu vào cho việc truy vấn API danh sách sản phẩm trong từng danh mục. Do các thông tin sản phẩm trong danh sách sản phẩm còn thiếu sót, nên chúng ta sẽ cần sử dụng API chứa dữ liệu của từng sản phẩm để có thêm thông tin với tham số đầu vào là mã sản phẩm có được từ danh sách sản phẩm. Hoặc nếu yêu cầu dữ liệu không cần quá nhiều trường thông tin, chúng ta có thể dừng ngay tại bước này, thu thập thông tin các sản phẩm được hiển thị trong danh sách.
- Truy vấn API chứa thông tin sản phẩm: Khi đã trích xuất được mã sản phẩm có từ danh sách sản phẩm trong mỗi danh mục. với mỗi mã sản phẩm, chúng ta sẽ tiến hành truy vấn vào từng API tương ứng mỗi sản phẩm để thu thập dữ liệu. Sau khi có được đầy đủ thông tin về sản phẩm, chúng ta cần tiến hành tối ưu dữ liệu để phục vụ quá trình phân tích và đánh giá, đồng thời giảm thiểu việc sử dụng bộ nhớ cho việc lưu trữ. Để làm điều này, chúng ta sẽ cắt bỏ các trường thông tin không cần thiết và xử lý các trường đặc biệt trong các trường còn lại. Sau đó, dữ liệu được lưu trữ trong bảng **product** sẽ được tối ưu và chuẩn hóa để đảm bảo hiệu quả trong việc thực hiện các phân tích và đánh giá sau này.

Điều này cho phép chúng ta duy trì một bộ sưu tập đáng tin cậy về các sản phẩm với đầy đủ thông tin để tiếp tục thực hiện các tác vụ xử lý dữ liệu, phân tích, và truy vấn một cách hiệu quả. Quá trình này sẽ cho phép chúng ta tận dụng tối đa dữ liệu đã thu thập được từ Tiki.vn và lưu trữ chúng một cách cấu trúc và hiệu quả trong HBase để phục vụ cho các mục đích kinh doanh và nghiên cứu sau này.

- Truy vấn API chứa đánh giá và bình luận: Khi chúng ta nhận được một mã sản phẩm từ Kafka Consumer, chúng ta có thể sử dụng mã này làm tham số đầu vào để truy vấn API chứa thông tin về đánh giá và bình luận của người dùng cho sản phẩm tương ứng. Sau khi thu thập dữ liệu từ API đánh giá và bình luận, chúng ta tiến hành xử lý dữ liệu, loại bỏ thông tin không cần thiết và giữ lại các trường quan trọng. Cuối cùng, chúng ta lưu trữ dữ liệu này vào bảng **rating** trong Apache HBase.

Việc lưu trữ dữ liệu đánh giá và bình luận trong HBase cho phép chúng ta xây dựng một nguồn dữ liệu phong phú về phản hồi của người dùng đối với các sản phẩm trên Tiki.vn. Điều này giúp chúng ta có thể xử lý và phân tích dữ liệu này để hiểu thêm về ý kiến và sự hài lòng của khách hàng về các sản phẩm.

Dưới đây là hình mô tả luồng thu thập dữ liệu:



Hình 4.3: Thu thập dữ liệu

Các quá trình lưu dữ liệu danh mục, truy vấn API có thể được xử lý song song, đảm bảo rằng dữ liệu được thu thập và xử lý một cách đồng thời và đầy đủ. Trong mỗi quá trình, việc truy vấn API được thực hiện bằng thư viện requests của Python, và sau đó kết quả trả về được sử dụng để bóc tách thông tin cần thiết.

Sau khi một dữ liệu bất kỳ được lưu vào HBase, nó cũng sẽ được biến đổi, xử lý

các trường cần thiết để lưu trong Elasticsearch và gửi vào một Kafka Producer để lớp phục vụ có thể lắng nghe, truy xuất và lưu dữ liệu.

Điều này cho phép chúng ta tận dụng tối đa khả năng xử lý đa luồng của hệ thống và giảm thời gian hoàn thành quá trình thu thập dữ liệu, đảm bảo dữ liệu thu thập được từ Tiki.vn là đầy đủ và chính xác.

Qua quá trình trên, chúng ta đã thành công trong việc tự động và liên tục thu thập dữ liệu từ Tiki.vn và lưu trữ nó vào các bảng tương ứng trong Apache HBase. Nhờ vào HBase, dữ liệu được tổ chức và lưu trữ một cách hiệu quả, cho phép thực hiện các truy vấn và phân tích dữ liệu theo các yêu cầu cụ thể của hệ thống và ứng dụng.

4.4 Lớp phục vụ

Lớp phục vụ - Serving Layer trong hệ thống này đóng vai trò quan trọng trong việc cung cấp dữ liệu từ hệ thống lưu trữ Apache HBase cho các ứng dụng, người dùng và hệ thống khác. Bằng cách sử dụng lớp phục vụ, chúng ta có thể truy xuất dữ liệu từ HBase một cách hiệu quả và đáp ứng nhanh chóng các yêu cầu của người dùng cuối và ứng dụng.

4.4.1 Bài toán phân tích dữ liệu

Một trong những bài toán phân tích dữ liệu quan trọng mà lớp phục vụ có thể giải quyết là phân loại cảm xúc từ các bình luận của người dùng. Dựa trên dữ liệu thu thập được từ bảng **rating** trong HBase, chúng ta có thể áp dụng các kỹ thuật phân loại và thuật toán Random Forest để xác định cảm xúc tích cực, tiêu cực hoặc trung lập của các bình luận. Kết quả phân loại này có thể được sử dụng để đánh giá sự hài lòng của người dùng và theo dõi đánh giá chất lượng sản phẩm.

Một bài toán khác có thể được xây dựng dựa trên tập dữ liệu được lưu trong HBase là xây dựng hệ thống gợi ý sản phẩm tương tự dựa trên dữ liệu thu thập từ HBase. Bằng cách áp dụng các thuật toán gợi ý như kỹ thuật TF - IDF, Collaborative Filtering hoặc Content-Based Filtering, chúng ta có thể tạo ra các gợi ý sản phẩm dựa trên sự tương đồng giữa các sản phẩm hoặc sở thích của người dùng. Kết quả gợi ý này có thể được sử dụng để cung cấp trải nghiệm mua sắm cá nhân hóa và tăng khả năng tìm kiếm sản phẩm phù hợp cho người dùng.

Chương sau sẽ đề cập đến hai bài toán một cách chi tiết.

4.4.2 Trực quan hóa dữ liệu

Bên cạnh việc cung cấp dữ liệu, lớp phục vụ cũng có thể hỗ trợ trực quan hóa dữ liệu bằng việc tích hợp Elasticsearch và Kibana.

Elasticsearch là một công cụ tìm kiếm và phân tích dữ liệu phân tán, cho phép

tìm kiếm và truy xuất dữ liệu nhanh chóng. Kibana là một giao diện người dùng để trực quan hóa và khám phá dữ liệu. Bằng cách kết hợp Elasticsearch và Kibana với dữ liệu trong HBase, chúng ta có thể xây dựng các biểu đồ, đồ thị và bảng tin trực quan để hiển thị thông tin dữ liệu một cách dễ hiểu và tương tác. Điều này giúp người dùng và các cửa hàng, doanh nghiệp có cái nhìn tổng quan về dữ liệu và phân tích dữ liệu một cách rõ ràng.

Để làm được điều đó, Elasticsearch phải đồng bộ hóa, trích xuất được dữ liệu từ HBase một cách liên tục, đảm bảo dữ liệu được cập nhật đầy đủ vào Elasticsearch. Như đã giới thiệu ở mục 4.3.2 (Quy trình xử lý và lưu trữ giữ liệu), sau khi một dữ liệu được lưu vào HBase, thì dữ liệu sẽ được biến đổi thích hợp để lưu vào Elasticsearch. Khi đã xử lý xong, dữ liệu sẽ được gửi vào các topic Kafka tương ứng với loại dữ liệu.

Sau khi lắng nghe và lấy được từ Kafka, dữ liệu sẽ giải mã, xử lý và được đưa vào các chỉ mục tương ứng có trong Elasticsearch để lưu trữ, phục vụ việc trực quan hóa và phân tích dữ liệu.

Việc lắng nghe bằng Kafka Consumer và thực hiện tiến trình xử lý dữ liệu thu thập được một cách liên tục sẽ đảm bảo dữ liệu mới được cập nhật kịp thời, đầy đủ vào Elasticsearch, phục vụ yêu cầu phân tích, cũng như trực quan hóa dữ liệu một cách nhanh chóng, đầy đủ.

Thông tin chi tiết về vấn đề phân tích, trực quan hóa dữ liệu sẽ được cung cấp trong phần tiếp theo.

4.5 Lớp trực quan hóa dữ liệu

4.5.1 Mục đích

Lớp trực quan hóa dữ liệu trong hệ thống được tạo ra với mục đích chính là giúp người dùng và các nhà quản lý có cái nhìn tổng quan và hiểu rõ hơn về thông tin liên quan đến danh sách sản phẩm, bình luận và đánh giá của người dùng trên Tiki.vn. Điều này nhằm giúp họ nắm bắt các xu hướng, mô hình tiêu thụ của người dùng và đánh giá chất lượng sản phẩm một cách dễ dàng và hiệu quả.

Trực quan hóa dữ liệu trong hệ thống này sử dụng công nghệ Kibana để trực quan hóa các biểu đồ, đồ thị và bảng tin trực quan để thể hiện dữ liệu một cách rõ ràng, trực quan nhất. Nhờ vào việc biểu diễn thông tin một cách sinh động, người dùng có thể nhanh chóng nhận ra các mẫu số và quy luật trong cung cầu sản phẩm, thị trường và hành vi tiêu thụ. Điều này giúp họ đưa ra những quyết định chiến lược và cải thiện hiệu quả kinh doanh.

Ngoài ra, các cửa hàng và các doanh nghiệp cũng được hỗ trợ bởi lớp trực quan

hóa dữ liệu để theo dõi hoạt động của hệ thống, hiểu rõ hơn về sự phản hồi từ người dùng và tình hình đánh giá sản phẩm. Điều này giúp họ đưa ra các biện pháp tối ưu hóa và tăng cường trải nghiệm người dùng.

Tóm lại, mục đích chính của việc sử dụng lớp trực quan hóa dữ liệu trong hệ thống là cung cấp cái nhìn tổng quan và giúp người dùng cũng như các cửa hàng, doanh nghiệp hiểu rõ hơn về dữ liệu, xu hướng và mô hình tiêu thụ trên Tiki.vn. Nhờ đó, họ có thể đưa ra các quyết định thông minh và tối ưu hóa hoạt động kinh doanh một cách hiệu quả.

4.5.2 Hệ thống trực quan hóa

Hệ thống trực quan hóa dữ liệu - Visualization Layer trong hệ thống này được xây dựng bằng sử dụng tích hợp Elasticsearch và Kibana. Elasticsearch đóng vai trò là cơ sở dữ liệu phân tán, nơi lưu trữ dữ liệu từ HBase và cho phép tìm kiếm, truy xuất nhanh chóng. Kibana là một công cụ trực quan hóa dữ liệu mạnh mẽ, cung cấp giao diện người dùng để sử dụng để tạo ra các biểu đồ, đồ thị và bảng tin trực quan.

Hệ thống này sử dụng một quy trình đơn giản (được nêu rõ ở mục 4.4.2 - Trực quan hóa dữ liệu) để xử lý dữ liệu từ HBase và đồng bộ hóa với Elasticsearch. Dữ liệu sau khi được xử lý sẽ được lưu trữ trong Elasticsearch, tạo ra một chỉ mục dữ liệu để truy vấn và phân tích.

Kibana là giao diện người dùng chính trong hệ thống này. Nó kết nối với Elasticsearch và cung cấp các công cụ trực quan để hiển thị dữ liệu một cách trực quan và trực tiếp. Các biểu đồ thống kê, biểu đồ đường, biểu đồ cột, biểu đồ tròn và các biểu đồ khác được tạo ra từ dữ liệu, giúp hiển thị các xu hướng, mô hình và sự phụ thuộc trong dữ liệu.

Với Kibana, người dùng có thể thực hiện tìm kiếm và truy xuất dữ liệu một cách hiệu quả. Ngoài ra, Kibana còn cung cấp khả năng tạo báo cáo, gửi thông báo và theo dõi các chỉ số quan trọng. Từ đó, người dùng có thể theo dõi và phân tích dữ liệu một cách chi tiết và thuận tiện.

Thông qua hệ thống trực quan hóa dữ liệu, người dùng và các cửa hàng, doanh nghiệp có thể thấy được các thông tin quan trọng về danh sách sản phẩm, phản hồi của người dùng và xu hướng tiêu thụ. Điều này giúp họ đưa ra các quyết định và chiến lược dựa trên dữ liệu thực tế và cải thiện trải nghiệm của người dùng trên Tiki.vn.

4.6 Lớp giám sát hệ thống

4.6.1 Tính cấp thiết

Việc triển khai lớp giám sát hệ thống là cực kỳ cấp thiết trong môi trường sản xuất. Các thành phần như Kafka, HBase, Elasticsearch và Kibana đóng vai trò quan trọng trong việc xử lý dữ liệu, lưu trữ, tìm kiếm và trực quan hóa dữ liệu. Khi những thành phần này gặp sự cố hoặc không hoạt động ổn định, có thể gây ra các vấn đề nghiêm trọng cho hệ thống và ảnh hưởng đến hiệu suất và độ tin cậy. Xây dựng một lớp giám sát hệ thống là rất cấp thiết vì nó giúp chúng ta:

- **Phát hiện sớm sự cố:** Việc giám sát liên tục giúp phát hiện sớm các sự cố và ngăn chặn những vấn đề tiềm ẩn trước khi chúng trở thành những vấn đề lớn gây gián đoạn hoạt động của hệ thống.
- **Đảm bảo hiệu suất và sự tin cậy:** Khi có sự cố xảy ra, lớp giám sát sẽ cung cấp thông báo cho phép chúng ta can thiệp kịp thời để khắc phục vấn đề, tránh ảnh hưởng đến hoạt động kinh doanh, đảm bảo hiệu suất tối ưu và đáng tin cậy.
- **Quản lý tài nguyên:** Lớp giám sát cung cấp thông tin về việc sử dụng tài nguyên của các thành phần (Kafka, HBase, Elasticsearch và Kibana). Điều này giúp phân tích và đánh giá hiệu suất sử dụng tài nguyên, tối ưu hóa cấu hình và quản lý tài nguyên hiệu quả.
- **Đánh giá tải:** Giám sát giúp chúng ta đánh giá tải công việc trên hệ thống. Từ đó, chúng ta có thể mở rộng hoặc điều chỉnh cấu hình để đáp ứng yêu cầu vận hành của hệ thống.

Tóm lại, lớp giám sát hệ thống này cung cấp khả năng theo dõi, thu thập và hiển thị số liệu từ các nguồn khác nhau một cách hiệu quả, giúp chúng ta phát hiện sự cố sớm, tối ưu hóa hiệu suất, từ đó tăng tính sẵn sàng và đáng tin cậy của hệ thống, đảm bảo hệ thống hoạt động một cách liên tục và ổn định.

4.6.2 Quy trình giám sát hệ thống

Lớp giám sát hệ thống là một phần quan trọng trong việc quản lý và đảm bảo sự ổn định và hiệu suất của hệ thống. Trong môi trường hiện đại, có nhiều công cụ giám sát mạnh mẽ và phổ biến, và trong trường hợp này, chúng ta sẽ sử dụng Grafana và Prometheus để giám sát các thành phần có trong hệ thống như Kafka, HBase, Elasticsearch và Kibana.

Quy trình triển khai lớp giám sát hệ thống bằng Grafana và Prometheus cho hệ thống bao gồm các bước sau:

- (i) Cài đặt và cấu hình Grafana và Prometheus: Bắt đầu bằng việc cài đặt và cấu

hình Grafana và Prometheus vào hệ thống.

- (ii) Cấu hình giám sát các thành phần: Tiếp theo, cấu hình Prometheus để giám sát Kafka, HBase, Elasticsearch và Kibana. Điều này bao gồm việc xác định các điểm cuối (endpoint) hoặc công cụ trích xuất dữ liệu (exporter) để thu thập các chỉ số từ các thành phần này.
- (iii) Tạo bảng điều khiển (dashboard) trong Grafana: Sử dụng Grafana, tạo các bảng điều khiển để hiển thị các chỉ số được thu thập từ Prometheus. Bạn có thể tạo biểu đồ, đồ thị, và báo cáo để theo dõi hiệu suất và trạng thái của các thành phần.
- (iv) Theo dõi và quản lý hệ thống: Theo dõi các bảng điều khiển trong Grafana và theo dõi các thông báo cảnh báo để giám sát trạng thái và hiệu suất của các thành phần Kafka, HBase, Elasticsearch và Kibana. Khi phát hiện vấn đề, xử lý chúng nhanh chóng để đảm bảo tính khả dụng và hiệu suất của hệ thống.

Tóm lại, việc sử dụng Grafana và Prometheus để giám sát Kafka, HBase, Elasticsearch và Kibana giúp chúng ta đảm bảo tính khả dụng và hiệu suất của hệ thống, phát hiện và xử lý các vấn đề, tối ưu hóa hiệu suất và đáp ứng yêu cầu người dùng.

4.7 Kết luận

Chúng ta đã thiết kế và xây dựng thành công hệ thống tổ chức và xử lý dữ liệu từ Tiki.vn. Hệ thống này thu thập, lưu trữ và xử lý thông tin về danh sách sản phẩm, bình luận và đánh giá từ người dùng trên Tiki.vn sử dụng Apache HBase. Đồng thời, chúng ta đã tạo ra một giao diện trực quan để hiển thị và trực quan hóa thông tin liên quan đến dữ liệu này, giúp người dùng và nhà quản lý có cái nhìn tổng quan về xu hướng và mô hình tiêu thụ của người dùng. Cuối cùng, chúng ta đã xây dựng một hệ thống giám sát để theo dõi hoạt động của hệ thống và phát hiện sớm các vấn đề.

Sau khi hoàn thành thiết kế hệ thống, chúng ta có thể tự hỏi liệu dữ liệu thu thập và xử lý có thể được áp dụng vào những bài toán nào. Trong phần tiếp theo, chúng ta sẽ giới thiệu một số bài toán điển hình mà dữ liệu này có thể được sử dụng.

Khóa chính	Nhóm cột	Cột	Giá trị
time_product_id	basic_info	id	
		name	
		price	
		time	
		original_price	
		all_time_quantity_sold	
		rating_average	
		review_count	
		favourite_count	
		day_ago_created	
		description	
	reference	short_url	
		thumbnail_url	
	current_seller	id	
		sku	
		name	
		link	
		logo	
		price	
		product_id	
		store_id	
		is_best_store	
		is_offline_installment_supported	
	brand	id	
		name	
		slug	
	specifications	spec_name:attr_code	
	categories	id	
		name	
		is_leaf	
		parent_id	
		parent_name	

Bảng 4.2: Bảng *product*

CHƯƠNG 5. MỘT SỐ BÀI TOÁN PHÂN TÍCH DỮ LIỆU

Sau khi thu thập và lưu trữ dữ liệu từ sàn thương mại điện tử Tiki, chúng ta có thể tiến hành phân tích và xử lý dữ liệu cho các yêu cầu cụ thể. Dưới đây là một số bài toán phổ biến có thể được đặt ra:

5.1 Bài toán phân loại bình luận sản phẩm

5.1.1 Tính cấp thiết của bài toán

Trong lĩnh vực thương mại điện tử, hàng ngày có hàng triệu bình luận từ người dùng về các sản phẩm. Phân tích những bình luận này để đánh giá tính tích cực hay tiêu cực của chúng là một công việc quan trọng, giúp các doanh nghiệp, các cửa hàng hiểu về chất lượng sản phẩm và nhận thức được tâm lý của khách hàng. Điều này cho phép họ thực hiện các điều chỉnh phù hợp để nâng cao hoạt động kinh doanh.

Tiki.vn và các sàn thương mại điện tử khác đều cung cấp tính năng bình luận và đánh giá từ người dùng sau khi mua hàng, giúp khách hàng đánh giá chất lượng sản phẩm trước khi mua. Tuy nhiên, do sơ suất, có thể xảy ra trường hợp người dùng đánh giá 1 sao hoặc 2 sao nhưng thực tế muốn đánh giá cao hơn như 4 sao hoặc 5 sao. Điều này dẫn đến giảm điểm đánh giá trung bình của sản phẩm, ảnh hưởng đến quyết định mua hàng của người khác và đánh giá chung về sản phẩm, gây khó khăn cho doanh nghiệp và cửa hàng trong việc cải thiện sản phẩm.

Để giảm nguy cơ này, chúng ta cần một hệ thống kiểm tra lại mức độ đánh giá từ người dùng để có thể đánh giá sản phẩm một cách công bằng nhất. Với số lượng bình luận lớn, phân tích bằng phương pháp truyền thống là không khả thi. Đó là lúc các mô hình học máy chứng tỏ sức mạnh của mình. Các mô hình học máy có khả năng phân loại bình luận của khách hàng về các sản phẩm đã mua và sử dụng, giúp chủ doanh nghiệp, cửa hàng hiểu được mức độ sản phẩm tốt, xác định các vấn đề cần khắc phục khi đưa sản phẩm ra thị trường.

Trong phần tiếp theo, chúng ta sẽ nghiên cứu và tìm hiểu thuật toán được sử dụng để giải quyết bài toán trên.

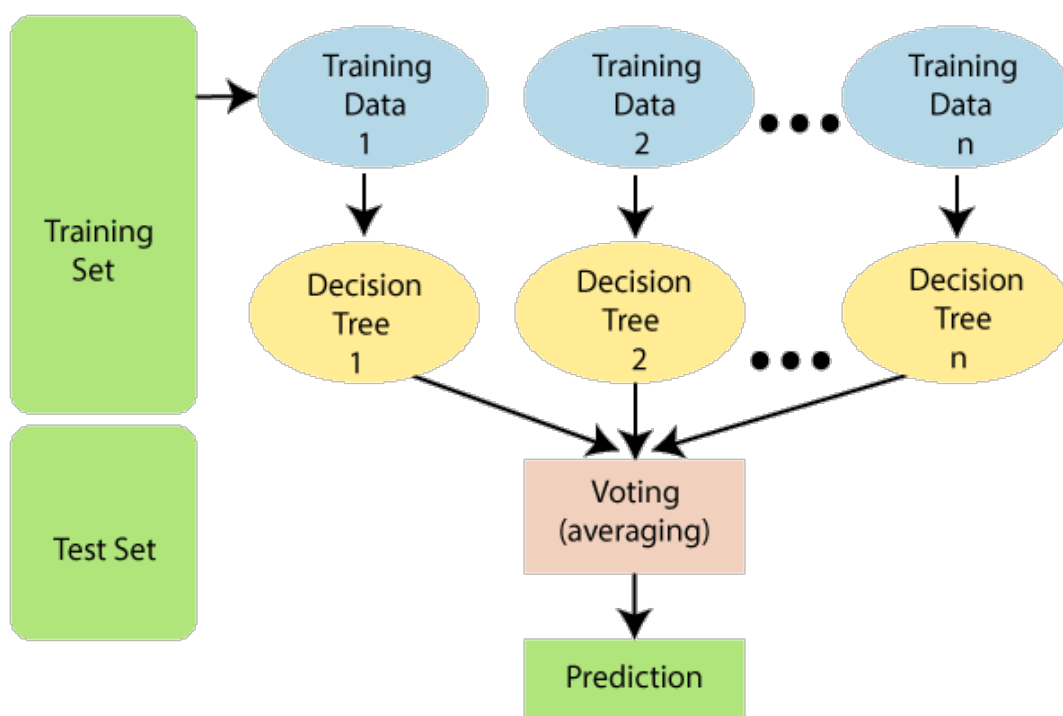
5.1.2 Thuật toán Random Forest

Trong việc giải quyết bài toán phân loại bình luận, có nhiều thuật toán khả dụng để xử lý. Tuy nhiên, chúng ta đã lựa chọn sử dụng thuật toán Random Forest để giải quyết vấn đề này, một thuật toán học máy được sử dụng rộng rãi trong việc phân loại và dự đoán. Nó là một dạng Ensemble Learning, tức là kết hợp nhiều mô hình học máy đơn giản để tạo ra một mô hình mạnh mẽ hơn.

Random là ngẫu nhiên, Forest là rừng, thuật toán Random Forest hoạt động bằng cách tiến hành xây dựng nhiều cây quyết định bằng thuật toán Decision Tree để tạo thành một rừng cây (Forest), tuy nhiên mỗi cây quyết định sẽ được xây dựng dựa trên một tập con ngẫu nhiên của dữ liệu huấn luyện (Random) được tạo ra bằng phương pháp lấy mẫu có hoàn lại. Điều này đảm bảo rằng mỗi cây quyết định trong Random Forest được huấn luyện trên một tập dữ liệu độc lập.

Lấy mẫu có hoàn lại - Bootstrap aggregating (hay Bagging) là việc lấy ngẫu nhiên một mẫu từ tập dữ liệu gốc và đưa vào tập dữ liệu con mới, sau đó trả lại mẫu đã lấy vào tập dữ liệu gốc. Lặp lại các hành động trên cho đến khi đủ số lượng mẫu cần lấy trong tập dữ liệu con mới. Vì mẫu đã lấy được trả lại tập dữ liệu gốc, do đó một dữ liệu có thể lặp lại nhiều lần trong một tập dữ liệu con mới.

Dưới đây là hình mô tả cách hoạt động của thuật toán Random Forest:



Hình 5.1: Thuật toán Random Forest

Nguồn: javatpoint, "Random Forest Algorithm"

Cụ thể:

- (i) Chọn các mẫu ngẫu nhiên từ tập dữ liệu đã cho: Từ tập dữ liệu ban đầu, chọn ngẫu nhiên một số mẫu từ tập dữ liệu ban đầu để xây dựng cây quyết định.
- (ii) Xây dựng từng cây quyết định và dự đoán mỗi cây: Thuật toán xây dựng từng cây quyết định và dự đoán mỗi cây độc lập. Mỗi cây sử dụng một phần dữ liệu và một phần thuộc tính ngẫu nhiên từ tập dữ liệu ban đầu. Sau đó, cây quyết

định dự đoán kết quả cho từng mẫu trong tập dữ liệu.

- (iii) **Bỏ phiếu:** Khi mỗi cây quyết định đã dự đoán kết quả, thuật toán sẽ "bỏ phiếu" để xác định kết quả dự đoán cuối cùng bằng cách sử dụng đa số phiếu (majority voting), tức là kết quả dự đoán cuối cùng là kết quả xuất hiện nhiều nhất trong tất cả các cây quyết định.
- (iv) **Dự đoán cuối cùng:** Cuối cùng, thuật toán sẽ chọn kết quả dự đoán là kết quả bỏ phiếu, được sử dụng để đưa ra dự đoán cho tập dữ liệu mới hoặc tập dữ liệu kiểm tra.

Tóm lại, Random Forest là một thuật toán mạnh mẽ và đáng tin cậy, hoạt động dựa trên việc xây dựng nhiều cây quyết định độc lập và kết hợp kết quả từ các cây này để đưa ra dự đoán cuối cùng. Qua các bước trên, Random Forest có khả năng tránh overfitting và giảm sự biến động, đồng thời tạo ra mô hình có khả năng dự đoán tốt trên dữ liệu mới.

5.1.3 Xây dựng thuật toán Random Forest

a. Tiền xử lý dữ liệu

Dữ liệu cho bài toán phân loại bình luận sản phẩm được thu thập từ các bình luận của người dùng đối với các sản phẩm có mặt trên sàn thương mại điện tử Tiki. Dữ liệu bao gồm nội dung bình luận, đánh giá của người dùng về sản phẩm và số sao đánh giá, biểu thị sự hài lòng của khách hàng đối với sản phẩm. Số sao được đánh giá từ 1 đến 5 sao.

Việc phân tích số sao một cách chuẩn xác không phải dễ, do đó để thống nhất và phân loại chính xác hơn, các đánh giá sẽ được gán nhãn với số sao tương ứng như sau:

- 1, 2 sao: "Negative" - Bình luận tiêu cực.
- 3 sao: "Neutral" - Bình luận mang tính trung tính.
- 4, 5 sao: "Positive" - Bình luận tích cực.

Việc thực hiện việc gán nhãn như trên giúp phân loại các bình luận vào các nhóm một cách chính xác và dễ hiểu hơn.

Ngoài ra, để tiền xử lý nội dung đánh giá (Content), chúng ta thực hiện các bước sau:

- i. Đầu tiên, kết hợp tiêu đề và nội dung làm nội dung bình luận. Sau đó tiến hành loại bỏ các bình luận có nội dung và số sao bình luận giống nhau, chỉ giữ lại 1 bình luận duy nhất.

- ii. Loại bỏ các dấu cách dư thừa trong văn bản, thay thế dấu cách liên tiếp bằng một dấu cách duy nhất.
- iii. Loại bỏ tất cả các ký tự đặc biệt trong văn bản, bao gồm cả dấu câu và ký tự không phải chữ và số. Những ký tự này sẽ được thay thế bằng một dấu cách.
- iv. Chuyển toàn bộ chữ hoa trong văn bản thành chữ thường.
- v. Loại bỏ các ký tự lặp lại liên tiếp nhiều lần trong từ.
- vi. Thay thế các từ viết tắt hoặc từ khóa tiếng Anh như "ok", "oki", "good",... bằng từ "hài lòng".
- vii. Sử dụng **ViTokenizer** của thư viện **pyvi**¹ để tách từ, loại bỏ các stop-word (các từ không mang ý nghĩa) và tách kết quả thành danh sách các từ dựa trên khoảng trắng.

Sau khi hoàn thành các bước tiền xử lý, chúng ta tiến hành xây dựng tập vector đặc trưng cho việc phân loại bình luận. Dữ liệu được chia ngẫu nhiên thành hai phần: Tập huấn luyện (Training Set) – Tập kiểm thử (Testing Set) (70%–30%).

b. Xây dựng mô hình phân loại

Sau khi tiền xử lý dữ liệu để có dữ liệu sử dụng cho bài toán, chúng ta sẽ thực hiện xây dựng mô hình phân loại theo các bước sau:

- i. Tiền xử lý dữ liệu: Tiến hành tiền xử lý dữ liệu như trên để tách từ và trích xuất các từ từ nội dung bình luận. Sau đó, loại bỏ stopwords từ dữ liệu sử dụng StopWordsRemover và file stopwords dành cho tiếng Việt có sẵn.
- ii. Xây dựng vector đặc trưng: Sử dụng mô hình Word2Vec để chuyển các từ thành các vector đặc trưng.
- iii. Chuyển đổi nhãn: Dùng StringIndexer để chuyển đổi nhãn phân loại "Negative", "Neutral" và "Positive" thành số.
- iv. Chuyển đổi đặc trưng (features): Dùng VectorIndexer để chuyển đổi các đặc trưng từ dạng danh sách các từ thành dạng vector số.
- v. Xây dựng mô hình Random Forest: Sử dụng RandomForestClassifier để xây dựng mô hình phân loại dựa trên các đặc trưng đã được tạo ra từ bước trước.
- vi. Đánh giá mô hình: Sử dụng tập kiểm thử để đánh giá hiệu suất của mô

¹Python Vietnamese Core NLP Toolkit

hình thông qua độ đo Accuracy. Kết quả Accuracy và Test Error được in ra màn hình.

vii. Lưu mô hình: Lưu lại mô hình huấn luyện để sử dụng sau này.

Ngoài ra, chúng ta sử dụng `IndexToString` để chuyển đổi các số nhãn dự đoán thành nhãn văn bản tương ứng, các số nhãn được chuyển đổi trở lại các nhãn ban đầu "Negative", "Neutral" và "Positive", giúp cho việc đánh giá và hiểu kết quả của mô hình phân loại trở nên dễ dàng hơn.

Tóm lại, đoạn mã trên thực hiện quá trình huấn luyện mô hình phân loại bình luận sản phẩm bằng thuật toán Random Forest và lưu các mô hình đã huấn luyện vào các tệp riêng biệt. Quá trình này giúp ta lựa chọn mô hình phù hợp và đánh giá hiệu suất của mô hình trên dữ liệu kiểm tra.

5.1.4 Tại sao thuật toán Random Forest tốt

Trong thuật toán Decision Tree, nếu cho phép cây quyết định có độ sâu tùy ý, có thể dẫn đến hiện tượng overfitting, trong đó cây sẽ phân loại đúng hết các điểm dữ liệu trong tập huấn luyện, nhưng lại không thể tổng quát hoá tốt cho dữ liệu mới. Điều này dẫn đến mô hình có high variance và có khả năng dự đoán kém trên tập kiểm tra hoặc tập validation.

Để giải quyết vấn đề này, thuật toán Random Forest ra đời. Random Forest bao gồm nhiều cây quyết định độc lập, mỗi cây quyết định đều có những yếu tố ngẫu nhiên:

- Lấy ngẫu nhiên dữ liệu để xây dựng cây quyết định.
- Lấy ngẫu nhiên các thuộc tính để xây dựng cây quyết định.

Điều này giúp tránh hiện tượng quá khớp (overfitting) và giảm sự biến động (variance) trong mô hình.

Dù mỗi cây quyết định trong Random Forest có thể dự đoán không tốt độc lập, nhưng thông tin từ các cây quyết định được tổng hợp lại để đưa ra dự đoán cuối cùng. Việc này giúp tận dụng sự đa dạng và "trí tuệ đám đông" (James Surowiecki, *The Wisdom of Crowds*, 2004) để cung cấp kết quả dự đoán tốt hơn. Thông tin từ các cây sẽ bổ sung cho nhau và giúp mô hình Random Forest có low bias và low variance, tức là mô hình có khả năng tổng quát hóa tốt và dự đoán chính xác trên dữ liệu mới.

Một ví dụ đơn giản để hiểu ý tưởng này là khi chúng ta đọc các đánh giá về một sản phẩm trên một trang thương mại điện tử Tiki.vn. Nếu chỉ đọc một đánh giá duy nhất, chúng ta có thể gặp phải ý kiến chủ quan, đánh giá không thể nói hết về chất

lượng sản phẩm. Nhưng khi đọc tất cả các đánh giá và kết hợp thông tin từ đa dạng nguồn, chúng ta có cái nhìn tổng quan và khả năng đưa ra quyết định cuối cùng chính xác hơn.

Tổng kết, thuật toán Random Forest là một phương pháp mạnh mẽ để giảm overfitting và cải thiện hiệu suất dự đoán. Bằng cách kết hợp các cây quyết định độc lập và sử dụng nguyên lý "trí tuệ đám đông", Random Forest giúp mô hình có khả năng tổng quát hoá tốt và dự đoán chính xác trên dữ liệu mới.

5.2 Bài toán gợi ý sản phẩm

5.2.1 Tính cấp thiết

Gợi ý sản phẩm đóng vai trò quan trọng trong việc tăng doanh số bán hàng và cải thiện trải nghiệm mua sắm của khách hàng. Thông qua việc gợi ý các sản phẩm phù hợp với từng khách hàng, doanh nghiệp có thể tăng tính hiệu quả trong việc tiếp cận và thu hút khách hàng.

Gợi ý sản phẩm có thể được thực hiện trên nhiều kênh khác nhau, bao gồm trang web thương mại điện tử, ứng dụng di động và email marketing. Điều này giúp khách hàng tiếp cận thông tin về các sản phẩm một cách thuận tiện và nhanh chóng.

Việc sử dụng mô hình gợi ý sản phẩm cũng giúp cải thiện trải nghiệm mua sắm của khách hàng. Khách hàng có thể dễ dàng tìm thấy những sản phẩm mà họ quan tâm và đang tìm kiếm, từ đó tiết kiệm thời gian và nâng cao mức độ hài lòng của họ.

Trong bối cảnh công nghệ và dữ liệu lớn ngày càng phát triển, việc gợi ý sản phẩm trở nên ngày càng quan trọng. Mô hình gợi ý sản phẩm đóng vai trò then chốt trong đồ án này, nhằm tối ưu hóa quá trình mua sắm và nâng cao hiệu quả kinh doanh của các doanh nghiệp thương mại điện tử.

5.2.2 Tổng quát về TF-IDF

Có nhiều phương pháp có thể được áp dụng trong bài toán trên, như: Gợi ý dựa trên lọc cộng tác, Gợi ý dựa trên nội dung, Gợi ý dựa trên phiên,... Trong đồ án này, phương pháp gợi ý dựa trên nội dung được lựa chọn, cụ thể sử dụng kỹ thuật TF-IDF cho bài toán gợi ý. Lý do cho việc lựa chọn này là do hệ thống chưa có dữ liệu về lịch sử tương tác của người dùng và sản phẩm. Ngoài ra, TF-IDF cũng được lựa chọn vì tính đơn giản và hiệu quả mà nó mang lại trong việc xử lý dữ liệu lớn và đưa ra các gợi ý sản phẩm phù hợp cho người dùng.

Về TF-IDF (Term Frequency-Inverse Document Frequency), đó là một phương pháp được sử dụng trong lĩnh vực xử lý ngôn ngữ tự nhiên và khai phá dữ liệu văn

bản. Nó là một giải pháp hiệu quả để đo độ quan trọng của một từ (term) trong một tập hợp các văn bản.

Ý tưởng cơ bản của TF-IDF là tính toán giá trị của mỗi từ trong một văn bản, đánh dấu sự đặc trưng và quan trọng của từ đó trong văn bản so với các văn bản khác. Cụ thể, đó là sự kết hợp của 2 thành phần:

- TF- Term Frequency: Đo độ phổ biến của một từ trong một văn bản cụ thể. Nó tính toán tần số xuất hiện của từ đó trong văn bản. Cách tính:

$$tf(t, d) = \frac{f(t, d)}{\max\{f(w, d) : w \in d\}}$$

Trong đó:

- $tf(t, d)$: Tần suất xuất hiện của từ t trong văn bản d
- $f(t, d)$: Số lần xuất hiện của từ t trong văn bản d
- $\max(\{f(w, d) : w \in d\})$: Số lần xuất hiện của từ có số lần xuất hiện nhiều nhất trong văn bản

Giá trị $tf(t, d)$ càng cao trong một văn bản, tức là từ đó xuất hiện nhiều lần trong văn bản đó.

- IDF – Inverse Document Frequency: Đo độ quan trọng của một từ trong toàn bộ tập hợp các văn bản. Nó giảm giá trị của từ phổ biến xuất hiện trong nhiều văn bản và tăng giá trị của từ hiếm xuất hiện trong ít văn bản. Cách tính:

$$idf(t, D) = \log \frac{|D|}{|d \in D : t \in d|}$$

Trong đó:

- $idf(t, D)$: Giá trị idf của từ t trong tập văn bản D
- $|D|$: Tổng số văn bản trong tập D
- $|\{d \in D : t \in d\}|$: Số văn bản trong tập D có chứa từ t . Nếu từ đó không xuất hiện ở bất cứ 1 văn bản nào trong tập thì mẫu số sẽ bằng 0 => phép chia cho không không hợp lệ, vì thế người ta thường thay bằng mẫu thức $1 + |\{d \in D : t \in d\}|$.

Giá trị $idf(t, D)$ càng cao, tức là từ đó xuất hiện ít trong toàn bộ tập hợp các văn bản, làm tăng độ quan trọng của từ đó trong mỗi văn bản cụ thể.

Kết hợp hai thành phần TF và IDF, ta có công thức TF-IDF để đánh giá tầm

quan trọng của từ trong một văn bản và tập hợp các văn bản:

$$tfidf(t, d, D) = tf(t, d) \times idf(t, D)$$

Trong đó: $tfidf(t, d, D)$ là giá trị $tfidf$ của từ t trong văn bản d và tập văn bản D .

Những từ có giá trị $tfidf(t, d, D)$ cao là những từ xuất hiện nhiều trong văn bản này, và xuất hiện ít trong các văn bản khác. Việc này giúp lọc ra những từ phổ biến và giữ lại những từ có giá trị cao (từ khoá của văn bản đó).

5.2.3 Sử dụng TF-IDF cho bài toán gợi ý

Chúng ta xây dựng một hệ thống đề xuất sản phẩm (recommendation system) dựa trên việc sử dụng cosine similarity giữa các sản phẩm. Hệ thống này sẽ giúp đưa ra các sản phẩm tương tự khi người dùng cung cấp tên của một sản phẩm. Bên dưới là các bước thực hiện của hệ thống đề xuất sản phẩm:

i. Tiền xử lý dữ liệu:

- Thay thế khoảng trắng trong tên người bán (seller) bằng dấu gạch dưới để tránh xử lý sai trong việc tạo siêu dữ liệu (Metadata).
- Tạo siêu dữ liệu bằng cách ghép các thông tin quan trọng của sản phẩm như tên (name), danh mục (category) và người bán (seller).
- Tiền xử lý dữ liệu như trong bài toán phân loại đánh giá để tách từ và trích xuất các từ từ siêu dữ liệu. Kết quả của bước này là danh sách các từ.

ii. Xây dựng TF-IDF:

- Sử dụng HashingTF để chuyển đổi từ vựng thành vector tần suất xuất hiện.
- Sử dụng IDF kết hợp các vector tần suất để tính TF-IDF.

iii. Xây dựng ma trận độ tương đồng Cosine:

- Sử dụng VectorAssembler để kết hợp các vector đặc trưng (features) thành một vector duy nhất, sau đó biến đổi thành RDD để tính toán độ tương đồng Cosine giữa các sản phẩm..
- Tiến hành tính toán độ tương đồng Cosine giữa các sản phẩm.
- Sử dụng độ tương đồng Cosine (Cosine Similarity) để tìm các sản phẩm tương tự nhất dựa trên tên của sản phẩm được cung cấp.

iv. Gợi ý sản phẩm:

- Nhân đầu vào là tên sản phẩm cùng số lượng sản phẩm tương tự muốn đề

xuất.

- Sau đó, tiến hành lấy ID của sản phẩm dựa trên tên sản phẩm và tìm các sản phẩm có độ tương đồng Cosine cao nhất với sản phẩm đã chọn và loại bỏ sản phẩm đã chọn khỏi kết quả.
- Cuối cùng, trả về tên của các sản phẩm tương tự được đề xuất.

Các bước trên đã tạo ra một hệ thống đề xuất sản phẩm đơn giản dựa trên thông tin tên, danh mục và người bán của sản phẩm. Để cải thiện hiệu suất của hệ thống, có thể áp dụng thêm các kỹ thuật và phương pháp phức tạp hơn như gợi ý dựa trên lọc cộng tác, gợi ý dựa trên phiên, gợi ý dựa trên tri thức,...

Trên đây là một số bài toán điển hình mà đồ án đã triển khai. Ngoài ra, còn rất nhiều bài toán khác như phân loại danh mục sản phẩm, dự đoán giá sản phẩm, phân tích đánh giá khách hàng,... mà chúng ta có thể thực hiện để phân tích dữ liệu. Các bài toán này sẽ giúp chúng ta hiểu rõ hơn về dữ liệu sản phẩm và bình luận, từ đó đưa ra các quyết định kinh doanh thông minh và tối ưu hóa trải nghiệm mua sắm của người dùng.

CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG

6.1 Môi trường thực nghiệm

Sau khi hoàn thành thiết kế hệ thống và quyết định được các công nghệ sẽ sử dụng, chúng ta tiến vào giai đoạn triển khai và đánh giá hệ thống. Phần thực nghiệm đã được thực hiện trên một máy tính cá nhân có các thông số như sau:

Thông số	Giá trị
Hệ điều hành	Ubuntu 22.04.2 LTS
Kiến trúc	x86_64
RAM	12GB
Vi xử lý	Intel(R) Core(TM) i5-5200U CPU @ 2.20GHz
Số nhân xử lý	4 Cores

Bảng 6.1: Thông tin môi trường thực nghiệm

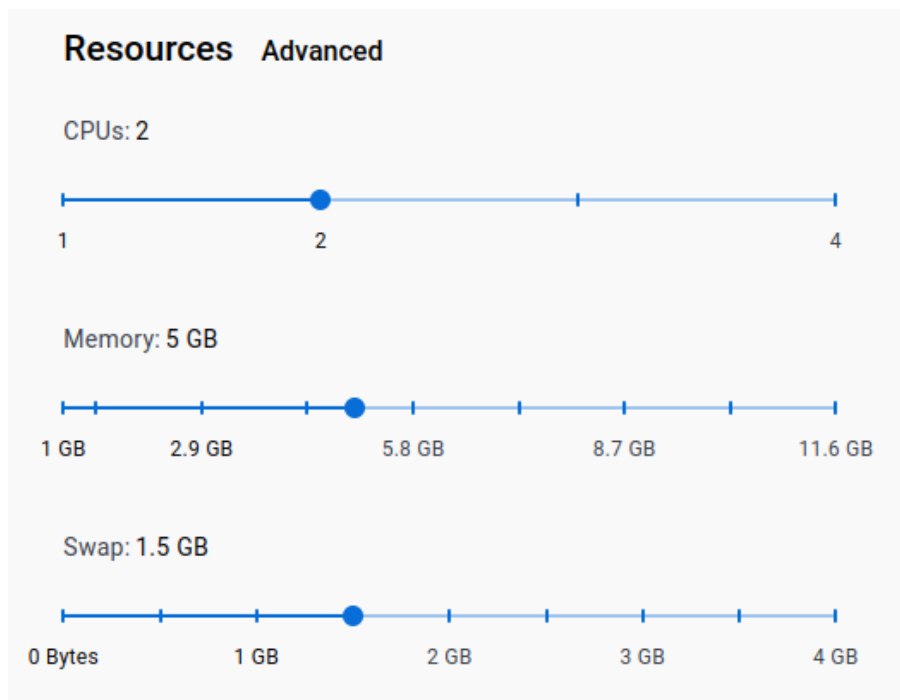
Các công nghệ được triển khai bằng một phương pháp phổ biến là sử dụng Docker¹, một nền tảng ảo cho phép chúng ta tạo môi trường ảo và đóng gói ứng dụng cùng với các thành phần cần thiết, đảm bảo sự linh hoạt và dễ dàng quản lý. Sau đó, những yêu cầu, nhiệm vụ của các công nghệ sẽ được xử lý bằng ngôn ngữ lập trình Python².

Với Docker, đó là một nền tảng ảo hóa cấp phép mở giúp đóng gói ứng dụng và các phụ thuộc của chúng thành các "containers". Các container này độc lập và có thể chạy trên bất kỳ môi trường nào hỗ trợ Docker, bao gồm cả máy tính cá nhân và các môi trường sản xuất. Sử dụng Docker giúp cho việc triển khai và quản lý hệ thống trở nên dễ dàng, linh hoạt và đáng tin cậy.

Để triển khai hệ thống, chúng ta sẽ tạo các Docker container cho từng công nghệ đã chọn, bao gồm cả ứng dụng và các phụ thuộc cần thiết. Việc đóng gói và triển khai các công nghệ trong các container Docker giúp đảm bảo rằng chúng hoạt động một cách đồng nhất trên mọi môi trường. Phiên bản Docker được sử dụng là version 23.0.1. Sau đây là tài nguyên được cung cấp cho Docker để triển khai hệ thống:

¹[12] I. Docker, Docker overview. [Online]. Available: <https://docs.docker.com/get-started/>

²<https://docs.python.org/3.10/>



Hình 6.1: Tài nguyên Docker

6.2 Triển khai hệ thống

Hệ thống được triển khai trên môi trường thực nghiệm bằng cách sử dụng Docker Compose và bao gồm các dịch vụ sau đây:

1. Zookeeper-1, Zookeeper-2, Zookeeper-3:

- Image: `confluentinc/cp-zookeeper`
- Phiên bản: 7.4.0
- Mô tả: Các container zookeeper đại diện cho các nút ZooKeeper trong hệ thống.
- Cấu hình:
 - **ZOOKEEPER_SERVER_ID**: ID của ba nút tương ứng là **1, 2, 3**.
 - **ZOOKEEPER_CLIENT_PORT**: Port để kết nối với ba nút đều là **2181**.
 - **ZOOKEEPER_TICK_TIME**: Thời gian chờ giữa các tick của các nút là **2000** mili giây.
 - **ZOOKEEPER_QUORUM_LISTEN_ON_ALL_IPS**: Được đặt là **true**, cho phép lắng nghe trên tất cả địa chỉ IP.
 - **ZOOKEEPER_SERVERS**: Cấu hình danh sách các nút ZooKeeper trong quorum, với giá trị là **zookeeper-1:2888:3888;zookeeper-2:2888:3888;zookeeper-3:2888:3888**.

- Cổng: Chuyển tiếp cổng **2181** của các container sang cổng **12181**, **22181**, **32181** ở máy cục .

2. Kafka-1, Kafka-2, Kafka-3:

- Image: `confluentinc/cp-kafka`
- Phiên bản: 7.4.0
- Mô tả: Các container Kafka đại diện cho các môi trường broker trong hệ thống.
- Cấu hình:
 - **KAFKA_BROKER_ID**: ID của Broker Kafka.
 - **KAFKA_ZOOKEEPER_CONNECT**: Địa chỉ kết nối tới ZooKeeper.
 - **KAFKA_ADVERTISED_LISTENERS**: Cấu hình giao thức lắng nghe của Kafka.
 - **KAFKA_LISTENER_SECURITY_PROTOCOL_MAP**: Bảo mật giao thức lắng nghe của Kafka.
 - **KAFKA_INTER_BROKER_LISTENER_NAME**: Tên giao thức trung gian giữa các Broker.
 - **KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR**: Số lượng bản sao của Topic Offsets.
 - **KAFKA_TRANSACTION_STATE_LOG_MIN_ISR**: Số lượng bản sao (Replica) cần phải xác nhận để hoàn tất giao dịch.
 - **KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR**: Số lượng bản sao của nhật ký trạng thái giao dịch.
- Cổng: Chuyển tiếp cổng **19092**, **29092**, **39092** của Container sang các cổng **19092**, **29092**, **39092** ở máy cục bộ.

3. Elasticsearch:

- Image: `docker.elastic.co/elasticsearch/elasticsearch`
- Phiên bản: 7.17.10
- Mô tả: Container Elasticsearch đại diện cho dịch vụ cơ sở dữ liệu Elasticsearch.
- Cấu hình:

- **xpack.security.enabled**: Tắt tính năng bảo mật X-Pack.
- **discovery.type**: Cấu hình chế độ single-node.
- **ES_JAVA_OPTS**: Cấu hình JVM cho Elasticsearch.
- Cổng: Chuyển tiếp cổng **9200** của container sang cổng **9201** ở máy cục bộ.

4. Kibana:

- Image: `docker.elastic.co/kibana/kibana`
- Phiên bản: 7.17.10
- Mô tả: Container Kibana đại diện cho dịch vụ giao diện người dùng cho Elasticsearch.
- Cấu hình:
 - **ELASTICSEARCH_HOSTS**: Địa chỉ kết nối tới Elasticsearch.
- Cổng: Chuyển tiếp cổng 5601 của container sang cổng **5602** ở máy cục bộ.

5. Grafana:

- Image: `grafana/grafana`
- Phiên bản:
- Mô tả: Container Grafana đại diện cho dịch vụ biểu đồ và giám sát. Cấu hình:
 - **GF_SECURITY_ADMIN_USER**: Người dùng quản trị Grafana.
 - **GF_SECURITY_ADMIN_PASSWORD**: Mật khẩu người dùng quản trị Grafana.
- Cổng: Chuyển tiếp cổng **3000** của container sang cổng **3000** ở máy cục bộ.

6. Prometheus:

- Image: `prom/prometheus`
- Phiên bản: (2.45.0 kiểm tra lại)
- Mô tả: Container Prometheus đại diện cho dịch vụ thu thập và giám sát các metrics.
- Cấu hình: Tạo các job thu thập metrics từ Node-exporter, Kafka-exporter, Elasticsearch-exporter.

- Cổng: Chuyển tiếp cổng **9090**, **9093** của container sang cổng **9089**, **9093** ở máy cục bộ.

7. Node-exporter³:

- Image: `prom/node-exporter`
- Phiên bản: 1.1.2
- Mô tả: Container Node-exporter đại diện cho dịch vụ thu thập metrics của máy chủ.
- Cổng: Chuyển tiếp cổng **9100** của container sang cổng **9100** ở máy cục bộ.

8. Kafka-exporter⁴:

- Image: `redpandadata/kminion`
- Phiên bản: 2.2.5
- Mô tả: Container Kafka-exporter đại diện cho dịch vụ thu thập metrics của Kafka.
- Cấu hình: Kết nối tới các broker Kafka.
- Cổng: Chuyển tiếp cổng **8080** của container sang cổng **9304** ở máy cục bộ.

9. Elasticsearch-exporter⁵:

- Image: `prometheuscommunity/elasticsearch-exporter`
- Phiên bản: 1.6.0
- Mô tả: Container Elasticsearch-exporter đại diện cho dịch vụ thu thập metrics của Elasticsearch.
- Cấu hình: Kết nối tới Elasticsearch.
- Cổng: Chuyển tiếp cổng **9114** của container sang cổng **9114** ở máy cục bộ.

Mạng **tiki_net** là một mạng cục bộ (bridge network) được tạo ra trong Docker Compose để cung cấp một môi trường mạng cô lập cho các container trong hệ thống. Dưới đây là thông tin chi tiết về mạng **tiki_net**:

- Tên mạng: **tiki_net**

³Monitoring Linux Host metris with the Node Exporter

⁴Prometheus Exporter for Apache Kafka - KMinion

⁵Elasticsearch stats exporter for Prometheus

- Driver mạng: bridge (mạng cầu)
- IPAM (IP Address Management):
 - Driver: default
 - Cấu hình:
 - * Subnet: **172.20.0.0/27**
(địa chỉ IP mạng con từ **172.20.0.1** đến **172.20.0.30**)
 - * Gateway: **172.20.0.1** (địa chỉ IP của gateway trong mạng con)

Các container trong tệp Docker Compose được gán địa chỉ IP tĩnh trong mạng **tiki_net** bằng cách sử dụng thuộc tính **ipv4_address** trong mỗi service. Điều này cho phép các container có thể liên lạc và giao tiếp với nhau thông qua địa chỉ IP trong mạng cục bộ.

Các dịch vụ trong tệp Docker Compose được cấu hình để lưu trữ dữ liệu trong các thư mục riêng, được gắn kết vào các thư mục tương ứng trên máy chủ Docker. Điều này giúp bảo vệ và duy trì dữ liệu ngay cả khi các container bị xóa hoặc khởi động lại.

Ngoài các dịch vụ được triển khai trong môi trường Docker, chúng ta cũng sử dụng Apache HBase là một cơ sở dữ liệu trong hệ thống. Tuy nhiên, Apache HBase không được triển khai bằng Docker mà được triển khai trực tiếp trên máy cục bộ (on-premises) với cấu hình như sau.

- **hbase.rootdir**: Xác định đường dẫn HBase được lưu trữ trên HDFS, tại địa chỉ `hdfs://192.168.100.4:9000/hbase`.
- **hbase.zookeeper.property.clientPort**: cổng mà HBase sử dụng để kết nối với ZooKeeper là **10231**.
- **zookeeper.session.timeout**: Thời gian chờ timeout của session với ZooKeeper là **120000** mili giây.
- **hbase.zookeeper.property.tickTime**: Thời gian chờ giữa các tick của ZooKeeper là **6000** mili giây.
- **hbase.zookeeper.quorum**: Danh sách các máy chủ ZooKeeper trong cụm gồm `localhost:12181, localhost:22181, localhost:32181`.
- **hbase.thrift.server.socket.read.timeout**: Thời gian chờ (timeout) khi đọc từ socket của HBase Thrift Server là **1800000** mili giây.

Việc triển khai HBase ở máy cục bộ có những lợi ích và lý do tương tự như việc

triển khai một hệ thống cơ sở dữ liệu khác ở một nơi riêng biệt, giúp cho dữ liệu được hoàn toàn kiểm soát cũng như cho phép tối ưu hiệu suất và độ trễ. Dữ liệu được lưu trữ và xử lý trên các máy chủ trong mạng nội bộ, giúp giảm thiểu độ trễ mạng và tăng tốc độ truy cập dữ liệu.

6.3 Kết quả thực nghiệm

Sau khi triển khai các dịch vụ, thành phần cần thiết bằng Docker và cài đặt Apache HBase ở máy cục bộ, chúng ta sẽ thu được một hệ thống như hình ảnh được mô tả sau đây.

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS	NAMES	
b01ef0fd7fcd	docker.elastic.co/kibana/kibana:7.17.10	"/bin/tini -- /usr/l..."	2 weeks ago
Up 30 minutes	0.0.0.0:5602->5601/tcp	kibana	
3206a0ccacd0	confluentinc/cp-kafka:latest	"/etc/confluent/dock..."	2 weeks ago
Up 9 minutes	9092/tcp, 0.0.0.0:19092->19092/tcp	kafka-1	
b8edea7d7a99	confluentinc/cp-kafka:latest	"/etc/confluent/dock..."	2 weeks ago
Up 9 minutes	9092/tcp, 0.0.0.0:29092->29092/tcp	kafka-2	
ccbef9eb1063	confluentinc/cp-kafka:latest	"/etc/confluent/dock..."	2 weeks ago
Up 10 minutes	9092/tcp, 0.0.0.0:39092->39092/tcp	kafka-3	
66126053a4ef	confluentinc/cp-zookeeper:latest	"/etc/confluent/dock..."	2 weeks ago
Up 10 minutes	2888/tcp, 3888/tcp, 0.0.0.0:22181->22181/tcp	zookeeper-2	
079022b42358	confluentinc/cp-zookeeper:latest	"/etc/confluent/dock..."	2 weeks ago
Up 10 minutes	2888/tcp, 3888/tcp, 0.0.0.0:32181->32181/tcp	zookeeper-3	
4e4120304b28	confluentinc/cp-zookeeper:latest	"/etc/confluent/dock..."	2 weeks ago
Up 10 minutes	2888/tcp, 3888/tcp, 0.0.0.0:12181->12181/tcp	zookeeper-1	
f1302eed07c0	docker.elastic.co/elasticsearch/elasticsearch:7.17.10	"/bin/tini -- /usr/l..."	2 weeks ago
Up 34 minutes	9300/tcp, 0.0.0.0:9201->9200/tcp	elasticsearch	
c1b4175c9d67	prom/prometheus	"/bin/prometheus --c..."	2 weeks ago
Up 35 minutes	0.0.0.0:9093->9093/tcp, 0.0.0.0:9089->9090/tcp	prometheus	
2f41a629ffc4	redpandadata/kminion:v2.2.5	"/app/kminion"	2 weeks ago
Up 9 minutes	0.0.0.0:9304->8080/tcp	kafka-exporter	
845cfc4d49c6	prometheuscommunity/elasticsearch-exporter:latest	"/bin/elasticsearch_..."	2 weeks ago
Up 35 minutes	7979/tcp, 0.0.0.0:9114->9114/tcp	elasticsearch_exporter	
ccd992da3c33	prom/node-exporter:v1.1.2	"/bin/node_exporter ..."	2 weeks ago
Up 35 minutes	0.0.0.0:9100->9100/tcp	node-exporter	
404203245c9a	grafana/grafana	"/run.sh"	2 weeks ago
Up 35 minutes	0.0.0.0:3000->3000/tcp	grafana	

Hình 6.2: Triển khai các Container

Sau khi triển khai các container chứa các dịch vụ cần thiết, chúng ta đã xây dựng thành công hệ thống như sau:

6.3.1 Lớp thu thập dữ liệu

Như đã giới thiệu ở mục 4.3.2 - Quy trình xử lý và lưu trữ giữ liệu, chúng ta sẽ thực hiện việc gửi yêu cầu 60 giây 1 lần đến API của Tiki.vn để lấy thông tin danh mục, nếu danh mục đó chưa được gửi đi trước đó thì Kafka Producer sẽ gửi dữ liệu vào Kafka Topic. Sử dụng asyncio và aiohttp giúp xử lý yêu cầu không đồng bộ và tối ưu hóa việc lắng nghe sự kiện và gửi dữ liệu. Nếu đã sang ngày mới, danh sách loại hàng đã gửi được gửi vào Topic. Sau khi tiến trình truy vấn API trích xuất được mã danh mục, và tên sản phẩm, kết quả sẽ được Kafka Producer gửi vào Topic category của Kafka. Dữ liệu được gửi vào Kafka như sau:

```
(843, 'Tiểu sử - Hồi ký')
(3320, 'Sách Làm Cha Mẹ')
(1583, 'Sửa rửa mặt')
(1856, 'Sách Học Tiếng Anh')
(1630, 'Bao cao su')
(49656, 'Vali nhựa')
(24062, 'Máy xay sinh tố gia đình')
(385, 'Sách kỹ năng làm việc')
(1984, 'Chảo các loại')
(49620, 'Giày thể thao nam cổ thấp')
(23622, 'Tủ')
(5995, 'Áo mưa, ô dù và phụ kiện đi mưa')
(1521, 'Truyện ngôn tình')
(8400, 'Tai Nghe True Wireless')
(53190, 'Ổ cắm, Ổn áp, biến áp')
(8239, 'Nước giặt')
(879, 'Sách Khoa Học - Kỹ Thuật')
```

Hình 6.3: Dữ liệu danh mục trong category_topic

Sau khi dữ liệu được gửi vào, các Kafka Consumer chỉ cần lắng nghe Topic `category_topic` là sẽ có dữ liệu để xử lý các bước tiếp theo.

6.3.2 Lớp xử lý và lưu trữ dữ liệu

Khác với các công nghệ có trong đề tài, Apache HBase được triển khai dưới máy cục bộ. Dưới đây là hình ảnh kết quả triển khai HBase:

Regions in Transition

1 region(s) in transition. 0 region(s) in transition for more than 60000 milliseconds.

Region	State	RIT time (ms)	Retries
e21ac09069af0380666f3087764a1a4c	871,,1684860311652.e21ac09069af0380666f3087764a1a4c. state=OPENING, ts=Tue Jul 11 22:49:51 ICT 2023 (2s ago), server=null	2919	6

prev 1 next

Region Servers

Base Stats	Memory	Requests	Storefiles	Compactions	Replications
ServerName	Start time	Last contact	Version	Requests Per Second	Num. Regions
ubuntu22_16020,1689090541023	Tue Jul 11 22:49:01 ICT 2023	2 s	2.5.4	3	6
Total:1				3	6

Hình 6.4: HBase Region Server

Hệ thống cơ sở dữ liệu bao gồm bốn bảng: `category`, `overview_rating`, `product`, `rating`. Trong đó, `category` và `overview_rating` được khởi tạo như sau:

4 table(s) in set.

Table	Description
category	'category', {TABLE_ATTRIBUTES => {METADATA => {hbase.store.file-tracker.impl => 'DEFAULT'}}}, {NAME => 'data', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)}}
overview_rating	'overview_rating', {TABLE_ATTRIBUTES => {METADATA => {hbase.store.file-tracker.impl => 'DEFAULT'}}}, {NAME => 'rating', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'review_photo', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'reviews_count', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'stars', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)}}

Hình 6.5: Bảng category và overview_rating

Và hai bảng còn lại, product cùng rating được khởi tạo như sau:

product	'product', {TABLE_ATTRIBUTES => {METADATA => {hbase.store.file-tracker.impl => 'DEFAULT'}}}, {NAME => 'basic_info', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'brand', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'categories', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'current_seller', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'reference', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'specifications', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)}}
rating	'rating', {TABLE_ATTRIBUTES => {METADATA => {hbase.store.file-tracker.impl => 'DEFAULT'}}}, {NAME => 'created_by', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'images', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'review_info', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)'}}, {NAME => 'timeline', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)}}

Hình 6.6: Bảng product và rating

Sau khi đã triển khai và thiết kế các bảng trên HBase, hệ thống cơ sở dữ liệu đã được xây dựng và chuẩn bị sẵn sàng để thu thập, lưu trữ và cung cấp dữ liệu cho các ứng dụng và dịch vụ khác trong hệ thống.

Bằng cách lắng nghe các message trong Topic `category_topic`, chúng ta đã lấy được các tham số, thực hiện truy vấn API theo từng loại, khi đó sẽ thu được dữ liệu và lưu vào HBase. Sau đây là hình ảnh bảng sản phẩm sau khi có sản phẩm lưu vào:

6.3.3 Lớp phục vụ

Sau khi dữ liệu được lưu vào HBase, dữ liệu đó sẽ được xử lý phù hợp, sau đó mã hóa thành chuỗi bytes sử dụng bảng mã UTF-8 được gửi vào các Topic tương ứng với từng loại dữ liệu. Dữ liệu về sản phẩm (bảng product) sẽ được gửi vào Topic `product_topic`, dữ liệu về tổng quan đánh giá sản phẩm (bảng overview_rating) được gửi vào Topic `overview_topic`, còn dữ liệu về

đánh giá (bảng rating) được gửi vào bảng `rating_topic`.

Các dữ liệu được gửi vào Kafka dưới dạng như sau:

```
b'{"key": "2023-07-23_116410119", "id": 116410119, "name": "T\\u00fai N\\u01b0\\u1edbc gi\\u1eb7t Omo N\\u01b0\\u1edbc gi\\u1eb7t c\\u1eeda tr\\u01b0\\u1edbc h\\u01b0\\u01a1ng Lavender Kh\\u1eed m\\u00f9i th\\u01b0 th\\u00e1i 3.6KG", "price": 175000, "time": "2023-07-23", "original_price": 227000, "all_time_quantity_sold": 13328, "rating_average": 4.8, "review_count": 1692, "favourite_count": 0, "day_ago_created": 719, "description": "OMO MATIC - L\\u1ef0A CH\\u1eccN TH\\u00d4NG MINH \\u0110\\u1ec2 CH\\u0102M S\\u00d3C \\u00c1O QU\\u1ea6N GIA \\u0110\\u00ccNH M \\u1ed8T C\\u00c1CH TO\\u00c0N DI\\u1ec6N NH\\u1ea4T\\nN\\u01b0\\u1edbc Gi\\u1eb7t Omo Matic Comfort m\\u1edbi \\u0111\\u01b0\\u1ec1nh, thu\\u1ebf nh\\u1eadp kh\\u1ea9u (\\u0111\\u1ed1i v\\u1edbi \\u0111\\u01a1n h\\u00e0ng giao t\\u1eeb n\\u01b0\\u1edbc ngo\\u00e0i c\\u00f3 gi\\u00e1 tr\\u1ecb tr\\u00ean 1 tri\\u1ec7u \\u0111\\u1ed3ng)....", "store_id": 40395, "is_best_store": false, "seller_name": "Tiki Trading", "seller_price": 175000, "brand_id": 33323, "brand_name": "Omo", "category_id": 2549, "category_name": "\\u0110\\u1ed3 Ch\\u01a1i - M\\u1eb9 & B\\u00e9", "category_parent_id": 8239, "category_parent_name": "N\\u01b0\\u1edbc gi\\u1eb7t"}'
```

Hình 6.7: Dữ liệu sản phẩm trong `product_topic`

```
b'{"product_id": 116410119, "name_product": "T\\u00fai N\\u01b0\\u1edbc gi\\u1eb7t Omo N\\u01b0\\u1edbc gi\\u1eb7t c\\u1eeda tr\\u01b0\\u1edbc h\\u01b0\\u01a1ng Lavender Kh\\u1eed m\\u00f9i th\\u01b0 th\\u00e1i 3.6KG", "stars_1_count": 23, "stars_1_percent": 1, "stars_2_count": 10, "stars_2_percent": 1, "stars_3_count": 15, "stars_3_percent": 1, "stars_4_count": 140, "stars_4_percent": 9, "stars_5_count": 1517, "stars_5_percent": 88, "rating_average": 4.8, "reviews_count": 1705, "review_photo": {"total": 171, "total_photo": 240}}'
```

Hình 6.8: Dữ liệu tổng quan đánh giá sản phẩm trong `overview_topic`

```
b'{"id": 16834499, "title": "C\\u1ef1c k\\u00ec h\\u00e0i l\\u00f2ng", "content": "Ch\\u1ea5t l\\u01b0\\u1ee3ng t\\u1ed1t ! M\\u00e0 sao em mua kh\\u00f4ng c\\u00f3 c\\u00e1i b\\u00ecnh sho p \\u01a1i....", "customer_id": 1300639, "rating": 5}'
```

Hình 6.9: Dữ liệu đánh giá sản phẩm trong `rating_topic`

Sau khi các Consumer lắng nghe dữ liệu trong các Topic tương ứng, một tiến trình sẽ thực hiện công việc giải mã dữ liệu nhận được, biến đổi dữ liệu phù hợp để lưu vào Elasticsearch.

Các chỉ mục để lưu dữ liệu trong Elasticsearch được thiết kế đảm bảo khả năng trực quan hóa cũng như truy xuất vào các hệ thống khác, cụ thể như sau:

Với chỉ mục lưu trữ dữ liệu về sản phẩm:

```

1 GET /product_index/_mapping
2 {
3   "product_index" : {
4     "mappings" : {
5       "properties" : {
6         "all_time_quantity_sold" : {
7           "type" : "integer"
8         },
9         "brand_id" : {
10          "type" : "integer"
11        },
12        "brand_name" : {
13          "type" : "keyword"
14        },
15        "category_id" : {
16          "type" : "integer"
17        },
18        "category_name" : {
19          "type" : "keyword"
20        },
21        "category_parent_id" : {
22          "type" : "integer"
23        },
24        "category_parent_name" : {
25          "type" : "keyword"
26        },
27        "day_ago_created" : {
28          "type" : "integer"
29        },
30        "description" : {
31          "type" : "text"
32        },
33        "favourite_count" : {
34          "type" : "integer"
35        },
36        "id" : {
37          "type" : "integer"
38        },
39        "is_best_store" : {
40          "type" : "boolean"
41        },
42        "is_leaf" : {
43          "type" : "boolean"
44        },
45        "key" : {
46          "type" : "keyword"
47        },
48        "name" : {
49          "type" : "keyword"
50        },
51        "original_price" : {
52          "type" : "integer"
53        },
54        "price" : {
55          "type" : "integer"
56        },
57        "rating_average" : {
58          "type" : "float"
59        },
60        "review_count" : {
61          "type" : "integer"
62        },
63        "store_id" : {
64          "type" : "integer"
65        },
66        "store_name" : {
67          "type" : "keyword"
68        },
69        "store_price" : {
70          "type" : "integer"
71        },
72        "time" : {
73          "type" : "date"
74        }
75      }
76    }
77  }

```

Hình 6.10: Chỉ mục lưu trữ dữ liệu sản phẩm

Với chỉ mục lưu trữ dữ liệu tổng quan đánh giá của mỗi sản phẩm:

```
1 GET /overview_rating_index/_mapping
2 {
3   "overview_rating_index" : {
4     "mappings" : {
5       "properties" : {
6         "name_product" : {
7           "type" : "keyword"
8         },
9         "product_id" : {
10          "type" : "integer"
11        },
12        "rating_average" : {
13          "type" : "float"
14        },
15        "review_photo" : {
16          "type" : "nested",
17          "properties" : {
18            "total" : {
19              "type" : "integer"
20            },
21            "total_photo" : {
22              "type" : "integer"
23            }
24          }
25        },
26        "reviews_count" : {
27          "type" : "integer"
28        },
29        "stars_1_count" : {
30          "type" : "integer"
31        },
32        "stars_1_percent" : {
33          "type" : "float"
34        },
35        "stars_2_count" : {
36          "type" : "integer"
37        },
38        "stars_2_percent" : {
39          "type" : "float"
40        },
41        "stars_3_count" : {
42          "type" : "integer"
43        },
44        "stars_3_percent" : {
45          "type" : "float"
46        },
47        "stars_4_count" : {
48          "type" : "integer"
49        },
50        "stars_4_percent" : {
51          "type" : "float"
52        },
53        "stars_5_count" : {
54          "type" : "integer"
55        },
56        "stars_5_percent" : {
57          "type" : "float"
58        }
59      }
60    }
61  }
```

Hình 6.11: Chỉ mục lưu trữ dữ liệu tổng quan đánh giá mỗi sản phẩm

Với chỉ mục lưu trữ đánh giá của người dùng về sản phẩm:



Hình 6.12: Chỉ mục lưu trữ dữ liệu đánh giá

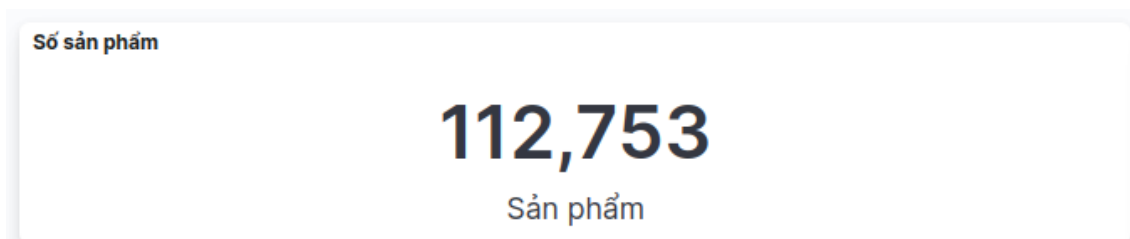
Sau khi đã triển khai và thiết kế các chỉ mục trong Elasticsearch, chúng ta đã hoàn tất việc xây dựng lớp phục vụ và chuẩn bị sẵn sàng để lưu trữ và cung cấp dữ liệu cho các ứng dụng và dịch vụ bên ngoài.

6.3.4 Lớp trực quan hóa dữ liệu

Sau khi dữ liệu từ HBase đã được đồng bộ vào Elasticsearch, chúng ta có thể sử dụng Kibana để tạo biểu đồ và phân tích dữ liệu. Để làm điều này, chúng ta cần ánh xạ các chỉ mục từ Elasticsearch sang Kibana. Dưới đây là danh sách các chỉ mục được tạo ra:

Hình ảnh các chỉ mục

Các chỉ mục đã được gán sẽ được liên kết để truy xuất dữ liệu từ Elasticsearch. Điều này cho phép chúng ta thực hiện công việc trực quan hóa dữ liệu, một số biểu đồ để phân tích và hiểu dữ liệu như sau: Về tổng số lượng sản phẩm:



Hình 6.13: Số lượng sản phẩm

Chúng ta cũng có thể thu được danh sách sản phẩm được bán nhiều nhất cùng số lượng đã bán.

Top 10 sản phẩm bán chạy nhất		
Mã sản phẩm ▾	Tên sản phẩm ▾	Số lượng đã bán ▾
632,680	Dầu Gội Chống Gàu Selsun (100ml)	151,593
26,114,399	Nóng Giận Là Bản Năng , Tĩnh Lặng Là Bản Lĩnh	69,792
750,578	Khi Hơi Thở Hóa Thình Không	60,434
10,005,245	Đàn Ông Sao Hỏa Đàn Bà Sao Kim	50,289
20,541,866	Dầu Gội Chống Gàu Selsun 50ml	41,165
41,319,582	Camera IP Wifi Trong Nhà EZVIZ C6N 1080p - Hàng...	40,950
1,600,005	Tai Nghe Nhét Tai Mi Basic Xiaomi HSEJ03JY - Hàn...	40,503
1,313,871	Sữa Chống Nắng Sunplay Cực Mạnh Sunplay Super ...	37,825
377,644	Chờ Đến Mẫu Giáo Thì Đã Muộn	37,333
73,124,602	Bình Giữ Nhiệt Bằng Thép Không Gỉ Lock&Lock Vacu...	36,581

Hình 6.14: Danh sách sản phẩm được bán nhiều nhất

Có thể thấy rằng các sản phẩm được bán nhiều nhất là các sản phẩm dân dụng, phổ biến trong đời sống hàng ngày. Chẳng hạn như các sản phẩm mỹ phẩm, chăm sóc cơ thể, sách hay bình giữ nhiệt,...

Tương tự, chúng ta cũng có danh sách về sản phẩm có giá bán cao nhất hiện tại cùng giá gốc của sản phẩm:

Top 10 sản phẩm có giá trị nhất				
Mã sản phẩm	Tên sản phẩm	Giá hiện bán	Giá gốc	
208,285,308	Xe Tay Ga Piaggio MP3 400 HPE Sport Mà...	330,000,000	330,000,000	
208,286,129	Xe Tay Ga Piaggio MP3 400 HPE Sport Mà...	330,000,000	330,000,000	
142,419,353	Honda Forza 300 Limited Editon	290,000,000	294,000,000	
54,014,791	Ghế massage Gintell DeSpace UFO-X	140,000,000	249,999,000	
200,653,060	Ghế Massage Toàn Thân Nhật Bản Synca K...	230,000,000	235,000,000	
122,240,374	Ghế Massage thương gia cao cấp Buheung ...	190,000,000	199,000,000	
249,502,008	[Lắp đặt tại nhà] Ghế Massage Toàn Thân ...	190,000,000	196,000,000	
210,426,536	Ghế Massage Toàn Thân S9 Superchair C...	170,000,000	179,999,999	
197,638,372	Ghế massage toàn thân cao cấp KLC K797...	170,000,000	179,000,000	
207,256,565	Ghế massage trị liệu toàn thân FUJICARE J...	160,000,000	168,800,000	

Hình 6.15: Danh sách sản phẩm giá hiện bán cao nhất

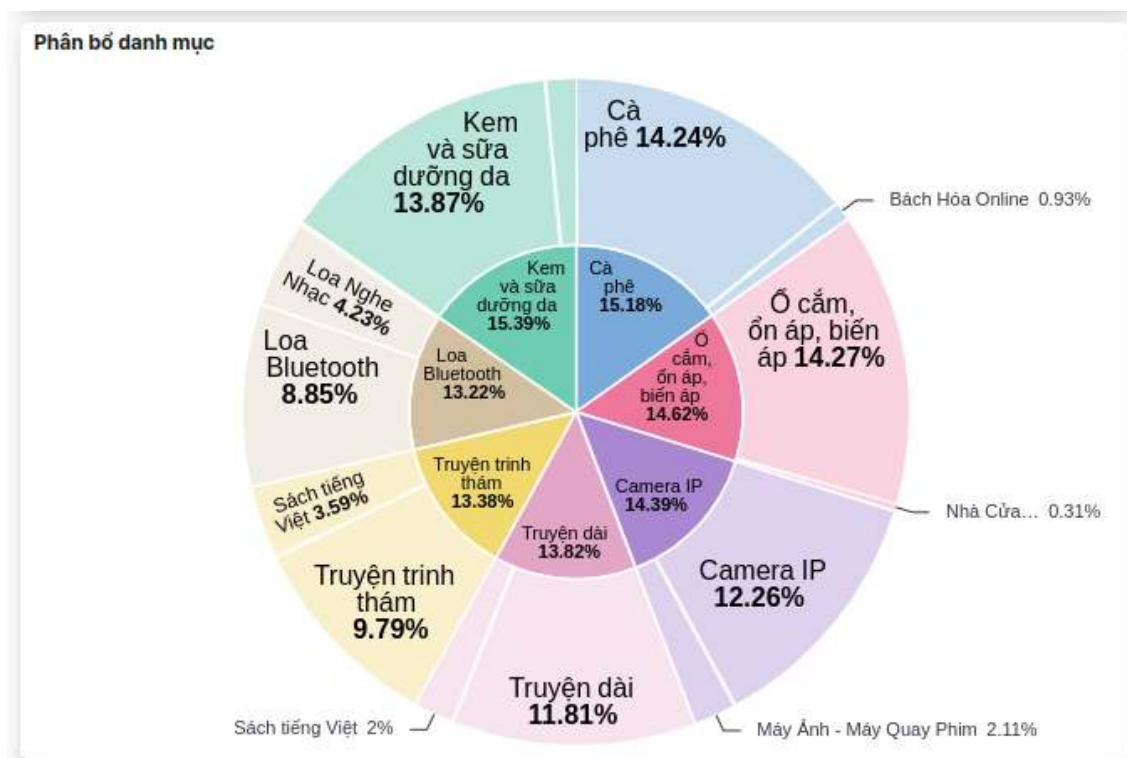
Có thể thấy, các sản phẩm có giá trị, được bán với giá cao đều là những mặt hàng về xe tay ga, ghế Massage, các mặt hàng khác đều xếp sau 2 loại sản phẩm này.

Số lượng các danh mục tổng đã thu thập được

Số danh mục
74
Danh mục

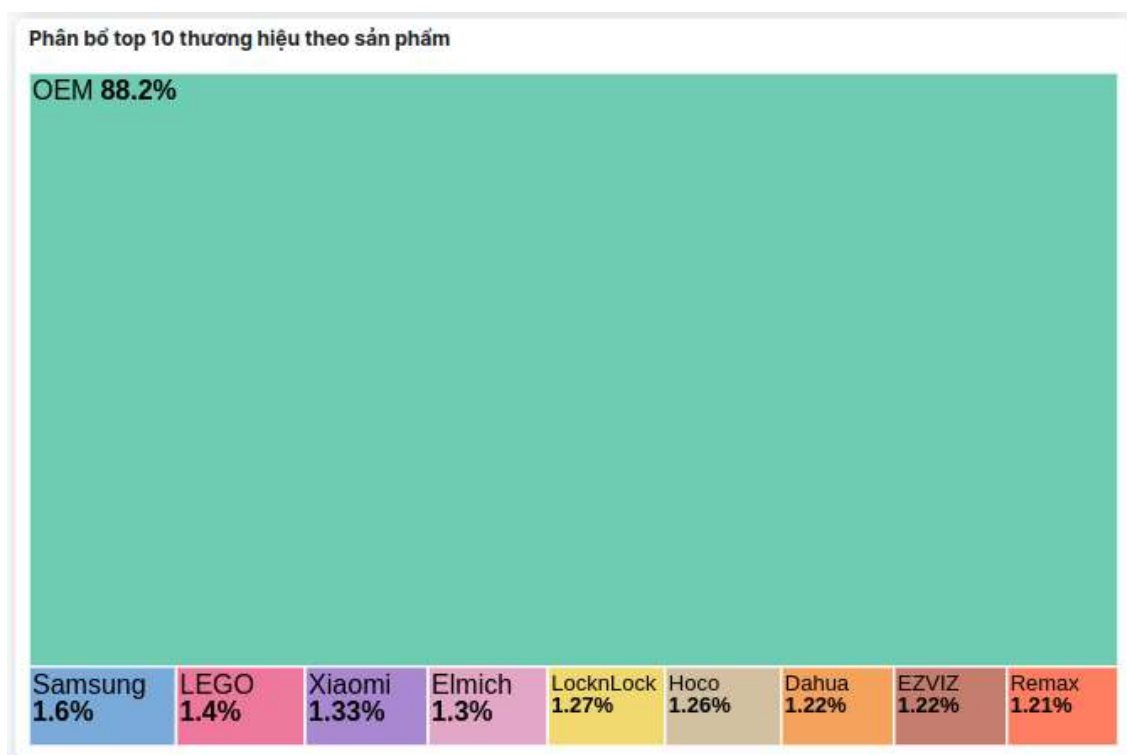
Hình 6.16: Số lượng danh mục tổng

Phân bổ các danh mục theo số lượng sản phẩm, trong đó mỗi danh mục tổng sẽ có rất nhiều danh mục con, các danh mục con sẽ chiếm một tỉ lệ nhất định trong danh mục tổng.



Hình 6.17: Phân bố danh mục

Biểu đồ sau đây thể hiện phân bố số lượng sản phẩm theo từng thương hiệu:



Hình 6.18: Phân bố sản phẩm trong các thương hiệu

Danh sách thương hiệu có giá trị nhất.

Top 10 thương hiệu có giá trị nhất				
Mã thương hiệu	Tên thương hiệu	Số sản phẩm	Tổng giá trị	
152,855	Honda	275	18,002,062,681	
111,461	OEM	24,236	14,878,410,481	
18,802	Samsung	434	4,220,832,150	
245,331	Vespa	41	3,929,800,000	
47,687	DAIKIN	179	3,561,346,069	
164,815	Piaggio	36	3,546,800,000	
153,444	Yamaha	89	2,761,477,500	
18,804	Panasonic	174	2,090,784,905	
114,527	Garmin	108	1,714,000,000	
18,851	LG	97	1,489,992,499	

Hình 6.19: Danh sách thương hiệu có giá trị nhất

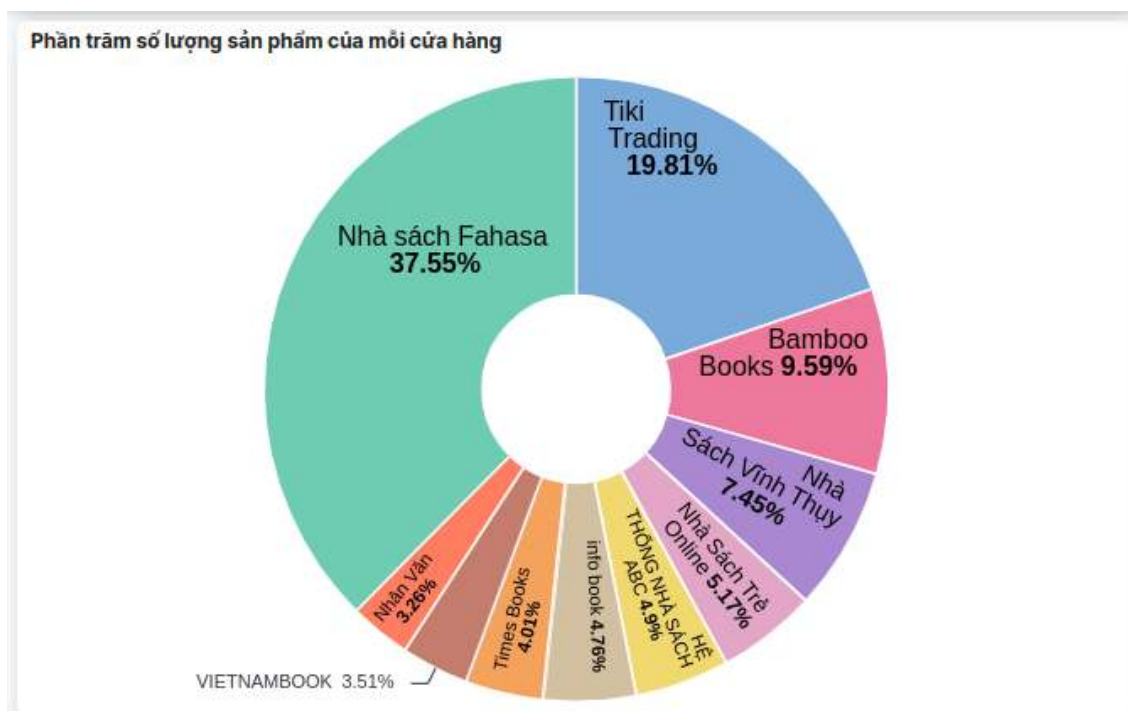
Có thể dễ thấy, với các sản phẩm được bán giá cao, đương nhiên Honda sẽ có giá trị lớn nhất. Với OEM, đó là những sản phẩm được bán trên Tiki mà không cần cung cấp bất kỳ giấy tờ nào cho Tiki, người bán có thể cạnh tranh với nhau mà không vắn phải đăng ký sở hữu trí tuệ cho 1 sản phẩm nhất định. Do đó, sản phẩm này phổ biến nhất trên Tiki. Tuy nhiên, do thương hiệu này đều là những sản phẩm có giá trị không phải quá cao, do đó, giá trị tổng thể chỉ xếp thứ 2. Ngoài ra, những thương hiệu khác đề là những thương hiệu nổi tiếng trên thế giới.

Tổng số lượng cửa hàng đã thu thập được.

Số cửa hàng
6,889
Cửa hàng

Hình 6.20: Số lượng cửa hàng

Chúng ta sẽ vẽ được biểu đồ thể hiện phân bố số lượng sản phẩm trên môi cửa hàng.



Hình 6.21: Phân bố sản phẩm theo cửa hàng

Có thể thấy, Nhà sách Fahasa có rất nhiều sản phẩm, khi cửa hàng này bán sách là chủ yếu. Còn Tiki Trading là cửa hàng tập hợp sản phẩm chính hãng do Tiki trực tiếp chịu trách nhiệm tuyệt đối. Do đó, số lượng sản phẩm của 2 cửa hàng này chiếm hơn một nửa.

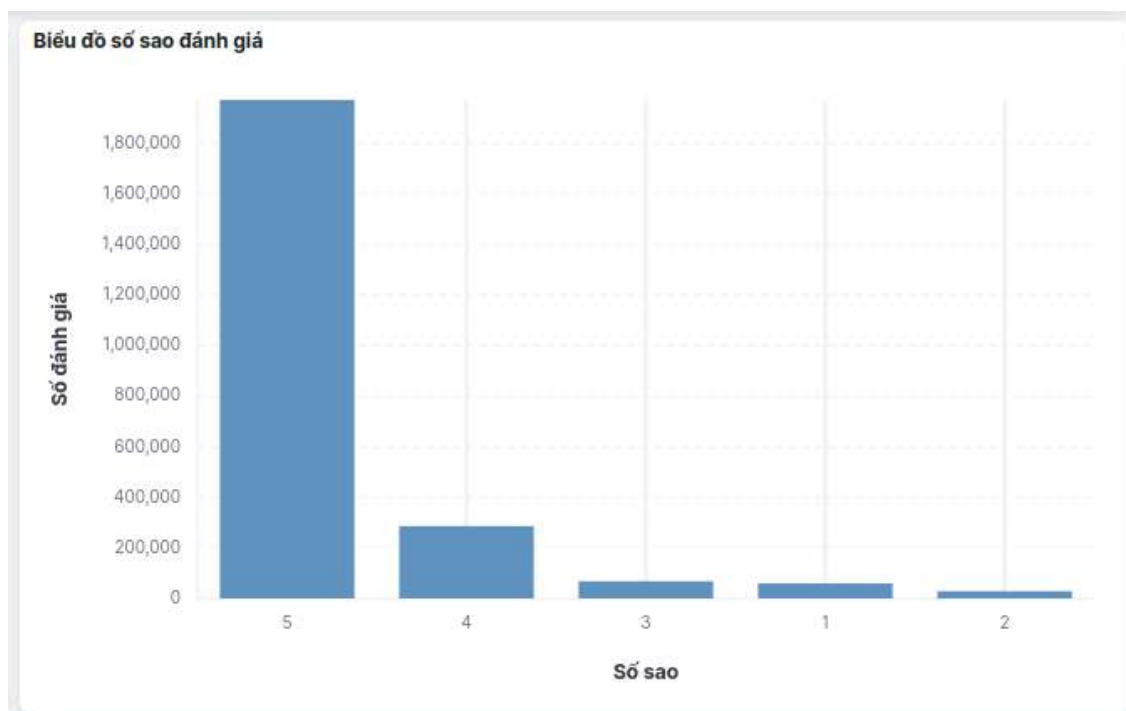
Về đánh giá, chúng ta sẽ có một số biểu đồ sau:

Về tổng số đánh giá đã thu thập được trong các sản phẩm.



Hình 6.22: Số đánh giá

Biểu đồ số sao đánh giá, bình luận của người dùng.



Hình 6.23: Biểu đồ số sao đánh giá

Có thể thấy, 5 sao chiếm đa số, điều đó cho thấy sản phẩm do Tiki mang lại đều có chất lượng tốt, làm hài lòng người dùng. Và dưới đây là danh sách các sản phẩm có nhiều đánh giá nhất, cũng như có nhiều đánh giá 5 sao nhất.

Danh sách sản phẩm có nhiều đánh giá 5 sao nhất

Mã sản phẩm	Tên sản phẩm	Phần trăm 5 sao	Số đánh giá
74,021,317	Cây Cam Ngọt Cửa Tôi	90	12,503
71,345,381	Sách Thay Đổi Cuộc Sống Với Nhân ...	88	7,500
299,461	Chuột Không Dây Logitech M331 Sil...	86	7,140
378,448	Nhà Giả Kim	82	6,231
26,114,399	Nóng Giận Là Bản Năng , Tĩnh Lặng ...	85	6,030
52,789,367	Nhà Giả Kim (Tái Bản 2020)	90	6,029
94,173,462	Muôn Kiếp Nhân Sinh 2	91	5,914
106,863,963	Tâm Lý Học - Phác Họa Chân Dung ...	87	5,904
54,748,899	Sách Muôn Kiếp Nhân Sinh (Bìa Mề...	88	5,863
36,626,940	Nồi Chiên Không Dầu Điện Tử Lock&...	81	5,377

Hình 6.24: Danh sách sản phẩm có nhiều đánh giá 5 sao nhất

Với mỗi đánh giá, bình luận của khách hàng, Tiki đều cho phép người dùng tạo những tiêu đề ngắn gọn để mô tả về đánh giá. Dưới đây là danh sách các tiêu đề phổ biến, với số sao tương ứng.

Tiêu đề đánh giá	Số sao	Số lượng
Cực kì hài lòng	5	1,893,229
Cực kì hài lòng	4	4,225
Cực kì hài lòng	3	770
Cực kì hài lòng	1	365
Cực kì hài lòng	2	222
Hài lòng	4	261,678
Hài lòng	5	10,733
Hài lòng	3	864
Hài lòng	2	142
Hài lòng	1	119
Bình thường	3	57 703

Hình 6.25: Danh sách tiêu đề đánh giá nhiều nhất

Có thể thấy, tiêu đề *Cực kì hài lòng* thể hiện một thái độ, cảm xúc, cũng như trải nghiệm tốt với sản phẩm, tuy nhiên vẫn có những đánh giá, bình luận 1, 2 sao tương ứng với tiêu đề này. Có thể do một số nhầm lẫn từ người dùng, do đó số sao trung bình sẽ ảnh hưởng rất nhiều bởi những nhầm lẫn kiểu này.

Trên đây là kết quả của việc triển khai lớp trực quan hóa, cùng với đó là một số biểu đồ có thể dùng cho công việc phân tích, đánh giá thị trường, cũng như hiểu bình luận.

6.3.5 Lớp giám sát hệ thống

Sau khi triển khai các thành phần cơ bản của hệ thống, chúng ta sẽ triển khai lớp giám sát hệ thống sử dụng Grafana và Prometheus để đảm bảo hoạt động ổn định và phát hiện kịp thời các lỗi sai.

Để trực quan hóa các thành phần trong hệ thống dưới dạng biểu đồ giám sát, chúng ta cần thu thập các chỉ số (metrics) cần thiết của mỗi thành phần bằng các công cụ trích xuất dữ liệu tương ứng. Đối với các chỉ số về tài nguyên hệ thống đang sử dụng, chúng ta sẽ thu thập bằng `prom/node-exporter` như đã triển khai trước đó. Các chỉ số của Kafka được thu thập bằng `redpandadata/kminion`,

và các chỉ số của Elasticsearch được thu thập bằng `prometheuscommunity/elasticsearch-exporter`. Các chỉ số của các thành phần này sẽ được thu thập bởi Prometheus.

Dưới đây là kết quả của quá trình triển khai lớp giám sát.

Đầu tiên, Prometheus sẽ được cấu hình các điểm cuối (endpoints) để chủ động kéo (pull) các chỉ số về từ các công cụ trích xuất dữ liệu:

Targets

All scrape pools
All Unhealthy Expand All
Q Filter by endpoint or labels

elasticsearch (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://172.20.0.23:9114/metrics	UP	instance="172.20.0.23:9114" job="elasticsearch"	923.000ms ago	118.612ms	

kafka (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://172.20.0.26:8080/metrics	UP	instance="172.20.0.26:8080" job="kafka"	3.673s ago	100.446ms	

node_exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://172.20.0.24:9100/metrics	UP	instance="172.20.0.24:9100" job="node_exporter"	3.781s ago	98.765ms	

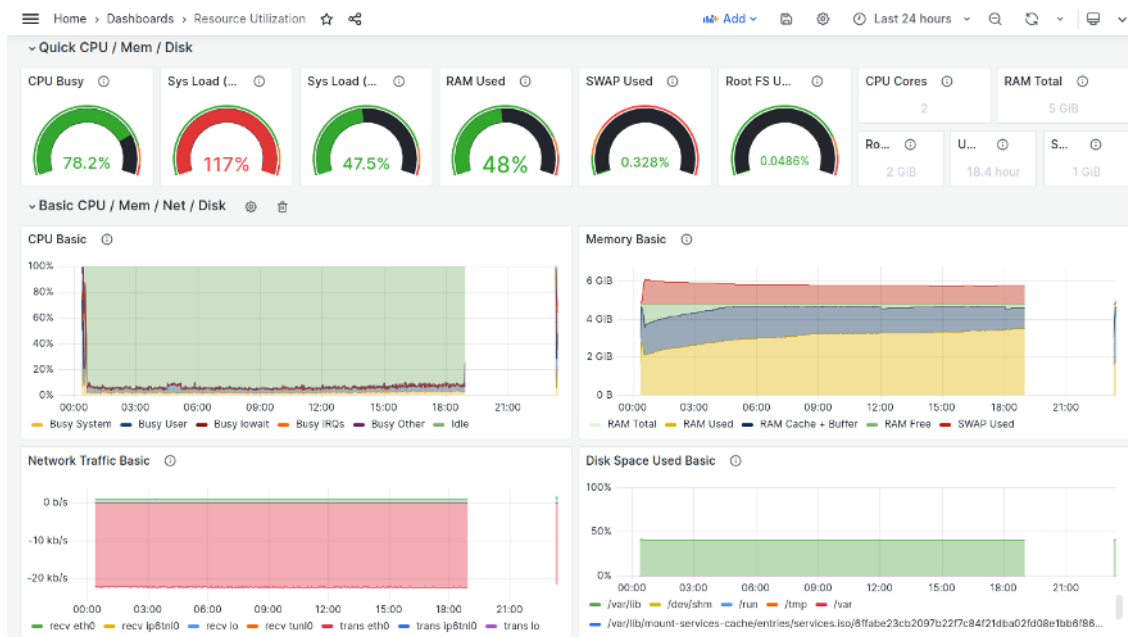
prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	15.133s ago	5.673ms	

Hình 6.26: Prometheus

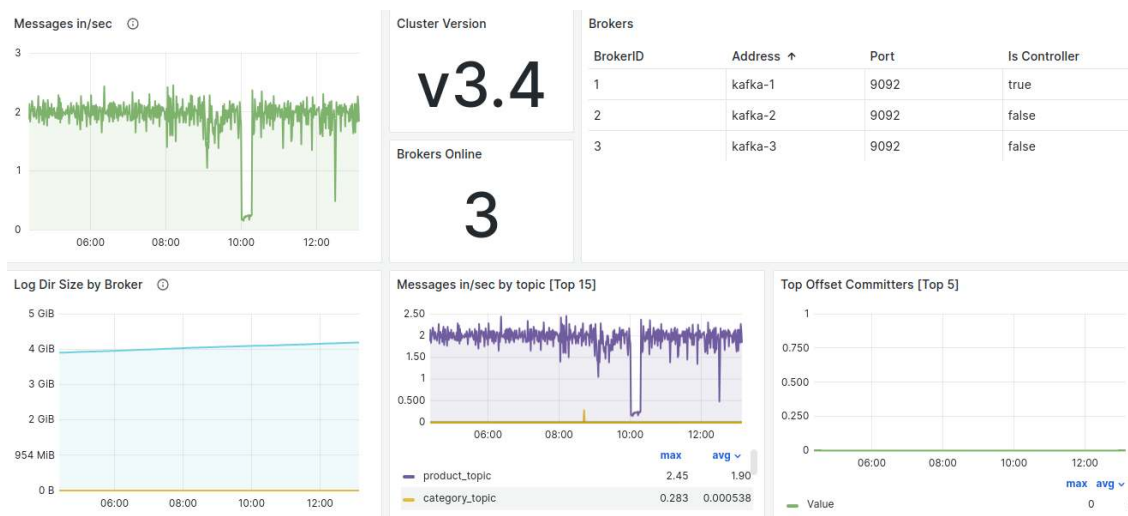
Khi đã thu thập được các chỉ số, chúng ta sẽ thực hiện trực quan hóa các chỉ số để theo dõi và giám sát hệ thống, phát hiện kịp thời các lỗi sai hoặc vấn đề, giúp chúng ta có cái nhìn tổng quan về hiệu suất và hoạt động của các thành phần trong hệ thống. Các biểu đồ được triển khai như sau:

Đầu tiên là hình vẽ biểu đồ trực quan hóa tài nguyên hệ thống, thể hiện các chỉ số về bộ nhớ, thời gian sử dụng,...



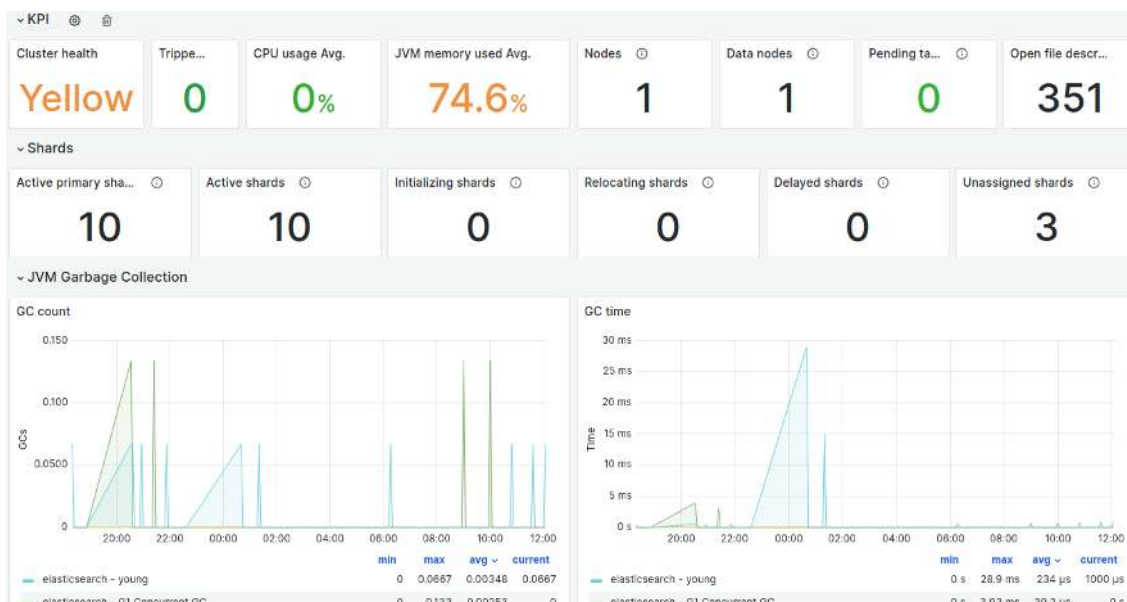
Hình 6.27: Biểu đồ giám sát tài nguyên hệ thống

Biểu đồ trực quan hóa các chỉ số của Kafka, cho thấy thời gian hoạt động của cụm, thông tin các nút có trong cụm,...



Hình 6.28: Biểu đồ giám sát Kafka

Cuối cùng là biểu đồ trực quan hóa chỉ số của Elasticsearch, cho thấy trạng thái của cụm, số lượng các Shards,...



Hình 6.29: Biểu đồ giám sát Elasticsearch

Sau quá trình triển khai lớp giám sát, chúng ta đã thành công trong việc thu thập và trực quan hóa các chỉ số của các thành phần trong hệ thống. Điều này giúp chúng ta có cái nhìn tổng quan về hiệu suất và hoạt động của hệ thống, từ đó có thể đánh giá và giám sát một cách hiệu quả. Qua đó, chúng ta có thể phát hiện sớm các vấn đề tiềm ẩn và thực hiện các biện pháp cải thiện để đảm bảo sự ổn định và hiệu quả của hệ thống.

6.4 Các bài toán phân tích dữ liệu

Sau khi triển khai phân tích dữ liệu, đây là hình ảnh kết quả của 2 bài toán đã nêu ở Chương 5 - Một số bài toán phân tích dữ liệu:

6.4.1 Bài toán phân loại bình luận sản phẩm

Đầu tiên, sau khi trải qua các bước tiền xử lý đã trình bày ở mục 5.1.3, chúng ta sẽ sử dụng dữ liệu có dạng như sau:

```
+-----+-----+
|          content|   label|rating|
+-----+-----+
|quá hay mình là n...|positive|    5|
|cực kì hài lòng t...|positive|    5|
|quyên này là tạp ...|positive|    4|
|hài lòng nội dung...|positive|    4|
|cực kì hài lòng n...|positive|    5|
|không rất không h...|negative|    1|
|hài lòng mình đã ...|positive|    4|
|cực kì hài lòng s...|positive|    5|
|không không hài l...|negative|    2|
|cực kì hài lòng s...|positive|    5|
+-----+-----+
only showing top 10 rows
```

Hình 6.30: Dữ liệu dùng cho bài toán phân loại

Khi đó, phân bố nhãn của tất cả bình luận như sau:

```
+-----+-----+
|   label| count|
+-----+-----+
|positive|563955|
|neutral | 52515|
|negative| 83534|
+-----+-----+
```

Hình 6.31: Phân bố dữ liệu bài toán phân loại

Chúng ta có thể thấy tổng số đánh giá, bình luận thu được sau bước tiền xử lý đã bị giảm đi rất nhiều so với toàn bộ đánh giá, bình luận thu thập được từ Tiki, 700004 so với 2413270. Lý do bởi bước tiền xử lý đã tiến hành loại bỏ các bình luận có tiêu đề, nội dung, cùng số sao giống nhau và chỉ giữ lại duy nhất một bình luận. Việc giảm mạnh số lượng cho thấy rất nhiều bình luận giống nhau, làm giảm lượng dữ liệu có thể sử dụng cho bài toán.

Sau khi tiến hành tiền xử lý dữ liệu, chúng ta thực hiện chạy mô hình phân loại, và đây là kết quả sau quá trình phân loại:

```
Accuracy: 0.8225850948457923
Test Error = 0.177415
RandomForestClassificationModel: uid=RandomForestClassifier_ea90644a250c, numTrees=20, numClasses=3, numFeatures=200
```

Hình 6.32: Kết quả dự đoán

6.4.2 Bài toán gợi ý sản phẩm

6.5 Kiểm thử

Sau khi triển khai hệ thống thành công, chúng ta sẽ tiến hành kiểm tra chất lượng và khả năng xử lý lỗi của hệ thống thông qua các quy trình kiểm thử sau:

STT	Mục đích	Quy trình	Kết quả		Kết luận
			Mong muốn	Thực tế	
1	Lưu dữ liệu khi chưa tạo Kafka Topic	Không tạo Kafka Topic tương ứng	Tự động tạo mới Kafka Topic	Hệ thống tự tạo Kafka Topic tương ứng	Thành công
2	Lưu dữ liệu khi 1 Kafka server bất kỳ bị tắt	Tắt bất kỳ 1-2/3 Kafka server	Tiếp tục quá trình thu thập dữ liệu	Hệ thống tiếp tục thu thập dữ liệu dùng Kafka còn lại	Thành công
3	Thêm danh mục mới khi lắng nghe API thu thập dữ liệu	Tạo mới một danh mục, sau đó gửi vào Kafka Topic tương ứng	Kafka Consumer lắng nghe được danh mục đã được thêm, tiến hành thu thập dữ liệu	Hệ thống lắng nghe được danh mục mới được thêm vào và tiến hành thu thập dữ liệu	Thành công
4	Tiếp tục lưu dữ liệu khi bật lại HMaster đã bị tắt	Tắt đột xuất HMaster khi đang thu thập dữ liệu	Tự động thu thập lại khi bật HMaster	Hệ thống tiếp tục thu thập dữ liệu nhưng bỏ qua danh mục trước đó đang thu thập	Thành công
5	Tiếp tục lưu dữ liệu khi bật lại Elasticsearch đã bị tắt	Tắt đột xuất Elasticsearch khi đang lưu dữ liệu	Tiếp tục lưu lại từ vị trí cũ khi bật Elasticsearch	Hệ thống tiếp tục lưu dữ liệu vào Elasticsearch?	Thành công
6	Theo dõi được các thông số của các công nghệ sử dụng	Tắt đột xuất Kafka hoặc Elasticsearch hoặc cả 2	Lớp giám sát hỗ trợ trực quan hóa việc ngừng hoạt động của các công nghệ khi chúng bị tắt	Biểu đồ trực quan hiển thị thời điểm chính xác khi các công nghệ bị ngừng hoạt động	Thành công

Bảng 6.2: Kiểm thử tính sẵn sàng

Các trường hợp trên đại diện cho việc kiểm thử tính sẵn sàng của hệ thống. Dưới đây là thời gian kiểm thử tính nhanh chóng của hệ thống khi lưu trữ và truy xuất dữ liệu. Lưu ý rằng thời gian lưu trữ được tính từ thời điểm lắng nghe các thông tin danh mục trong Topic của Kafka, sau đó tiến hành xử lý và lưu trữ các dữ liệu tương ứng, biến T ở trong bảng chỉ thời gian thực hiện quy trình (đơn vị là giây).

STT	Quy trình	Thời gian
1	Lưu trữ dữ liệu danh mục vào HBase	$0.0027 \leq T \leq 0.0319$
2	Lưu trữ dữ liệu sản phẩm vào HBase và gửi vào Kafka	$0.2689 \leq T \leq 1.2219$
3	Lưu trữ dữ liệu tổng quan đánh giá vào HBase và gửi vào Kafka	$0.4925 \leq T \leq 2.7539$
4	Lưu trữ dữ liệu đánh giá vào HBase và gửi vào Kafka	$0.4631 \leq T \leq 1.4498$
5	Lắng nghe dữ liệu sản phẩm từ Kafka Topic và lưu vào Elasticsearch	$0.0105 \leq T \leq 0.2147$
6	Lắng nghe dữ liệu tổng quan đánh giá từ Kafka Topic và lưu vào Elasticsearch	$0.0162 \leq T \leq 0.0891$
7	Lắng nghe dữ liệu đánh giá từ Kafka Topic và lưu vào Elasticsearch	$0.0099 \leq T \leq 0.0531$
8	Truy vấn dữ liệu sản phẩm trong HBase	$0.0085 \leq T \leq 0.2167$
9	Truy vấn dữ liệu tổng quan đánh giá trong HBase	$0.0079 \leq T \leq 0.0294$
10	Truy vấn dữ liệu đánh giá trong HBase	$0.0083 \leq T \leq 0.1750$
11	Truy vấn dữ liệu danh mục trong HBase	$0.0042 \leq T \leq 0.0159$

Bảng 6.3: Kiểm thử hiệu năng

6.6 Đánh giá

Hệ thống thu thập, lưu trữ và xử lý dữ liệu của Tiki đã được triển khai thành công với sử dụng Docker và các công nghệ như Kafka, Elasticsearch, và HBase. Qua quá trình phân loại bình luận, hệ thống đã đạt được những kết quả đáng chú ý.

Hệ thống thu thập dữ liệu đã sử dụng Apache Kafka, một nền tảng hàng đầu trong việc xử lý dữ liệu dòng, hệ thống có khả năng xử lý hàng triệu thông điệp mỗi giây một cách đáng tin cậy. Sau khi dữ liệu được thu thập, nó được lưu trữ và quản lý bởi HBase và Elasticsearch. HBase cho phép lưu trữ và xử lý lượng dữ liệu lớn từ Tiki, đồng thời cung cấp tính năng đáng tin cậy và khả năng mở rộng. Elasticsearch cung cấp khả năng tìm kiếm dữ liệu mạnh mẽ và tìm kiếm theo văn bản đầy đủ, đồng thời hỗ trợ các tính năng phân tích và truy vấn phức tạp. Sau khi dữ liệu được lưu trữ và quản lý, nó được hiển thị một cách trực quan thông qua Kibana, giúp chúng ta phân tích và đánh giá dữ liệu theo các yêu cầu cụ thể của người dùng, từ đó đáp ứng các yêu cầu về phân tích thị trường, theo dõi hiệu suất và đưa ra quyết định kinh doanh.

Hệ thống cũng được trang bị các công cụ giám sát như Prometheus và Grafana để theo dõi và giám sát hiệu suất hoạt động của các thành phần. Prometheus thu thập và lưu trữ các thông số và sự kiện về hệ thống, trong khi Grafana cung cấp

giao diện trực quan cho việc hiển thị biểu đồ, bảng điều khiển và báo cáo giám sát. Điều này giúp chúng ta theo dõi hiệu suất và tình trạng của hệ thống, phát hiện và giải quyết các vấn đề một cách nhanh chóng và hiệu quả.

Một số bài toán phân tích dữ liệu điển hình cũng đã được triển khai. Sử dụng thuật toán Random Forest để phân loại bình luận với độ chính xác khá cao. Tiền xử lý dữ liệu và loại bỏ stopwords đã cải thiện hiệu suất phân loại. Gợi ý sản phẩm có thể sử dụng TF-IDF để đánh giá tầm quan trọng của từ trong bình luận.

Ngoài ra, kết quả của quá trình kiểm thử cũng rất tốt, việc lưu trữ và đồng bộ hóa dữ liệu diễn ra một cách rất nhanh chóng, cho thấy rằng hệ thống hoạt động rất hiệu quả và đáng tin cậy. Thời gian xử lý và truy vấn dữ liệu đáp ứng đủ yêu cầu của một hệ thống thời gian thực.

Tổng quan lại, hệ thống thu thập, lưu trữ và xử lý dữ liệu sàn thương mại điện tử Tiki.vn đã đạt được những mục tiêu quan trọng trong việc thu thập dữ liệu, lưu trữ dữ liệu hiệu quả và trực quan hóa dữ liệu để hỗ trợ quyết định kinh doanh. Đồng thời, hệ thống cũng được giám sát để đảm bảo tính tin cậy và tránh lỗi. Bằng việc sử dụng các công nghệ như Kafka, HBase, Elasticsearch, Kibana, Grafana và nhiều công nghệ khác, hệ thống đã được xây dựng với sự mạnh mẽ và linh hoạt, đáp ứng các yêu cầu chính về hiệu suất, tính mở rộng và sẵn sàng, đồng thời cung cấp khả năng xử lý và truy xuất dữ liệu nhanh chóng và chính xác, giúp chúng ta có một hệ thống đáng tin cậy để nghiên cứu thị trường, nâng cao khả năng cạnh tranh và cung cấp giá trị tốt nhất cho người dùng.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết luận

Hệ thống của Tiki đã xây dựng và triển khai thành công, mang lại nhiều ưu điểm đáng kể. Đầu tiên, hệ thống đã giải quyết được vấn đề quan trọng trong thu thập và lưu trữ dữ liệu hiệu quả. Công nghệ như Apache Kafka, HBase và Elasticsearch đã đảm bảo tính tin cậy, mở rộng và hiệu suất trong xử lý dữ liệu lớn.

Một ưu điểm nổi bật của hệ thống là khả năng trực quan hóa dữ liệu thông qua Kibana và Grafana. Kibana cho phép hiển thị và phân tích dữ liệu qua biểu đồ, đồ thị và bảng điều khiển tương tác, giúp đưa ra quyết định kinh doanh thông minh. Grafana tập trung vào việc vẽ biểu đồ giám sát và theo dõi các chỉ số quan trọng của hệ thống, giúp người dùng theo dõi hiệu suất và giải quyết vấn đề nhanh chóng.

Bên cạnh đó, hệ thống hiện tại còn một số nhược điểm cần được cải thiện. Đầu tiên, trong phiên bản hiện tại, HBase, Elasticsearch và Kibana đang chạy trên một single node, điều này giới hạn khả năng mở rộng và xử lý dữ liệu lớn hơn. Hiện tại, chúng ta không có khả năng thu thập các metrics quan trọng từ HBase để đánh giá và theo dõi hiệu suất của nó. Ngoài ra, Grafana cũng chưa được cấu hình để hỗ trợ cài đặt cảnh báo.

Tuy nhiên, với sự mạnh mẽ và linh hoạt của các công nghệ được sử dụng, hệ thống đã đạt được nhiều thành công và mang lại giá trị lớn cho Tiki. Sự phát triển và tối ưu hóa trong tương lai sẽ nâng cao khả năng mở rộng, tăng cường giám sát và quản lý hiệu suất, từ đó tạo nên một hệ thống mạnh mẽ và đáng tin cậy để nghiên cứu thị trường và đáp ứng nhu cầu của người dùng.

Trong quá trình thực hiện đồ án, bản thân tôi đã thu thập được nhiều kiến thức, từ tìm hiểu về thị trường thương mại điện tử, đến xây dựng luồng cho hệ thống từ thu thập, lưu trữ, xử lý và trực quan hóa dữ liệu, cũng như đóng gói các thành phần bằng Docker, mang lại lợi ích về linh hoạt, tiết kiệm tài nguyên và đơn giản hóa triển khai ứng dụng. Tôi tin, đây là bài học và kinh nghiệm quý giá để giúp tôi phát triển chuyên môn trong tương lai.

7.2 Hướng phát triển

Với những ưu điểm và nhược điểm như đã trình bày. Chắc chắn, hệ thống được xây dựng trong đồ án vẫn cần phải tiếp tục hoàn thiện hơn để đáp ứng các yêu cầu ngày càng tăng về quy mô, hiệu suất và quản lý dữ liệu. Một trong những hướng phát triển quan trọng là mở rộng các thành phần chính như HBase, Elasticsearch lên nhiều node, từ đó tăng khả năng mở rộng và xử lý dữ liệu lớn hơn. Mặt khác,

cần thu thập các chỉ số của HBase để cung cấp thông tin quan trọng về trạng thái và hiệu suất của hệ thống, từ đó giúp xác định và giải quyết các vấn đề sớm hơn. Đồng thời, đặt các cảnh báo trong Grafana để theo dõi các chỉ số quan trọng và phát hiện sự cố ngay khi chúng vượt quá ngưỡng được đặt trước. Hệ thống trực quan hóa có thể xây dựng giao diện webapp để phân tích từng yêu cầu cụ thể hơn, phân tích được nhiều thông tin hơn như biến động giá của một sản phẩm bất kỳ,...

Ngoài ra, trong tương lai, có thể xem xét mở rộng tích hợp với Spark Streaming hay với các công nghệ khác như Apache Flink¹ hoặc Apache Druid² để xử lý dữ liệu phức tạp và phân tích thời gian thực. Việc kết hợp các công nghệ này giúp tăng cường khả năng xử lý và phân tích dữ liệu, từ đó mang lại thông tin cần thiết cho việc ra quyết định kinh doanh và định hướng chiến lược.

Tổng quát lại, hướng phát triển cho hệ thống thu thập, lưu trữ và xử lý dữ liệu của Tiki tập trung vào việc mở rộng, tối ưu hóa và tích hợp các công nghệ để đạt được khả năng mở rộng, hiệu suất và quản lý cao hơn. Những phát triển này sẽ đóng vai trò quan trọng trong việc nghiên cứu thị trường, cải thiện khả năng cạnh tranh và mang lại giá trị tối đa cho người dùng.

¹Apache Flink® — Stateful Computations over Data Streams

²Apache Druid: a high performance real-time analytics database.

TÀI LIỆU THAM KHẢO

- [1] N. Marz and J. Warren, *Big Data: Principles and best practices of scalable realtime data systems*. MANNING, 2015.
- [2] T. S. F. Apache, *Kafka 3.5 documentation*. [Online]. Available: <https://kafka.apache.org/documentation/index.html>.
- [3] T. S. F. Apache, *Apache hbase™ reference guide*. [Online]. Available: <https://hbase.apache.org/book.html>.
- [4] N. Elastic, *Introducing elasticsearch service*. [Online]. Available: <https://www.elastic.co/guide/en/cloud/current/ec-getting-started.html>.
- [5] N. Elastic, *Kibana—your window into elastic*. [Online]. Available: <https://www.elastic.co/guide/en/kibana/current/introduction.html>.
- [6] C. N. C. Foundation, *What is prometheus?* [Online]. Available: https://prometheus.io/docs/tutorials/getting_started/.
- [7] I. Raintank, *Get started with grafana and prometheus*. [Online]. Available: <https://grafana.com/docs/grafana/latest/getting-started/get-started-grafana-prometheus/>.
- [8] T. K. Ho, “Random decision forests,” *ATT Bell Laboratories*, 1995.
- [9] B. Das and S. Chakraborty, “An improved text sentiment classification model using tf-idf and next word negation,” *IEEE*, 2018.
- [10] S. Lohr, “The origins of ‘big data’: An etymological detective story,” *New York Times*, 2013.
- [11] D. Borthakur, “The hadoop distributed file system: Architecture and design,” *The Software Founction Apache*, 2006.
- [12] I. Docker, *Docker overview*. [Online]. Available: <https://docs.docker.com/get-started/>.

PHỤ LỤC

A. CÔNG NGHỆ SỬ DỤNG

Quá trình đọc, ghi dữ liệu của HBase

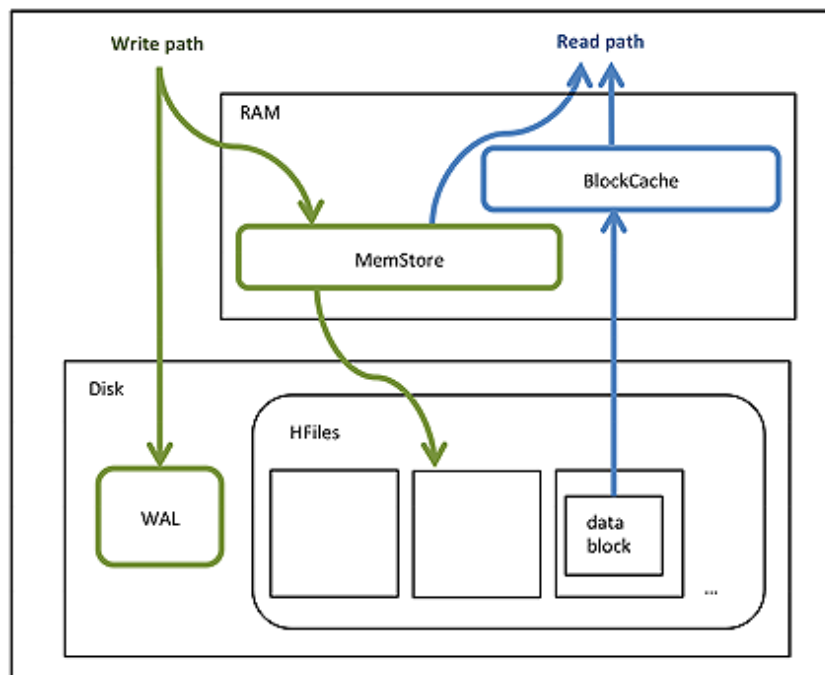
Sau khi đã trình bày kiến trúc của HBase như đã trình bày mục ??, chúng ta sẽ tiếp tục đi vào cách HBase thực hiện việc đọc và ghi dữ liệu. Hoạt động đọc/ghi dữ liệu liên quan đến các hoạt động tập trong HBase và bộ nhớ cache (RAM), bao gồm HFile, Block, BlockCache, MemStore và Write Ahead Log (WAL).

- HFile: chứa dữ liệu bảng, các chỉ mục liên quan tới dữ liệu đó, và siêu dữ liệu (metadata) về dữ liệu.
- Block: Là đơn vị dữ liệu nhỏ nhất có thể đọc từ một HFile, thường là một số byte nhất định. Mỗi HFile bao gồm một chuỗi các block.
- BlockCache: Là cơ chế chính của HBase để hỗ trợ các hoạt động đọc ngẫu nhiên với độ trễ thấp. BlockCache là một trong hai cơ chế bộ nhớ cache được duy trì bởi HBase. Khi một block được đọc từ HDFS, nó sẽ được lưu trữ trong BlockCache. Bằng cách lưu trữ các block trong bộ nhớ cache, HBase có thể truy xuất dữ liệu nhanh chóng mà không cần phải đọc từ đĩa, làm giảm thời gian truy xuất dữ liệu và cải thiện hiệu suất.
- MemStore (Memory Store): Là bộ nhớ trong (In-memory Storage) của một RegionServer. MemStore là một trong hai cấu trúc bộ nhớ đệm được duy trì bởi HBase. Khi dữ liệu mới được ghi, nó được đưa vào MemStore và sắp xếp theo Row Key. Nó tích lũy dữ liệu cho đến khi đầy, sau đó ghi (flushes) dữ liệu vào một tệp HFile mới trên đĩa. MemStore phục vụ hai mục đích: nó tăng tổng lượng dữ liệu được ghi vào đĩa trong một hoạt động đơn lẻ và duy trì dữ liệu vừa mới ghi vào trong bộ nhớ để cho việc đọc với độ trễ thấp sau này.
- Write Ahead Log (WAL): Là một tệp nhật ký (log file) ghi lại tất cả các thay đổi dữ liệu cho đến khi dữ liệu được ghi thành công vào đĩa (MemStore được đẩy xuống). WAL cho phép HBase khôi phục lại dữ liệu nếu có lỗi xảy ra trước khi dữ liệu ở MemStore được ghi xuống HFile.

BlockCache và MemStore nằm trong bộ nhớ truy cập ngẫu nhiên (RAM). HFiles và Write Ahead Log được lưu trữ trong HDFS. Các bước để ghi và đọc dữ liệu trong HBase được thực hiện như sau:

- Trong quá trình ghi dữ liệu, HBase ghi vào WAL (Write-Ahead Log) và MemStore (Memory Store). Dữ liệu được đẩy từ MemStore xuống đĩa theo giới hạn kích thước và khoảng thời gian định trước.

- Trong quá trình đọc dữ liệu, HBase sẽ đọc các khối dữ liệu từ BlockCache hoặc MemStore nếu chúng có sẵn trong các bộ nhớ đệm này. Nếu dữ liệu không có sẵn, HBase sẽ đọc từ đĩa và lưu một bản sao của dữ liệu đó vào BlockCache.



Hình A.1: Quá trình đọc, ghi dữ liệu của Apache HBase

Có thể nói, cơ chế ghi và đọc của HBase được tối ưu hóa để cung cấp hiệu suất cao và tính nhất quán cho việc lưu trữ và truy xuất dữ liệu phân tán trong môi trường phức tạp và đòi hỏi đáp ứng nhanh chóng.

B. HÌNH ẢNH HỆ THỐNG

Lớp trực quan hóa dữ liệu

Đây là các hình ảnh thêm về lớp trực quan hóa dữ liệu:

Top 10 sản phẩm có giá hiện bán cao nhất		
Mã sản phẩm ▾	Tên sản phẩm ▾	Giá hiện bán ▾
208,285,308	Xe Tay Ga Piaggio MP3 400 ...	330,000,000
208,286,129	Xe Tay Ga Piaggio MP3 400 ...	330,000,000
142,419,353	Honda Forza 300 Limited Ed...	294,000,000
200,653,060	Ghế Massage Toàn Thân Nh...	235,000,000
122,240,374	Ghế Massage thương gia ca...	199,000,000
249,502,008	[Lắp đặt tại nhà] Ghế Massa...	196,000,000
210,426,536	Ghế Massage Toàn Thân S9 ...	179,999,999
197,638,372	Ghế massage toàn thân cao ...	179,000,000
207,256,565	Ghế massage trị liệu toàn th...	168,800,000
206,548,589	Ghế massage toàn thân FUJI...	168,000,000

Hình B.1: Danh sách sản phẩm có giá trị nhất

Top 10 danh mục có giá trị nhất		
Mã danh mục	Tên danh mục	Tổng giá trị
11,878	Xe tay ga	28,577,658,181
3,865	Máy lạnh - Máy điều hòa	12,892,649,415
2,340	Máy massage toàn thân	9,128,218,381
2,324	Loa Bluetooth	5,944,053,972
1,795	Điện thoại Smartphone	5,139,265,995
4,381	Bàn ghế làm việc	4,101,330,504
8,123	Nồi chiên	3,417,358,488
5,121	Camera IP	2,969,192,651
2,657	Đồng Hồ Thông Minh	2,816,101,613
23,622	Tủ	2,653,526,739

Hình B.2: Danh sách danh mục có giá trị nhất

Từ biểu đồ trên, ta có thể nhìn thấy rằng danh mục Xe tay ga có giá trị nhất. Điều này dễ hiểu vì danh mục này chứa các sản phẩm được bán với giá trị rất cao. Ngoài ra, các danh mục khác cũng có giá trị lớn và đều liên quan đến các thiết bị điện tử, như Máy lạnh - Máy điều hòa, Máy Massage toàn thân, ...

Danh sách cửa hàng có giá trị nhất.

Top 10 cửa hàng có giá trị nhất				
Mã cửa hàng	Tên cửa hàng	Số sản phẩm	Tổng giá trị	
3,172	Vespa SAPA (HCM)	50	5,154,500,000	
10,842	GIA DỤNG VIỆT	470	2,972,288,000	
40,395	Tiki Trading	6,424	2,516,334,196	
84,162	Sài Gòn BikeMart	33	2,327,970,000	
116,158	HEAD Tân Long Vân	33	2,296,150,000	
14,667	Hệ Thống HEAD Dũ...	34	2,246,190,000	
167,589	Honda ủy nhiệm Ho...	25	1,697,087,000	
70,421	Shop1 Store	168	1,667,385,000	
1,431	Honda Sài Gòn	27	1,624,107,681	
282,701	ĐIỆN MÁY NGUYỄN ...	86	1,611,580,900	

Hình B.3: Danh sách cửa hàng có giá trị nhất

Danh sách các sản phẩm có nhiều đánh giá nhất.

Top 10 sản phẩm có nhiều đánh giá nhất			
Mã sản phẩm	Tên sản phẩm	Số đánh giá	
74,021,317	Cây Cam Ngọt Của Tôi	13,644	
71,345,381	Sách Thay Đổi Cuộc Sống Với Nhân Số Học - Lê Đỗ Quỳnh...	8,472	
299,461	Chuột Không Dây Logitech M331 Silent - Hàng Chính Hãng	8,353	
378,448	Nhà Giả Kim	7,650	
26,114,399	Nóng Giận Là Bản Năng , Tĩnh Lặng Là Bản Lĩnh	7,081	
106,863,963	Tâm Lý Học - Phác Họa Chân Dung Kẻ Phạm Tội	6,863	
54,748,899	Sách Muôn Kiếp Nhân Sinh (Bìa Mềm) - Nguyễn Phong	6,688	
36,626,940	Nồi Chiên Không Dầu Điện Tử Lock&Lock EJF357BLK (5.2...	6,673	
52,789,367	Nhà Giả Kim (Tái Bản 2020)	6,637	
94,173,462	Muôn Kiếp Nhân Sinh 2	6,478	

Hình B.4: Danh sách sản phẩm có đánh giá nhiều nhất

Danh sách sản phẩm có nhiều đánh giá 1 sao nhất

Top 10 sản phẩm có đánh giá 1 sao nhất			
Mã sản phẩm ▾	Tên sản phẩm	Phần trăm 1 sao ▾	Số đánh giá ▾
38,245,956	Giấy ăn gấu trúc siêu dai, an toàn khi...	6	327
21,921,714	Camera wifi Yoosee chuẩn 3 râu 11 ...	16	170
73,217,658	Chảo chống dính sâu lòng 18cm	28	167
91,552,224	Giấy ăn gấu trúc SiPiao chính hãng [...	38	163
36,626,940	Nồi Chiên Không Dầu Điện Tử Lock&...	2	153
389,145	Chuột không dây Logitech B175 - đầ...	2	150
14,963,349	Ổ cắm điện đa năng tích hợp cổng c...	33	149
299,461	Chuột Không Dây Logitech M331 Sil...	1	140
47,321,729	Pin Sạc Dự Phòng Xiaomi Redmi 200...	2	137
14,790,642	Tai nghe Bluetooth nhét tai không dâ...	19	131

Hình B.5: Danh sách sản phẩm có nhiều đánh giá 1 sao nhất